VULKAN

Explicit GPU programming from first principles

UNIT 4

Rendering and the Whole Frame

The graphics pipeline, presentation, compute meets graphics, and porting from OpenGL

4.1 The graphics pipeline

4.2 Presentation

4.3 Drawing with data

4.4 Compute meets graphics

4.5 From OpenGL to Vulkan

4.6 Scaling up

4.7 Capstone: a GPU-driven sandbox

First edition, October 2026

Contents

About this unit . . . 2
- Before you start . . . 2
- What this unit needs . . . 2

4.1 The graphics pipeline . . . 3
- Core ideas . . . 3
- Reading . . . 16
- Labs . . . 16
- Problem set . . . 18

4.2 Presentation . . . 19
- Core ideas . . . 19
- Reading . . . 28
- Labs . . . 28
- Problem set . . . 30

4.3 Drawing with data . . . 31
- Core ideas . . . 31
- Reading . . . 39
- Labs . . . 39
- Problem set . . . 40

4.4 Compute meets graphics . . . 41
- Core ideas . . . 41
- Reading . . . 50
- Labs . . . 50
- Problem set . . . 51

4.5 From OpenGL to Vulkan . . . 52
- Core ideas . . . 52
- Reading . . . 62
- Labs . . . 62
- Problem set . . . 64

4.6 Scaling up . . . 65
- Core ideas . . . 65
- Reading . . . 71
- Labs . . . 71
- Problem set . . . 73

4.7 Capstone: a GPU-driven sandbox . . . 74
- Core ideas . . . 74
- Reading . . . 82
- Labs . . . 83
- Problem set . . . 84

Sign-off . . . 85

Vulkan, First edition, October 2026. © 2026 O'Neill Group LLC. Vulkan and the Vulkan logo are registered trademarks of the Khronos Group Inc. OpenGL and the oval logo are trademarks or registered trademarks of Hewlett Packard Enterprise in the United States and other countries, used by permission by Khronos. All other trademarks belong to their owners. This course is not endorsed by the Khronos Group.

UNIT 4

About this unit

Unit 4 turns to rendering and to the frame as a whole. It builds the graphics pipeline from a triangle to a textured scene, presents frames through a swapchain, and joins compute and graphics in GPU-driven rendering. It ports a program from OpenGL and measures what Vulkan saves, scales recording across threads, and ends with a capstone that combines everything in the course: simulation, culling, async compute, frames in flight and a render graph, with every pass timed and synchronisation validation silent.

Chapter	You build	Hours
4.1	A triangle and a depth-tested cube, offscreen, checked against a CPU rasteriser	6
4.2	A presentation library and a spinning cube in a window and headless	6
4.3	Textured meshes with mipmaps, uniform buffers per frame and a bindless table	6
4.4	A million particles, GPU culling with indirect draws, and compute post-processing	7
4.5	An OpenGL program ported to Vulkan, with the CPU cost of each measured	6
4.6	Multithreaded recording, pipeline caches and memory budgets	6
4.7	The GPU-driven sandbox	10

Before you start

Finish Units 1 to 3. Install GLFW as the Handbook describes.

What this unit needs

The same machine as before, with a display for the windowed labs; every program also runs offscreen or headless.

CHAPTER 4.1

The graphics pipeline

TIME	6 hours
YOU BUILD	A triangle with interpolated colours, then a depth-tested cube in front of a wall, rendered offscreen with dynamic rendering, written to PNG files and checked pixel by pixel against a CPU rasteriser and a ray caster; and the same triangle drawn with a render pass and a framebuffer, as older code does.
NEEDS	Units 1 to 3. The vector and matrix algebra of 3D graphics.

WHY IT MATTERS

Every program so far has been a compute program, with one programmable stage fed by descriptors. Rendering adds a pipeline of fixed-function hardware around two programmable stages, and it is still most of what GPUs are built to do. The objects (pipelines, descriptor sets, push constants, barriers and layouts) are familiar, so this chapter concentrates on what is new: the stages a triangle passes through, the state each needs, the attachments it renders into, and the coordinate systems that put it on the screen. It renders offscreen and checks every result on the CPU, so that a rendering bug fails a test even when the picture looks almost right.

SKIP TEST

1. Name the stages a triangle passes through between the vertex buffer and the colour attachment, and say which are programmable.
2. What does a graphics pipeline fix when it is created, and what can you leave dynamic?
3. What are an attachment's load and store operations, and why is a depth attachment often stored with `VK_ATTACHMENT_STORE_OP_DONT_CARE`?
4. In Vulkan's normalised device coordinates, which way does +y point, and what is the range of depth?
5. What does dynamic rendering replace, and why will you still meet `VkRenderPass`?

If all five are easy, read the section on render passes and do Labs 4.1.3 and 4.1.4.

Core ideas

The pipeline, stage by stage

A draw call sends vertices through a fixed sequence of stages (Figure 4.1.1). *Vertex input* fetches each vertex's attributes from vertex buffers, as the pipeline's bindings and attributes describe. *Input assembly* groups vertices into primitives, here triangles. The *vertex shader* runs once per vertex and outputs its position in *clip space*, `gl_Position`, together with any values to pass on. Fixed hardware then clips primitives to the view, divides by the fourth coordinate, and maps the result through the *viewport* to pixels. The *rasteriser* determines which pixels each triangle covers, discarding triangles that face away if culling is on, and interpolates the vertex shader's outputs across the triangle. The *fragment shader* runs once per covered pixel and computes its colour. The *depth test* compares each fragment's depth with the depth attachment and discards hidden ones, and *colour blending* combines the result with what the colour attachment already holds.

The first program draws one triangle whose vertices carry a two-dimensional position and a colour, from a host-visible vertex buffer:

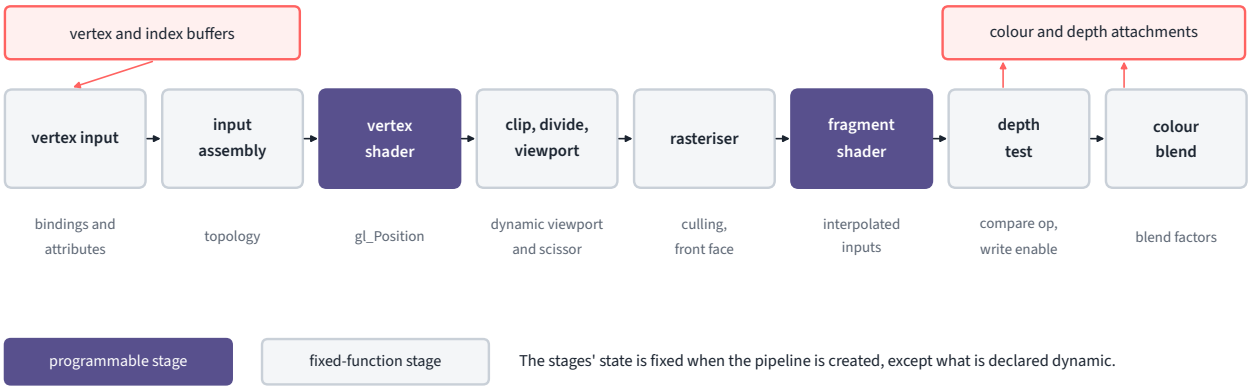


Figure 4.1.1. The graphics pipeline. Two stages run your shaders; the rest are fixed-function hardware configured by the pipeline's state.

Three vertices, each with a position and a colour — code/u4/triangle/main.cpp

```
struct TriangleVertex {
    float position[2];
    float color[3];
};

constexpr TriangleVertex Triangle[] = {
    {{0.0f, -0.7f}, {1.0f, 0.0f, 0.0f}},
    {{-0.7f, 0.6f}, {0.0f, 0.0f, 1.0f}},
    {{0.7f, 0.6f}, {0.0f, 1.0f, 0.0f}},
};
```

The pipeline's *vertex input state* describes how to fetch them: one *binding*, a buffer with a stride per vertex, and one *attribute* per shader input, with its location, format and offset within the vertex.

Vertex input: one binding and two attributes — code/u4/triangle/main.cpp

```
constexpr VkVertexInputBindingDescription TriangleBinding{
    .binding = 0,
    .stride = sizeof(TriangleVertex),
    .inputRate = VK_VERTEX_INPUT_RATE_VERTEX,
};
constexpr VkVertexInputAttributeDescription TriangleAttributes[] = {
    {.location = 0,
     .binding = 0,
     .format = VK_FORMAT_R32G32_SFLOAT,
     .offset = offsetof(TriangleVertex, position)},
    {.location = 1,
     .binding = 0,
     .format = VK_FORMAT_R32G32B32_SFLOAT,
     .offset = offsetof(TriangleVertex, color)},
};
```

The shaders read the attributes by location. The vertex shader passes the colour on, and the rasteriser interpolates it across the triangle for the fragment shader:

The vertex shader — code/u4/triangle/triangle.vert

```
#version 460

layout(location = 0) in vec2 inPosition;
layout(location = 1) in vec3 inColor;

layout(location = 0) out vec3 vertexColor;

void main() {
    gl_Position = vec4(inPosition, 0.0, 1.0);
    vertexColor = inColor;
}
```

The fragment shader — code/u4/triangle/triangle.frag

```
#version 460

layout(location = 0) in vec3 vertexColor;
layout(location = 0) out vec4 outColor;

void main() {
    outColor = vec4(vertexColor, 1.0);
}
```

Graphics pipeline state

A graphics pipeline bakes in everything about the stages that the hardware may compile into the shaders or configure ahead of time. The course's helper, `u4::createGraphicsPipeline`, fills the structures from a description, and each part maps to a stage of Figure 4.1.1. There are two shader stages:

The vertex and fragment stages — code/u4/shared/graphics.cpp

```
const VkPipelineShaderStageCreateInfo stages[] = {
    {.sType = VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO,
     .stage = VK_SHADER_STAGE_VERTEX_BIT,
     .module = desc.vertexShader,
     .pName = "main",
     .pSpecializationInfo = desc.specialization},
    {.sType = VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO,
     .stage = VK_SHADER_STAGE_FRAGMENT_BIT,
     .module = desc.fragmentShader,
     .pName = "main",
     .pSpecializationInfo = desc.specialization},
};
```

The vertex input and input assembly state describe the vertices, and the viewport, rasterisation and multisample state configure the rasteriser:

Vertex input and input assembly — code/u4/shared/graphics.cpp

```
const VkPipelineVertexInputStateCreateInfo vertexInput{
    .sType = VK_STRUCTURE_TYPE_PIPELINE_VERTEX_INPUT_STATE_CREATE_INFO,
    .vertexBindingDescriptionCount = static_cast<uint32_t>(desc.vertexBindings.size()),
    .pVertexBindingDescriptions = desc.vertexBindings.data(),
    .vertexAttributeDescriptionCount = static_cast<uint32_t>(desc.vertexAttributes.size()),
    .pVertexAttributeDescriptions = desc.vertexAttributes.data(),
};
const VkPipelineInputAssemblyStateCreateInfo inputAssembly{
    .sType = VK_STRUCTURE_TYPE_PIPELINE_INPUT_ASSEMBLY_STATE_CREATE_INFO,
    .topology = desc.topology,
};
```

One viewport and scissor, filled polygons, back-face culling, one sample per pixel — code/u4/shared/graphics.cpp

```
const VkPipelineViewportStateCreateInfo viewport{
    .sType = VK_STRUCTURE_TYPE_PIPELINE_VIEWPORT_STATE_CREATE_INFO,
    .viewportCount = 1,
    .scissorCount = 1,
};
const VkPipelineRasterizationStateCreateInfo rasterization{
    .sType = VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_STATE_CREATE_INFO,
    .polygonMode = VK_POLYGON_MODE_FILL,
    .cullMode = desc.cullMode,
    .frontFace = VK_FRONT_FACE_COUNTER_CLOCKWISE,
    .lineWidth = 1.0f,
};
const VkPipelineMultisampleStateCreateInfo multisample{
    .sType = VK_STRUCTURE_TYPE_PIPELINE_MULTISAMPLE_STATE_CREATE_INFO,
    .rasterizationSamples = VK_SAMPLE_COUNT_1_BIT,
};
```

`frontFace` says which winding order counts as facing the viewer, counter-clockwise here, and `cullMode` discards triangles facing away, which halves the work for closed meshes. The depth and blend state configure the last two stages:

The depth test, enabled when the pipeline has a depth attachment — code/u4/shared/graphics.cpp

```
const bool depth = desc.depthFormat != VK_FORMAT_UNDEFINED;
const VkPipelineDepthStencilStateCreateInfo depthStencil{
    .sType = VK_STRUCTURE_TYPE_PIPELINE_DEPTH_STENCIL_STATE_CREATE_INFO,
    .depthTestEnable = depth,
    .depthWriteEnable = depth && desc.depthWrite,
    .depthCompareOp = desc.depthCompare,
};
```

Colour blending: opaque, additive, or alpha — code/u4/shared/graphics.cpp

```
VkPipelineColorBlendAttachmentState attachment{
    .blendEnable = desc.blend != Blend::Opaque,
    .colorWriteMask = VK_COLOR_COMPONENT_R_BIT | VK_COLOR_COMPONENT_G_BIT |
                      VK_COLOR_COMPONENT_B_BIT | VK_COLOR_COMPONENT_A_BIT,
};
if (desc.blend == Blend::Additive) {
    attachment.srcColorBlendFactor = VK_BLEND_FACTOR_ONE;
    attachment.dstColorBlendFactor = VK_BLEND_FACTOR_ONE;
    attachment.srcAlphaBlendFactor = VK_BLEND_FACTOR_ONE;
    attachment.dstAlphaBlendFactor = VK_BLEND_FACTOR_ONE;
} else if (desc.blend == Blend::Alpha) {
    attachment.srcColorBlendFactor = VK_BLEND_FACTOR_SRC_ALPHA;
    attachment.dstColorBlendFactor = VK_BLEND_FACTOR_ONE_MINUS_SRC_ALPHA;
    attachment.srcAlphaBlendFactor = VK_BLEND_FACTOR_ONE;
    attachment.dstAlphaBlendFactor = VK_BLEND_FACTOR_ONE_MINUS_SRC_ALPHA;
}
const VkPipelineColorBlendStateCreateInfo blend{
    .sType = VK_STRUCTURE_TYPE_PIPELINE_COLOR_BLEND_STATE_CREATE_INFO,
    .attachmentCount = 1,
    .pAttachments = &attachment,
};
```

Dynamic state is the exception to baking everything in: state named in `VkPipelineDynamicStateCreateInfo` is set by commands in the command buffer instead. Every pipeline in the course leaves the viewport and scissor dynamic, so that one pipeline serves any size of image. Vulkan 1.3 allows much more state to be dynamic, such as the cull mode and the depth test.

Viewport and scissor, set at draw time — code/u4/shared/graphics.cpp

```
const VkDynamicState dynamicStates[] = {VK_DYNAMIC_STATE_VIEWPORT,
                                        VK_DYNAMIC_STATE_SCISSOR};
const VkPipelineDynamicStateCreateInfo dynamic{
    .sType = VK_STRUCTURE_TYPE_PIPELINE_DYNAMIC_STATE_CREATE_INFO,
    .dynamicStateCount = 2,
    .pDynamicStates = dynamicStates,
};
```

Finally, a pipeline used with *dynamic rendering* declares the formats of the attachments it will render into, in a `VkPipelineRenderingCreateInfo` chained into the create-info:

Creating the graphics pipeline — code/u4/shared/graphics.cpp

```
const VkPipelineRenderingCreateInfo rendering{
    .sType = VK_STRUCTURE_TYPE_PIPELINE_RENDERING_CREATE_INFO,
    .colorAttachmentCount = 1,
    .pColorAttachmentFormats = &desc.colorFormat,
    .depthAttachmentFormat = desc.depthFormat,
};
const VkGraphicsPipelineCreateInfo info{
    .sType = VK_STRUCTURE_TYPE_GRAPHICS_PIPELINE_CREATE_INFO,
    .pNext = desc.renderPass == VK_NULL_HANDLE ? &rendering : nullptr,
    .stageCount = 2,
    .pStages = stages,
    .pVertexInputState = &vertexInput,
    .pInputAssemblyState = &inputAssembly,
    .pViewportState = &viewport,
    .pRasterizationState = &rasterization,
    .pMultisampleState = &multisample,
    .pDepthStencilState = &depthStencil,
    .pColorBlendState = &blend,
    .pDynamicState = &dynamic,
    .layout = desc.layout,
    .renderPass = desc.renderPass,
};
VkPipeline pipeline = VK_NULL_HANDLE;
VKF_CHECK(
    vkCreateGraphicsPipelines(ctx.device(), desc.cache, 1, &info, nullptr, &pipeline));
```

> **COMING FROM OPENGL**
> OpenGL sets every piece of this state with separate calls (`glEnable(GL_DEPTH_TEST)`, `glBlendFunc`, `glCullFace`, `glVertexAttribPointer` and so on) at any time before a draw. The driver must then compile or patch shaders for the combination it finds at the draw, which is where OpenGL's unpredictable stalls come from. Vulkan asks for the combination up front, in the pipeline, and lets you name the few pieces of state that should vary freely.

Attachments and dynamic rendering

The images a draw renders into are *attachments*: a colour attachment, and optionally a depth attachment. The program creates both as optimally tiled images, with the usage flags that rendering and the readback copy need:

A colour attachment, a depth attachment, and a buffer to read the result into — code/u4/triangle/main.cpp

```
Target createTarget(const vkf::Context& ctx, VkExtent2D extent, VkFormat depthFormat) {
    Target target{.extent = extent};
    target.color = vkf::createImage(
        ctx,
        {.format = ColorFormat,
         .width = extent.width,
         .height = extent.height,
         .usage = VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT | VK_IMAGE_USAGE_TRANSFER_SRC_BIT},
        "colour attachment");
    if (depthFormat != VK_FORMAT_UNDEFINED) {
        target.depth = vkf::createImage(ctx,
                                        {.format = depthFormat,
                                         .width = extent.width,
                                         .height = extent.height,
                                         .usage = VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT,
                                         .aspect = VK_IMAGE_ASPECT_DEPTH_BIT},
                                        "depth attachment");
    }
    target.readback = vkf::createBuffer(ctx, VkDeviceSize(extent.width) * extent.height * 4,
                                        VK_BUFFER_USAGE_TRANSFER_DST_BIT,
                                        vkf::MemoryUse::Readback, "readback");
    return target;
}
```

Before rendering, both images move from `UNDEFINED` to their attachment layouts. The barriers name the stages that will touch them: colour attachment output for the colour image, and the early and late fragment tests for depth, which both read and write it.

Transitions into the attachment layouts — code/u4/triangle/main.cpp

```
void beginAttachments(VkCommandBuffer cmd, const Target& target) {
    const VkImageMemoryBarrier2 barriers[] = {
        {
            .sType = VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER_2,
            .srcStageMask = VK_PIPELINE_STAGE_2_NONE,
            .srcAccessMask = VK_ACCESS_2_NONE,
            .dstStageMask = VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT,
            .dstAccessMask = VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT,
            .oldLayout = VK_IMAGE_LAYOUT_UNDEFINED,
            .newLayout = VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL,
            .srcQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
            .dstQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
            .image = target.color,
            .subresourceRange = {VK_IMAGE_ASPECT_COLOR_BIT, 0, 1, 0, 1},
        },
        {
            .sType = VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER_2,
            .srcStageMask = VK_PIPELINE_STAGE_2_NONE,
            .srcAccessMask = VK_ACCESS_2_NONE,
            .dstStageMask = VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT |
                            VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT,
            .dstAccessMask = VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_READ_BIT |
                             VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT,
            .oldLayout = VK_IMAGE_LAYOUT_UNDEFINED,
            .newLayout = VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL,
            .srcQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
            .dstQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
            .image = target.depth,
            .subresourceRange = {VK_IMAGE_ASPECT_DEPTH_BIT, 0, 1, 0, 1},
        },
    };
    const VkDependencyInfo dependency{
        .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
        .imageMemoryBarrierCount = target.depth.image != VK_NULL_HANDLE ? 2u : 1u,
        .pImageMemoryBarriers = barriers,
    };
    vkCmdPipelineBarrier2(cmd, &dependency);
}
```

`vkCmdBeginRendering` starts rendering into them. Each attachment has a *load operation*, what happens to its contents when rendering starts, and a *store operation*, what happens when it ends. `VK_ATTACHMENT_LOAD_OP_CLEAR` fills it with a clear value, `LOAD` keeps its contents and `DONT_CARE` makes no promise. `STORE_OP_STORE` writes the results to memory and `DONT_CARE` lets them be discarded. The depth attachment is needed only during rendering, so its results are not stored. On tile-based GPUs, such as Apple's and most mobile GPUs, which render each region of the screen in on-chip memory, a discarded attachment need never be written to memory at all.

Clearing both attachments and keeping only the colour — code/u4/triangle/main.cpp

```
void beginRendering(VkCommandBuffer cmd, const Target& target) {
    const VkRenderingAttachmentInfo color{
        .sType = VK_STRUCTURE_TYPE_RENDERING_ATTACHMENT_INFO,
        .imageView = target.color.view,
        .imageLayout = VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL,
        .loadOp = VK_ATTACHMENT_LOAD_OP_CLEAR,
        .storeOp = VK_ATTACHMENT_STORE_OP_STORE,
        .clearValue = {.color = Background},
    };
    const VkRenderingAttachmentInfo depth{
        .sType = VK_STRUCTURE_TYPE_RENDERING_ATTACHMENT_INFO,
        .imageView = target.depth.view,
        .imageLayout = VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL,
        .loadOp = VK_ATTACHMENT_LOAD_OP_CLEAR,
        .storeOp = VK_ATTACHMENT_STORE_OP_DONT_CARE,
        .clearValue = {.depthStencil = {1.0f, 0}},
    };
    const VkRenderingInfo info{
        .sType = VK_STRUCTURE_TYPE_RENDERING_INFO,
        .renderArea = {{0, 0}, target.extent},
        .layerCount = 1,
        .colorAttachmentCount = 1,
        .pColorAttachments = &color,
        .pDepthAttachment = target.depth.view != VK_NULL_HANDLE ? &depth : nullptr,
    };
    vkCmdBeginRendering(cmd, &info);
}
```

Inside, the program binds the pipeline, sets the dynamic viewport and scissor, binds the vertex buffer and draws three vertices. `vkCmdDraw` takes a vertex count, an instance count, and the first vertex and instance:

Recording the triangle — code/u4/triangle/main.cpp

```
void renderTriangle(const vkf::Context& ctx, const Target& target, VkPipeline pipeline,
                    VkBuffer vertices) {
    vkf::submitNow(ctx, [&](VkCommandBuffer cmd) {
        beginAttachments(cmd, target);
        beginRendering(cmd, target);
        vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_GRAPHICS, pipeline);
        u4::setViewportAndScissor(cmd, target.extent);
        const VkDeviceSize offset = 0;
        vkCmdBindVertexBuffers(cmd, 0, 1, &vertices, &offset);
        vkCmdDraw(cmd, 3, 1, 0, 0);
        vkCmdEndRendering(cmd);
        endAttachments(cmd, target);
    });
}
```

After rendering, a barrier from colour attachment writes to transfer reads, with a transition to `TRANSFER_SRC_OPTIMAL`, lets a copy move the image into the readback buffer:

From the colour attachment to the host — code/u4/triangle/main.cpp

```
void copyToReadback(VkCommandBuffer cmd, const Target& target) {
    const VkBufferImageCopy region{
        .imageSubresource = {VK_IMAGE_ASPECT_COLOR_BIT, 0, 0, 1},
        .imageExtent = {target.extent.width, target.extent.height, 1},
    };
    vkCmdCopyImageToBuffer(cmd, target.color, VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL,
                           target.readback, 1, &region);
    vkf::bufferBarrier(cmd, target.readback, VK_PIPELINE_STAGE_2_COPY_BIT,
                       VK_ACCESS_2_TRANSFER_WRITE_BIT, VK_PIPELINE_STAGE_2_HOST_BIT,
                       VK_ACCESS_2_HOST_READ_BIT);
}

void endAttachments(VkCommandBuffer cmd, const Target& target) {
    vkf::imageBarrier(cmd, target.color, VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL,
                      VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL,
                      VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT,
                      VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT, VK_PIPELINE_STAGE_2_COPY_BIT,
                      VK_ACCESS_2_TRANSFER_READ_BIT);
    copyToReadback(cmd, target);
}
```

Checking a rendering

A rendering can be wrong in ways that nobody notices by looking at it. The program therefore checks its triangle against what the rasteriser must produce. For every pixel whose centre lies at least one pixel inside the triangle, it computes the *barycentric weights* of the centre, applies them to the three vertex colours, and requires the rendered colour to be within two levels of 8 bits. Every pixel at least one pixel outside the triangle must be exactly the background. Pixels on the edges are skipped, because which of them a triangle covers depends on the rasteriser's fill rules, which the check does not model.

The CPU's rasteriser: barycentric interpolation at every pixel centre — code/u4/triangle/main.cpp

```
bool checkTriangle(const std::vector<uint8_t>& pixels, VkExtent2D extent) {
    float x[3], y[3];
    for (int i = 0; i < 3; ++i) {
        x[i] = (Triangle[i].position[0] + 1) / 2 * float(extent.width);
        y[i] = (Triangle[i].position[1] + 1) / 2 * float(extent.height);
    }
    auto edge = [&](int a, int b, float px, float py) {
        return (x[b] - x[a]) * (py - y[a]) - (y[b] - y[a]) * (px - x[a]);
    };
    const float area = edge(0, 1, x[2], y[2]);
    const float side[3] = {std::hypot(x[2] - x[1], y[2] - y[1]),
                           std::hypot(x[0] - x[2], y[0] - y[2]),
                           std::hypot(x[1] - x[0], y[1] - y[0])};
    Tally inside, outside;
    for (uint32_t row = 0; row < extent.height; ++row) {
        for (uint32_t column = 0; column < extent.width; ++column) {
            const float px = float(column) + 0.5f, py = float(row) + 0.5f;
            const float weight[3] = {edge(1, 2, px, py) / area, edge(2, 0, px, py) / area,
                                     edge(0, 1, px, py) / area};
            float nearestEdge = std::numeric_limits<float>::infinity();
            Vec3 expected;
            for (int i = 0; i < 3; ++i) {
                nearestEdge = std::min(nearestEdge, weight[i] * std::abs(area) / side[i]);
                const float* c = Triangle[i].color;
                expected = expected + Vec3{c[0], c[1], c[2]} * weight[i];
            }
            const uint8_t* pixel = &pixels[(size_t(row) * extent.width + column) * 4];
            if (nearestEdge >= 1.0f) inside.add(difference(pixel, expected), 2);
            if (nearestEdge <= -1.0f) outside.add(difference(pixel, BackgroundColor), 1);
        }
    }
    vkf::print(
        "  inside: {} pixels, {} more than 2 from the interpolated colour (largest {})\n",
        inside.compared, inside.wrong, inside.worst);
    vkf::print("  outside: {} pixels, {} differ from the background\n", outside.compared,
```

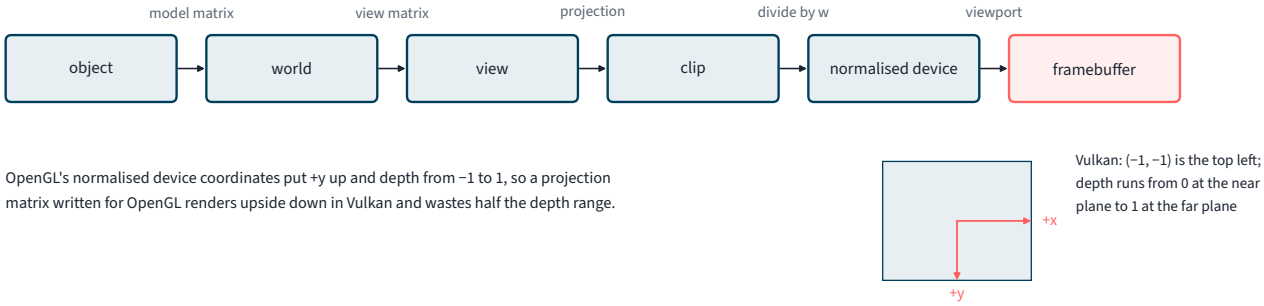


Figure 4.1.2. From an object's coordinates to the framebuffer. Vulkan's normalised device coordinates differ from OpenGL's in two ways that every projection matrix must respect.

```
                outside.wrong);
    return inside.compared > 1000 && inside.wrong == 0 && outside.wrong == 0;
}
```

Into three dimensions

A 3D scene needs a chain of transformations (Figure 4.1.2). A *model matrix* places an object in the world; a *view matrix* places the world in front of the camera; a *projection matrix* maps the camera's view volume to clip space, from which the fixed hardware divides by w and applies the viewport.

Vulkan's conventions differ from OpenGL's, and matrices written for one give wrong pictures in the other. In Vulkan's normalised device coordinates, +y points down the image, and depth runs from 0 at the near plane to 1 at the far plane. The course's projection matrix flips y and maps depth to that range:

A perspective projection for Vulkan's clip space — code/u4/shared/vecmath.hpp

```
// Vulkan's clip space: +y points down the image and depth runs from 0 (near) to 1 (far).
inline Mat4 perspective(float fovY, float aspect, float zNear, float zFar) {
    const float f = 1.0f / std::tan(fovY / 2);
    Mat4 r;
    r.m[0][0] = f / aspect;
    r.m[1][1] = -f;
    r.m[2][2] = zFar / (zNear - zFar);
    r.m[2][3] = -1;
    r.m[3][2] = zNear * zFar / (zNear - zFar);
    r.m[3][3] = 0;
    return r;
}
```

The view matrix of a camera looking at a point — code/u4/shared/vecmath.hpp

```
// A right-handed camera at eye looking at target: it looks down its own -z axis with +y up.
inline Mat4 lookAt(Vec3 eye, Vec3 target, Vec3 up) {
    const Vec3 f = normalize(target - eye);
    const Vec3 s = normalize(cross(f, up));
    const Vec3 u = cross(s, f);
    Mat4 r;
    r.m[0][0] = s.x;
    r.m[1][0] = s.y;
    r.m[2][0] = s.z;
    r.m[0][1] = u.x;
    r.m[1][1] = u.y;
    r.m[2][1] = u.z;
    r.m[0][2] = -f.x;
    r.m[1][2] = -f.y;
    r.m[2][2] = -f.z;
    r.m[3][0] = -dot(s, eye);
    r.m[3][1] = -dot(u, eye);
    r.m[3][2] = dot(f, eye);
    return r;
}
```

The scene is a cube in front of a wall, and the wall is the same cube mesh, scaled flat. The cube's vertices carry a position and a normal, and each face has its own four vertices so that its normal is constant across it:

A cube with a separate quadrilateral for each face — code/u4/shared/mesh.hpp

```
// Four vertices per face, for per-face normals; front faces wind counter-clockwise.
inline Mesh cubeMesh() {
    struct Face {
        Vec3 normal, right, up;
    };
    const Face faces[] = {
        {{1, 0, 0}, {0, 0, -1}, {0, 1, 0}}, {{-1, 0, 0}, {0, 0, 1}, {0, 1, 0}},
        {{0, 1, 0}, {1, 0, 0}, {0, 0, -1}}, {{0, -1, 0}, {1, 0, 0}, {0, 0, 1}},
        {{0, 0, 1}, {1, 0, 0}, {0, 1, 0}},  {{0, 0, -1}, {-1, 0, 0}, {0, 1, 0}},
    };
    Mesh mesh;
    for (const Face& f : faces) {
        const auto first = static_cast<uint16_t>(mesh.vertices.size());
        mesh.vertices.push_back({f.normal - f.right - f.up, f.normal, 0, 1});
        mesh.vertices.push_back({f.normal + f.right - f.up, f.normal, 1, 1});
        mesh.vertices.push_back({f.normal + f.right + f.up, f.normal, 1, 0});
        mesh.vertices.push_back({f.normal - f.right + f.up, f.normal, 0, 0});
        for (int corner : {0, 1, 2, 0, 2, 3}) {
            mesh.indices.push_back(static_cast<uint16_t>(first + corner));
        }
    }
    return mesh;
}
```

Each box gets its own model matrix and tint, and the product of the camera's view and projection with the model matrix reaches the vertex shader as a push constant, written before each draw. The draws are *indexed*: an index buffer lists the vertices of each triangle, so that the four vertices of a face serve its two triangles.

One indexed draw per box, with its matrix in push constants — code/u4/triangle/main.cpp

```
void renderBoxes(const vkf::Context& ctx, const Target& target, VkPipeline pipeline,
                 VkPipelineLayout layout, const Mesh& cube, std::span<const Box> boxes,
                 const Camera& camera) {
    const Mat4 viewProjection = camera.viewProjection(target.extent);
    vkf::submitNow(ctx, [&](VkCommandBuffer cmd) {
        beginAttachments(cmd, target);
        beginRendering(cmd, target);
        vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_GRAPHICS, pipeline);
        u4::setViewportAndScissor(cmd, target.extent);
        const VkDeviceSize offset = 0;
        vkCmdBindVertexBuffers(cmd, 0, 1, &cube.vertices.buffer, &offset);
        vkCmdBindIndexBuffer(cmd, cube.indices, 0, VK_INDEX_TYPE_UINT16);
        for (const Box& box : boxes) {
            const DrawConstants constants{viewProjection * box.model,
                                          {box.tint.x, box.tint.y, box.tint.z, 1.0f}};
            vkCmdPushConstants(cmd, layout, VK_SHADER_STAGE_VERTEX_BIT, 0, sizeof(constants),
                               &constants);
            vkCmdDrawIndexed(cmd, cube.indexCount, 1, 0, 0, 0);
        }
        vkCmdEndRendering(cmd);
        endAttachments(cmd, target);
    });
}
```

The cube's vertex shader colours each face by its normal — code/u4/triangle/cube.vert

```
#version 460

layout(location = 0) in vec3 inPosition;
layout(location = 1) in vec3 inNormal;

layout(push_constant) uniform DrawConstants {
    mat4 mvp;
    vec4 tint;
} draw;

layout(location = 0) out vec3 faceColor;

void main() {
    gl_Position = draw.mvp * vec4(inPosition, 1.0);
    faceColor = (inNormal * 0.5 + 0.5) * draw.tint.rgb;
}
```

The depth test

The cube stands in front of the wall, but the program draws the cube first. Without a depth test the wall, drawn second, would cover it. With one, each fragment's depth is compared with the depth already stored at its pixel, and only a nearer fragment is drawn and its depth stored. The depth attachment's format is the first of three common formats that the device supports for depth attachments:

Choosing a depth format the device can render into — code/u4/shared/graphics.cpp

```
VkFormat chooseDepthFormat(const vkf::Context& ctx) {
    const VkFormat candidates[] = {VK_FORMAT_D32_SFLOAT, VK_FORMAT_X8_D24_UNORM_PACK32,
                                   VK_FORMAT_D16_UNORM};
    for (VkFormat format : candidates) {
        VkFormatProperties properties;
        vkGetPhysicalDeviceFormatProperties(ctx.physicalDevice(), format, &properties);
        if (properties.optimalTilingFeatures &
            VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT) {
            return format;
        }
    }
    throw vkf::Error(VK_ERROR_FORMAT_NOT_SUPPORTED, "no depth format is renderable");
}
```

To check the cube, the program casts a ray through the centre of every fourth pixel and finds the nearest face it hits, by intersecting the ray with each box in the box's own coordinates. Where a pixel and its four neighbours see the same face, the rendered colour must be that face's colour. Where the ray passes through the cube and on to the wall, the pixel must show the cube, which tests the depth test.

The ray caster that checks the rasteriser — code/u4/triangle/main.cpp

```
// Faces are numbered +x, -x, +y, -y, +z, -z.
Hit castRay(Vec3 origin, Vec3 direction, std::span<const Box> boxes) {
    Hit nearest;
    for (size_t i = 0; i < boxes.size(); ++i) {
        const Mat4 toBox = u4::inverseAffine(boxes[i].model);
        const Vec3 o = u4::transformPoint(toBox, origin);
        const Vec3 d = u4::transformDirection(toBox, direction);
        const float os[3] = {o.x, o.y, o.z}, ds[3] = {d.x, d.y, d.z};
        float enter = -std::numeric_limits<float>::infinity();
        float leave = std::numeric_limits<float>::infinity();
        int face = -1;
        for (int axis = 0; axis < 3; ++axis) {
            const float t0 = (-1 - os[axis]) / ds[axis], t1 = (1 - os[axis]) / ds[axis];
            if (std::min(t0, t1) > enter) {
                enter = std::min(t0, t1);
                face = axis * 2 + (t0 < t1 ? 1 : 0);
            }
            leave = std::min(leave, std::max(t0, t1));
        }
```

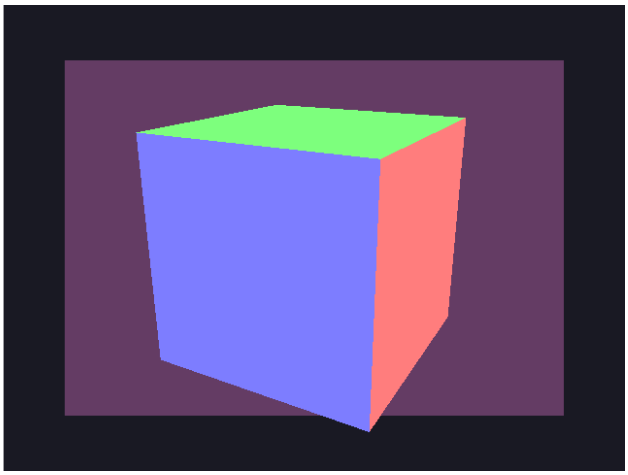



Figure 4.1.3. The two images the program wrote, `triangle.png` and `cube.png`.

```
        if (enter > leave || enter <= 0) continue;
        ++nearest.boxesHit;
        if (enter < nearest.t) nearest = {int(i), face, nearest.boxesHit, enter};
    }
    return nearest;
}
```

The triangle and the cube, both checked, on the M2 Pro

```
$ ./build/bin/u4_triangle
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
triangle: 640x480 VK_FORMAT_R8G8B8A8_UNORM, wrote triangle.png
  inside: 68678 pixels, 0 more than 2 from the interpolated colour (largest 0)
  outside: 236090 pixels, 0 differ from the background
cube: 640x480, depth VK_FORMAT_D32_SFLOAT, wrote cube.png
  centre (320, 240) is (128, 128, 255), face +z of the cube: as expected
  corners: background
  ray cast: 18771 sampled pixels compared, 0 differ (429 near edges skipped)
  depth order: 5422 of them see the wall behind the cube, 0 show the wall
PASS triangle and depth-tested cube
```

Render passes, for reading older code

Before Vulkan 1.3 made dynamic rendering core, every draw happened inside a *render pass*: an object created in advance that lists the attachments with their formats, load and store operations and layouts, divides rendering into *subpasses*, and declares the dependencies between them and the outside. A *framebuffer* binds actual image views to a render pass's attachments, and every pipeline must be created for a particular render pass. Most existing Vulkan code, and many tutorials, still use them, so the program can draw the triangle that way too:

A render pass with one colour attachment and one subpass — code/u4/triangle/main.cpp

```
VkRenderPass createRenderPass(const vkf::Context& ctx) {
    const VkAttachmentDescription color{
        .format = ColorFormat,
        .samples = VK_SAMPLE_COUNT_1_BIT,
        .loadOp = VK_ATTACHMENT_LOAD_OP_CLEAR,
        .storeOp = VK_ATTACHMENT_STORE_OP_STORE,
        .stencilLoadOp = VK_ATTACHMENT_LOAD_OP_DONT_CARE,
        .stencilStoreOp = VK_ATTACHMENT_STORE_OP_DONT_CARE,
        .initialLayout = VK_IMAGE_LAYOUT_UNDEFINED,
        .finalLayout = VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL,
    };
    const VkAttachmentReference colorReference{0, VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL};
    const VkSubpassDescription subpass{
        .pipelineBindPoint = VK_PIPELINE_BIND_POINT_GRAPHICS,
        .colorAttachmentCount = 1,
        .pColorAttachments = &colorReference,
    };
```

Subpass dependencies: the barriers into and out of the render pass, declared in advance — code/u4/triangle/main.cpp

```
    const VkSubpassDependency dependencies[] = {
        {.srcSubpass = VK_SUBPASS_EXTERNAL,
         .dstSubpass = 0,
         .srcStageMask = VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT,
         .dstStageMask = VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT,
         .srcAccessMask = 0,
         .dstAccessMask = VK_ACCESS_COLOR_ATTACHMENT_WRITE_BIT},
        {.srcSubpass = 0,
         .dstSubpass = VK_SUBPASS_EXTERNAL,
         .srcStageMask = VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT,
         .dstStageMask = VK_PIPELINE_STAGE_TRANSFER_BIT,
         .srcAccessMask = VK_ACCESS_COLOR_ATTACHMENT_WRITE_BIT,
         .dstAccessMask = VK_ACCESS_TRANSFER_READ_BIT},
    };
    const VkRenderPassCreateInfo info{
        .sType = VK_STRUCTURE_TYPE_RENDER_PASS_CREATE_INFO,
        .attachmentCount = 1,
        .pAttachments = &color,
        .subpassCount = 1,
        .pSubpasses = &subpass,
        .dependencyCount = 2,
        .pDependencies = dependencies,
    };
    VkRenderPass renderPass = VK_NULL_HANDLE;
    VKF_CHECK(vkCreateRenderPass(ctx.device(), &info, nullptr, &renderPass));
    return renderPass;
}
```

A framebuffer binds the image view to the attachment — code/u4/triangle/main.cpp

```
const VkFramebufferCreateInfo framebufferInfo{
    .sType = VK_STRUCTURE_TYPE_FRAMEBUFFER_CREATE_INFO,
    .renderPass = renderPass,
    .attachmentCount = 1,
    .pAttachments = &target.color.view,
    .width = target.extent.width,
    .height = target.extent.height,
    .layers = 1,
};
VkFramebuffer framebuffer = VK_NULL_HANDLE;
VKF_CHECK(vkCreateFramebuffer(ctx.device(), &framebufferInfo, nullptr, &framebuffer));
```

Recording the same triangle inside the render pass `code/u4/triangle/main.cpp`

```
const VkClearValue clear{.color = Background};
const VkRenderPassBeginInfo begin{
    .sType = VK_STRUCTURE_TYPE_RENDER_PASS_BEGIN_INFO,
    .renderPass = renderPass,
    .framebuffer = framebuffer,
    .renderArea = {{0, 0}, target.extent},
    .clearValueCount = 1,
    .pClearValues = &clear,
};
vkCmdBeginRenderPass(cmd, &begin, VK_SUBPASS_CONTENTS_INLINE);
vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_GRAPHICS, pipeline);
u4::setViewportAndScissor(cmd, target.extent);
const VkDeviceSize offset = 0;
vkCmdBindVertexBuffers(cmd, 0, 1, &vertices, &offset);
vkCmdDraw(cmd, 3, 1, 0, 0);
vkCmdEndRenderPass(cmd);
copyToReadback(cmd, target);
```

The render-pass version produces an identical image

```
$ ./build/bin/u4_triangle --render-pass
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
triangle: 640x480 VK_FORMAT_R8G8B8A8_UNORM, wrote triangle.png
  inside: 68678 pixels, 0 more than 2 from the interpolated colour (largest 0)
  outside: 236090 pixels, 0 differ from the background
triangle with VkRenderPass and VkFramebuffer: identical to dynamic rendering
cube: 640x480, depth VK_FORMAT_D32_SFLOAT, wrote cube.png
  centre (320, 240) is (128, 128, 255), face +z of the cube: as expected
  corners: background
  ray cast: 18771 sampled pixels compared, 0 differ (429 near edges skipped)
  depth order: 5422 of them see the wall behind the cube, 0 show the wall
PASS triangle and depth-tested cube
```

Render passes let the driver plan the use of tile memory across subpasses, which matters on tile-based GPUs, at the cost of objects that must match each other exactly. Dynamic rendering gives up that planning and is simpler to use. Vulkan 1.4's local-read feature lets a draw read, from tile memory, attachments that earlier draws in the same rendering wrote, which recovers most of what subpasses offered.

Reading

- The Vulkan specification, chapters “Graphics Pipelines” (in “Pipelines”), “Fixed-Function Vertex Processing”, “Rasterization”, “Fragment Operations”, “The Framebuffer” and “Render Pass”.
- The Vulkan Guide, “Pipeline Dynamic State”, “Primitive Topology”, “Depth” and “Vertex Input Data Processing”.
- Any treatment of 3D transformations, for example chapter 2 of Pharr, Jakob and Humphreys, *Physically Based Rendering*, with Vulkan’s clip-space conventions in mind.

Labs

LAB 4.1.1 Read the triangle’s pixels — 1 hour

Goal. Run the first rendering and understand its check.

Code. `code/u4/triangle`

1. Run `build/bin/u4_triangle` and open `triangle.png` and `cube.png`.
2. Change the second vertex’s colour to white and confirm that the check still passes, then change the CPU’s copy of the colour only and confirm that it fails.
3. Explain why the check skips the pixels on the triangle’s edges.

DONE WHEN

- [ ] The program passes with your new colour and fails when the CPU's expectation is wrong.
- [ ] Your explanation names the fill rules.

EVIDENCE FOR THE NOTEBOOK

- Both outputs and the explanation.

LAB 4.1.2 State you can change — 1.5 hours

Goal. See which state the pipeline fixes and which it does not.

Code. `code/u4/triangle`

1. Reverse the triangle's winding order with back-face culling on, and record what is drawn.
2. Make the cull mode dynamic with `VK_DYNAMIC_STATE_CULL_MODE` and set it per draw with `vkCmdSetCullMode`.
3. Render the cube with `VK_POLYGON_MODE_LINE`, after checking that `fillModeNonSolid` is enabled.

DONE WHEN

- [ ] Each variant renders as you predicted, with validation silent.
- [ ] You can say which of the three changes needed a new pipeline and which did not.

EVIDENCE FOR THE NOTEBOOK

- The three images and your notes.

LAB 4.1.3 Break the depth test — 1.5 hours

Goal. Connect the depth state to what the image shows.

Code. `code/u4/triangle`

1. Disable the depth test and run: record which checks fail and why.
2. Restore it, change the compare operation to `VK_COMPARE_OP_GREATER` and the clear value to 0, and explain the result.
3. Use an OpenGL-style projection matrix, with depth from −1 to 1 and y up, and describe what happens to the image.

DONE WHEN

- [ ] You have the output of all three runs.
- [ ] Each explanation refers to the stage and the state involved.

EVIDENCE FOR THE NOTEBOOK

- The images and explanations.

LAB 4.1.4 Render passes and dynamic rendering — 2 hours

Goal. Translate between render passes and dynamic rendering.

Code. `code/u4/triangle`

1. Run with `--render-pass` and confirm the images match.
2. Extend the render-pass version to draw the cube with its depth attachment: a second attachment, its reference in the subpass, and its dependencies.
3. List every piece of information the render pass holds that dynamic rendering supplies at `vkCmdBeginRendering` instead.

DONE WHEN

- [ ] The render-pass cube matches the dynamic-rendering cube exactly.
- [ ] Your list covers attachments, layouts, load and store operations, and dependencies.

EVIDENCE FOR THE NOTEBOOK

- The extended code and the list.

Problem set

1. A vertex has a position of three floats, a normal of three floats and a texture coordinate of two. Write its binding and attribute descriptions.
2. Why does the course's projection matrix negate the y scale? Show what happens to a point at the top of the view without it.
3. The depth attachment is cleared to 1.0 and compared with `LESS`. What would a clear value of 0.0 do? Why is depth often stored reversed, with 1 at the near plane?
4. Explain why the triangle check compares only pixels at least one pixel from an edge, and how a check of edge pixels would have to work.
5. Which of the render pass's two subpass dependencies corresponds to which of the dynamic-rendering version's barriers?

COMPLETION CHECKLIST

- [ ] I can name every stage of the graphics pipeline and the state that configures it.
- [ ] I can create a graphics pipeline with vertex input, depth testing, blending and dynamic state.
- [ ] I can render offscreen with dynamic rendering, with the right attachments, layouts, load and store operations and barriers.
- [ ] I can transform a scene into Vulkan's clip space, and read code that uses render passes.
- [ ] Lab 4.1.1–4.1.4 done-when criteria all hold, with evidence filed.

CHAPTER 4.2

Presentation

TIME	6 hours
YOU BUILD	A small presentation library, a window or a headless surface, a swapchain that is recreated when the window changes, and frames in flight; and with it a spinning cube that runs in a window, or headlessly for tests, through a forced resize, with validation silent.
NEEDS	Chapter 4.1 and Chapter 3.6. GLFW.

WHY IT MATTERS

Rendering into an image you created yourself is only part of putting pictures on a screen. The screen belongs to the window system, which owns the images that are shown, decides when each is shown, and can take them away when the window changes size. Vulkan's *window system integration* makes all of that explicit: a surface, a swapchain of images that the presentation engine lends you one at a time, semaphores that order rendering against presentation, and a protocol for when the swapchain no longer fits the window. This chapter builds it once, carefully, so that the rest of the unit only has to ask for the next image.

SKIP TEST

1. What do the FIFO and mailbox present modes do, and which is always available?
2. Why does the course keep one acquire semaphore per frame in flight but one ready-to-present semaphore per swapchain image?
3. What must a program do when `vkAcquireNextImageKHR` returns `VK_ERROR_OUT_OF_DATE_KHR`, and when it returns `VK_SUBOPTIMAL_KHR`?
4. An acquired image is in `VK_IMAGE_LAYOUT_UNDEFINED`. Write the barrier that prepares it for rendering, and explain its source stage.

If all four are easy, read the section on semaphores and do Labs 4.2.2 and 4.2.3.

Core ideas

Surfaces

A *surface*, `VkSurfaceKHR`, is Vulkan's handle to something that can display images: a window, or a whole display. Surfaces come from instance extensions: `VK_KHR_surface` and one per window system, such as `VK_KHR_win32_surface`, `VK_KHR_xcb_surface`, `VK_KHR_wayland_surface` or, on macOS, `VK_EXT_metal_surface`. GLFW knows which extensions the platform needs and creates the surface for a window. The window must be created with `GLFW_NO_API`, which tells GLFW not to create an OpenGL context for it.

A window for Vulkan — code/u4/present/display.cpp

```
Display::Display(const DisplayOptions& options) : headless_(options.headless) {
    if (headless_) {
        headlessExtent_ = {options.width, options.height};
        return;
    }
    glfwSetErrorCallback(onGlfwError);
#if GLFW_VERSION_MAJOR > 3 || GLFW_VERSION_MINOR >= 4
    // GLFW would otherwise look for a Vulkan loader of its own, which may not be on its path.
```

```
    glfwInitVulkanLoader(vkGetInstanceProcAddr);
#endif
    if (!glfwInit()) throw std::runtime_error("cannot open a window here; try --headless");
    if (!glfwVulkanSupported()) {
        glfwTerminate();
        throw std::runtime_error("GLFW cannot create Vulkan surfaces on this system");
    }
    glfwWindowHint(GLFW_CLIENT_API, GLFW_NO_API);
    window_ =
        glfwCreateWindow(static_cast<int>(options.width), static_cast<int>(options.height),
                         options.title.c_str(), nullptr, nullptr);
    if (window_ == nullptr) {
        glfwTerminate();
        throw std::runtime_error("glfwCreateWindow failed");
    }
    glfwSetWindowUserPointer(window_, this);
    glfwSetFramebufferSizeCallback(window_, onFramebufferResize);
}
```

For tests and for machines without a display, `VK_EXT_headless_surface` creates a surface whose images go nowhere. Everything else works as with a window, which makes it the right tool for checking a presentation loop automatically:

The instance extensions and the surface, from GLFW or headless — code/u4/present/display.cpp

```
std::vector<const char*> Display::instanceExtensions() const {
    if (headless_) {
        return {VK_KHR_SURFACE_EXTENSION_NAME, VK_EXT_HEADLESS_SURFACE_EXTENSION_NAME};
    }
    uint32_t count = 0;
    const char** names = glfwGetRequiredInstanceExtensions(&count);
    return {names, names + count};
}

VkSurfaceKHR Display::createSurface(VkInstance instance) const {
    VkSurfaceKHR surface = VK_NULL_HANDLE;
    if (headless_) {
        const auto create = reinterpret_cast<PFN_vkCreateHeadlessSurfaceEXT>(
            vkGetInstanceProcAddr(instance, "vkCreateHeadlessSurfaceEXT"));
        const VkHeadlessSurfaceCreateInfoEXT info{
            .sType = VK_STRUCTURE_TYPE_HEADLESS_SURFACE_CREATE_INFO_EXT,
        };
        VKF_CHECK(create(instance, &info, nullptr, &surface));
    } else {
        VKF_CHECK(glfwCreateWindowSurface(instance, window_, nullptr, &surface));
    }
    return surface;
}
```

`vkf::Context` accepts the extensions and a function that creates the surface, and chooses a queue family that can present to it, which it checks with `vkGetPhysicalDeviceSurfaceSupportKHR`. The window must outlive the context, which destroys the surface:

A display, a context that presents to it, and a presenter — code/u4/spin/main.cpp

```
u4::Display display(
    {.title = "u4_spin", .width = width, .height = height, .headless = headless});
vkf::Context ctx({
    .appName = "u4_spin",
    .instanceExtensions = display.instanceExtensions(),
    .createSurface =
        [&](VkInstance instance) { return display.createSurface(instance); },
});
u4::Presenter presenter(
    ctx, display,
    {.swapchain = {.presentMode = wantedMode,
                   .imageCount = static_cast<uint32_t>(args.integer("--images", 3)),
                   .optionalUsage = VK_IMAGE_USAGE_TRANSFER_SRC_BIT},
     .framesInFlight = static_cast<uint32_t>(args.integer("--frames-in-flight", 2))});
```

Swapchains

A *swapchain* is a set of images that the surface's *presentation engine* owns and lends to the program one at a time (Figure 4.2.1). Creating one starts with asking the surface what it supports: its capabilities, which include the allowed numbers of images and sizes, its formats with their colour spaces, and its present modes.

What the surface supports — code/u4/present/swapchain.cpp

```
VkSurfaceCapabilitiesKHR caps;
VKF_CHECK(vkGetPhysicalDeviceSurfaceCapabilitiesKHR(physical, surface, &caps));
uint32_t count = 0;
VKF_CHECK(vkGetPhysicalDeviceSurfaceFormatsKHR(physical, surface, &count, nullptr));
std::vector<VkSurfaceFormatKHR> formats(count);
VKF_CHECK(vkGetPhysicalDeviceSurfaceFormatsKHR(physical, surface, &count, formats.data()));
VKF_CHECK(vkGetPhysicalDeviceSurfacePresentModesKHR(physical, surface, &count, nullptr));
std::vector<VkPresentModeKHR> modes(count);
VKF_CHECK(
    vkGetPhysicalDeviceSurfacePresentModesKHR(physical, surface, &count, modes.data()));
```

The *format* is usually a four-channel, 8-bit one. The course prefers an `_SRGB` format in the `VK_COLOR_SPACE_SRGB_NONLINEAR_KHR` colour space, so that shaders output linear colours and the hardware applies the sRGB encoding that displays expect when it writes them:

Choosing an sRGB format — code/u4/present/swapchain.cpp

```
VkSurfaceFormatKHR chooseSurfaceFormat(const std::vector<VkSurfaceFormatKHR>& formats,
                                       bool srgb) {
    const VkFormat preferred[] = {
        srgb ? VK_FORMAT_B8G8R8A8_SRGB : VK_FORMAT_B8G8R8A8_UNORM,
        srgb ? VK_FORMAT_R8G8B8A8_SRGB : VK_FORMAT_R8G8B8A8_UNORM,
    };
    for (VkFormat format : preferred) {
        for (const VkSurfaceFormatKHR& candidate : formats) {
            if (candidate.format == format &&
                candidate.colorSpace == VK_COLOR_SPACE_SRGB_NONLINEAR_KHR) {
                return candidate;
            }
        }
    }
    return formats.front();
}
```

The *present mode* decides how presented images reach the screen. `VK_PRESENT_MODE_FIFO_KHR` queues them and shows one per refresh of the display, which is vertical synchronisation. It is the only mode every implementation must support. `MAILBOX` keeps only the newest presented image waiting, replacing older ones, which lowers latency without tearing. `IMMEDIATE` shows each image at once, even mid-refresh, which tears. `FIFO_RELAXED` is FIFO that tears instead of waiting when a frame is late.

The requested mode if supported, otherwise FIFO — code/u4/present/swapchain.cpp

```
VkPresentModeKHR choosePresentMode(const std::vector<VkPresentModeKHR>& modes,
                                   VkPresentModeKHR wanted) {
    if (std::find(modes.begin(), modes.end(), wanted) != modes.end()) return wanted;
    return VK_PRESENT_MODE_FIFO_KHR;
}
```

The image count must lie between the surface's minimum and maximum, where a maximum of zero means no limit. Three images let the program render one while one waits to be shown and one is on screen. The extent is the surface's current size, except on platforms that let the swapchain choose:

Image count and extent within the surface's limits — code/u4/present/swapchain.cpp

```
uint32_t chooseImageCount(const VkSurfaceCapabilitiesKHR& caps, uint32_t wanted) {
    uint32_t count = std::max(wanted, caps.minImageCount);
    if (caps.maxImageCount > 0) count = std::min(count, caps.maxImageCount);
    return count;
}

VkExtent2D chooseExtent(const VkSurfaceCapabilitiesKHR& caps, VkExtent2D framebuffer) {
    if (caps.currentExtent.width != UINT32_MAX) return caps.currentExtent;
    return {
        std::clamp(framebuffer.width, caps.minImageExtent.width, caps.maxImageExtent.width),
        std::clamp(framebuffer.height, caps.minImageExtent.height, caps.maxImageExtent.height),
    };
}
```

Creating the swapchain, replacing any old one — code/u4/present/swapchain.cpp

```
const VkSwapchainCreateInfoKHR info{
    .sType = VK_STRUCTURE_TYPE_SWAPCHAIN_CREATE_INFO_KHR,
    .surface = surface,
    .minImageCount = chooseImageCount(caps, options_.imageCount),
    .imageFormat = surfaceFormat_.format,
    .imageColorSpace = surfaceFormat_.colorSpace,
    .imageExtent = extent_,
    .imageArrayLayers = 1,
    .imageUsage = usage_,
    .imageSharingMode = VK_SHARING_MODE_EXCLUSIVE,
    .preTransform = caps.currentTransform,
    .compositeAlpha = alpha,
    .presentMode = presentMode_,
    .clipped = VK_TRUE,
    .oldSwapchain = swapchain_,
};
VkSwapchainKHR created = VK_NULL_HANDLE;
VKF_CHECK(vkCreateSwapchainKHR(device, &info, nullptr, &created));
destroyPerImage();
if (swapchain_ != VK_NULL_HANDLE) vkDestroySwapchainKHR(device, swapchain_, nullptr);
swapchain_ = created;
```

Passing the current swapchain as `oldSwapchain` lets the presentation engine hand over to the new one smoothly. Finally the program fetches the images, creates a view of each, and creates one semaphore per image for presentation, for reasons the next section gives:

The images, a view of each, and a semaphore for each — code/u4/present/swapchain.cpp

```
VKF_CHECK(vkGetSwapchainImagesKHR(device, swapchain_, &count, nullptr));
images_.resize(count);
VKF_CHECK(vkGetSwapchainImagesKHR(device, swapchain_, &count, images_.data()));
const VkImageUsageFlags viewable = VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT |
                                   VK_IMAGE_USAGE_SAMPLED_BIT | VK_IMAGE_USAGE_STORAGE_BIT;
for (uint32_t i = 0; i < count; ++i) {
    const VkImageViewCreateInfo viewInfo{
        .sType = VK_STRUCTURE_TYPE_IMAGE_VIEW_CREATE_INFO,
        .image = images_[i],
        .viewType = VK_IMAGE_VIEW_TYPE_2D,
        .format = surfaceFormat_.format,
        .subresourceRange = vkf::colorRange(),
    };
    VkImageView view = VK_NULL_HANDLE;
    if (usage_ & viewable) VKF_CHECK(vkCreateImageView(device, &viewInfo, nullptr, &view));
    views_.push_back(view);
    readyToPresent_.push_back(vkf::createSemaphore(*ctx_).release());
    const std::string name = "swapchain image " + std::to_string(i);
    ctx_->name(images_[i], name.c_str());
    ctx_->name(view, name.c_str());
    ctx_->name(readyToPresent_.back(), ("ready to present " + std::to_string(i)).c_str());
}
```

Acquire, render, present

A frame has three steps. `vkAcquireNextImageKHR` returns the index of the next image the program may use and signals a semaphore when the presentation engine has finished with the image, which may be after the call returns. The frame's submission waits for that semaphore before writing the image, and signals a second semaphore when it has finished. `vkQueuePresentKHR` waits for the second semaphore and gives the image back to the presentation engine.

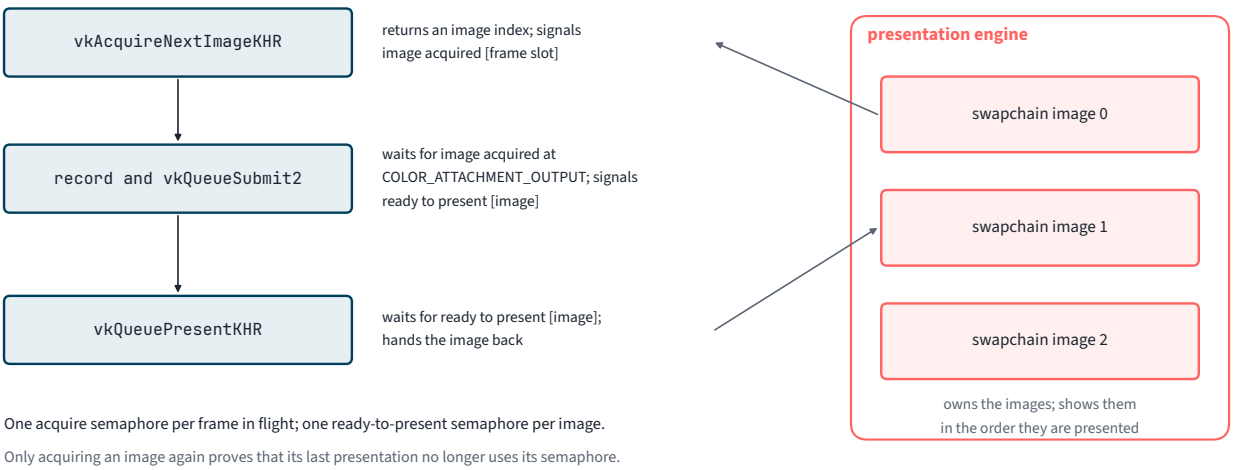


Figure 4.2.1. Acquire, render and present. The presentation engine lends an image, a submission renders into it, and presentation hands it back.

Acquiring an image, recreating the swapchain if it is out of date — code/u4/present/presenter.cpp

```
std::optional<AcquiredImage> Presenter::acquire(uint32_t slot) {
    if (display_->takeResized() && !recreate()) return std::nullopt;
    uint32_t index = 0;
    for (;;) {
        const VkResult result =
            vkAcquireNextImageKHR(ctx_->device(), swapchain_.handle(), UINT64_MAX,
                                  imageAcquired_[slot], VK_NULL_HANDLE, &index);
        if (result == VK_ERROR_OUT_OF_DATE_KHR) {
            if (!recreate()) return std::nullopt;
            continue;
        }
        // Suboptimal still signals the semaphore: render this frame, and present() recreates.
        if (result == VK_SUBOPTIMAL_KHR) suboptimal_ = true;
        vkf::check(result, "vkAcquireNextImageKHR");
        break;
    }
    return AcquiredImage{index,
                         swapchain_.image(index),
                         swapchain_.view(index),
                         swapchain_.extent(),
                         swapchain_.surfaceFormat().format,
                         imageAcquired_[slot],
                         swapchain_.readyToPresent(index)};
}
```

Submitting with the two semaphores, then presenting — code/u4/present/presenter.cpp

```
void Presenter::present(const AcquiredImage& image) {
    const VkSwapchainKHR swapchain = swapchain_.handle();
    const VkPresentInfoKHR info{
        .sType = VK_STRUCTURE_TYPE_PRESENT_INFO_KHR,
        .waitSemaphoreCount = 1,
        .pWaitSemaphores = &image.ready,
        .swapchainCount = 1,
        .pSwapchains = &swapchain,
        .pImageIndices = &image.imageIndex,
    };
    const VkResult result = vkQueuePresentKHR(ctx_->mainQueue(), &info);
    if (result == VK_ERROR_OUT_OF_DATE_KHR || result == VK_SUBOPTIMAL_KHR || suboptimal_) {
        recreate();
    } else {
        vkf::check(result, "vkQueuePresentKHR");
    }
}
```

The submission waits for the acquire semaphore only at `VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT`, so work before that stage, such as vertex processing, can start while the image is still being displayed. The first barrier on the image must therefore have the same stage as its source, which chains it to the semaphore wait, as Chapter 3.2 explained. It transitions the image from `UNDEFINED`, because its previous contents do not matter:

Preparing the acquired image and the depth buffer for rendering code/u4/spin/main.cpp

```
void beginFrame(VkCommandBuffer cmd, const u4::Frame& frame, const Scene& scene) {
    vkf::imageBarrier(cmd, frame.image, VK_IMAGE_LAYOUT_UNDEFINED,
                      VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL,
                      VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT, VK_ACCESS_2_NONE,
                      VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT,
                      VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT);
    vkf::imageBarrier(cmd, scene.depth, VK_IMAGE_LAYOUT_UNDEFINED,
                      VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL,
                      VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT,
                      VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT,
                      VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT |
                          VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT,
                      VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_READ_BIT |
                          VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT,
                      {VK_IMAGE_ASPECT_DEPTH_BIT, 0, 1, 0, 1});
}
```

The depth buffer is shared by every frame, so its barrier also waits for the previous frame's depth writes. After rendering, the image goes to `VK_IMAGE_LAYOUT_PRESENT_SRC_KHR`, the layout presentation requires. The semaphore signalled at the end of the submission makes the writes visible to the presentation engine, so the barrier needs no destination scope:

Handing the image to presentation code/u4/spin/main.cpp

```
void endFrame(VkCommandBuffer cmd, const u4::Frame& frame) {
    vkf::imageBarrier(
        cmd, frame.image, VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL,
        VK_IMAGE_LAYOUT_PRESENT_SRC_KHR, VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT,
        VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT, VK_PIPELINE_STAGE_2_NONE, VK_ACCESS_2_NONE);
}
```

> **KEY IDEA**
> A semaphore waited on by `vkQueuePresentKHR` is in use until the presentation engine has finished with the image, and nothing tells the program when that is, except that the same image can be acquired again. If each present semaphore belongs to its image, it is reused only after that image is acquired again, and so never too early.

The acquire semaphores, by contrast, belong to frames in flight, because a frame's acquire semaphore is free again once that frame's submission has completed, which the frame ring already waits for.

Frames in flight

The presenter keeps two frames in flight, as Chapter 3.6 taught, with a command pool for each slot and a timeline semaphore that each submission signals with its frame number. Starting a frame waits until the frame that last used the slot has finished, then resets the slot's pool:

Waiting for the slot's previous frame, then beginning its command buffer code/u4/present/frames.cpp

```
VkCommandBuffer FrameRing::begin() {
    slot_ = static_cast<uint32_t>(submitted_ % slots_.size());
    const vkf::CpuTimer timer;
    if (submitted_ ⩾ slots_.size()) waitFor(submitted_ + 1 - slots_.size());
    waitedMs_ = timer.elapsedMs();
    VKF_CHECK(vkResetCommandPool(ctx_->device(), slots_[slot_].pool, 0));
    vkf::beginCommands(slots_[slot_].cmd);
    return slots_[slot_].cmd;
}
```

Submitting, with the timeline signal added to the frame's own — code/u4/present/frames.cpp

```
void FrameRing::submit(VkQueue queue, std::span<const vkf::SemaphoreSubmit> waits,
                       std::span<const vkf::SemaphoreSubmit> signals) {
    const VkCommandBuffer cmd = slots_[slot_].cmd;
    vkf::endCommands(cmd);
    std::vector<vkf::SemaphoreSubmit> allSignals(signals.begin(), signals.end());
    allSignals.push_back({timeline_, submitted_ + 1, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT});
    vkf::submit(queue, std::span(&cmd, 1), waits, allSignals);
    ++submitted_;
}
```

When the swapchain no longer fits

A swapchain is created for one size of surface. When the window is resized, acquiring or presenting returns `VK_ERROR_OUT_OF_DATE_KHR`, which means the swapchain can no longer be used, or `VK_SUBOPTIMAL_KHR`, which means it can be used but no longer matches the surface exactly. Window systems differ in which they report and when, so the presenter also listens for GLFW's resize event. In each case it waits for the device to go idle, recreates the swapchain at the new size, and calls back into the program to recreate anything that depends on it. A minimised window has a size of zero, for which no swapchain can be created, so the presenter waits for events until the window is restored.

Recreating the swapchain — code/u4/present/presenter.cpp

```
bool Presenter::recreate() {
    VkExtent2D extent = display_->framebufferExtent();
    while (extent.width == 0 || extent.height == 0) {
        if (display_->closeRequested()) return false;
        display_->waitEvents();
        extent = display_->framebufferExtent();
    }
    VKF_CHECK(vkDeviceWaitIdle(ctx_->device()));
    swapchain_.recreate(extent);
    suboptimal_ = false;
    ++recreations_;
    if (onRecreate_) onRecreate_(swapchain_);
    return true;
}
```

The depth buffer follows the swapchain's size, and the pipeline its format — code/u4/spin/main.cpp

```
presenter.onRecreate([&](const u4::Swapchain& swapchain) {
    const VkExtent2D extent = swapchain.extent();
    depth = vkf::createImage(ctx,
                             {.format = depthFormat,
                              .width = extent.width,
                              .height = extent.height,
                              .usage = VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT,
                              .aspect = VK_IMAGE_ASPECT_DEPTH_BIT},
                             "depth");
    if (swapchain.surfaceFormat().format != pipelineFormat) {
        pipelineFormat = swapchain.surfaceFormat().format;
        pipeline = u4::createGraphicsPipeline(
            ctx, {.layout = layout,
                  .vertexShader = vertexShader,
                  .fragmentShader = fragmentShader,
                  .vertexBindings = std::span(&input.binding, 1),
                  .vertexAttributes = input.attributes,
                  .colorFormat = pipelineFormat,
                  .depthFormat = depthFormat,
                  .name = "spinning cube"});
    }
    vkf::print("swapchain {}x{}: {} images, {}, {} frames in flight\n", extent.width,
               extent.height, swapchain.imageCount(),
               string_VkPresentModeKHR(swapchain.presentMode()),
               presenter.framesInFlight());
});
```

Waiting for the whole device to go idle is the simplest correct approach and fine for a resize, which is rare.

The loop

The spinning cube's loop asks the presenter for a frame, records the barriers and the draw into the frame's command buffer, and hands the frame back:

The frame loop — code/u4/spin/main.cpp

```
while ((frames == 0 || presented < frames) && !display.closeRequested()) {
    display.pollEvents();
    if (presented == resizeAt) {
        vkf::print("frame {}: resizing to {}x{}\n", presented, width * 4 / 5,
                   height * 3 / 5);
        display.resize(width * 4 / 5, height * 3 / 5);
    }
    const std::optional<u4::Frame> frame = presenter.begin();
    if (!frame) break;
    const float seconds = static_cast<float>(clock.elapsedMs() / 1000);
    const float aspect = float(frame->extent.width) / float(frame->extent.height);
    const Mat4 mvp = u4::perspective(u4::Pi / 4, aspect, 0.1f, 100.0f) *
                     u4::lookAt({0, 1.5f, 6}, {0, 0, 0}, {0, 1, 0}) *
                     u4::rotateY(seconds * 0.9f) * u4::rotateX(seconds * 0.4f);

    const Scene scene{layout, pipeline, &cube, depth, depth.view};
    beginFrame(frame->cmd, *frame, scene);
    drawCube(frame->cmd, *frame, scene, mvp);
    const bool last = frames != 0 && presented + 1 == frames;
    if (last && canRead) {
        readback = vkf::createBuffer(
            ctx, VkDeviceSize(frame->extent.width) * frame->extent.height * 4,
            VK_BUFFER_USAGE_TRANSFER_DST_BIT, vkf::MemoryUse::Readback, "readback");
        endFrameWithReadback(frame->cmd, *frame, readback);
    } else {
        endFrame(frame->cmd, *frame);
    }
    presenter.end(*frame);
    ++presented;

    frameMs.push_back(frameTimer.elapsedMs());
    frameTimer.restart();
    slotWaitMs.push_back(frame->slotWaitMs);
    acquireMs.push_back(frame->acquireMs);
    presentMs.push_back(presenter.lastPresentMs());
}
presenter.drain();
```

Two seconds in a window on the M2 Pro's 120 Hz display, with validation on

```
$ ./build/bin/u4_spin
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
surface: window, VK_FORMAT_B8G8R8A8_SRGB, VK_COLOR_SPACE_SRGB_NONLINEAR_KHR
swapchain 1600x1200: 3 images, VK_PRESENT_MODE_FIFO_KHR, 2 frames in flight
240 frames in 2.01 s: frame time median 8.04 ms (min 7.39, mean 8.33, max 9.49)
  median wait: frame slot 0.042 ms, acquire 0.024 ms, submit and present 7.777 ms
final image 1600x1200: centre shows the cube, corners show the background
PASS 240 frames presented, 0 swapchain recreation(s)
```

The same headless, with a forced resize after thirty frames

```
$ ./build/bin/u4_spin --headless --resize-at 30
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
surface: headless, VK_FORMAT_B8G8R8A8_SRGB, VK_COLOR_SPACE_SRGB_NONLINEAR_KHR
swapchain 800x600: 3 images, VK_PRESENT_MODE_FIFO_KHR, 2 frames in flight
frame 30: resizing to 640x360
swapchain 640x360: 3 images, VK_PRESENT_MODE_FIFO_KHR, 2 frames in flight
240 frames in 0.05 s: frame time median 0.18 ms (min 0.08, mean 0.20, max 0.71)
  median wait: frame slot 0.108 ms, acquire 0.006 ms, submit and present 0.038 ms
swapchain recreated 1 time(s); extent 640x360 matches the display's 640x360
final image 640x360: centre shows the cube, corners show the background
PASS 240 frames presented, 1 swapchain recreation(s)
```

In the window, with FIFO, the frame time settles at the display's 120 Hz refresh, 8.3 ms, and almost all of it is spent submitting and presenting, because the presentation engine accepts new images only as fast as the display shows them. Headless, nothing waits for a display, and a frame takes 0.2 ms. The program reads back the last image in both cases and checks that its centre shows the cube and its corners the background.

PITFALL

A swapchain image's format may be `B8G8R8A8` rather than `R8G8B8A8`, depending on the platform. Anything that reads swapchain images back, as this program does to check them, must swap the channels for the format it actually got, and a pipeline that renders to them must be created for that format.

COMING FROM OPENGL

OpenGL presents the *default framebuffer* with `SwapBuffers`, through the window system's binding: `glfwSwapBuffers`, GDI's `SwapBuffers` on Windows or `glXSwapBuffers` on Linux. The driver keeps the back buffers, synchronises rendering against display, and resizes the default framebuffer with the window. Vertical synchronisation is a swap interval, set with `glfwSwapInterval`. In Vulkan, every piece of that (the images, their ownership, the waits and the resize) is an object or call that you manage.

Reading

- The Vulkan specification, chapter "Window System Integration (WSI)", in particular "Surface Queries" and "WSI Swapchain".
- The Vulkan Guide, "Window System Integration (WSI)" and "Swapchain Semaphore Reuse".
- The GLFW documentation, "Vulkan guide".

Labs

LAB 4.2.1 In a window and headless — 45 minutes

Goal. Run the presentation loop both ways.

Code. `code/u4/spin`

1. Run `build/bin/u4_spin`, then with `--headless --resize-at 30`.
2. Resize the window by hand while the windowed version runs with `--frames 0`, minimise it, restore it and close it.

DONE WHEN

- [ ] Every run exits with status 0 and validation silent.
- [ ] You can account for the difference in frame time between the two runs.

EVIDENCE FOR THE NOTEBOOK
- The outputs.

LAB 4.2.2 Present modes, image counts and latency — 1.5 hours

Goal. Measure how the presentation settings change frame timing.

Code. `code/u4/spin`

1. Run in a window with `--present-mode` set to `fifo`, `mailbox` and `immediate`; the program reports which it got.
2. With FIFO, run with `--images 2`, `3` and `4` and `--frames-in-flight 1`, `2` and `3`.
3. Record the frame time and the median waits for each combination.

DONE WHEN
- [ ] You have a table of every combination your platform supports.
- [ ] You can explain, for one combination, how many frames separate recording a frame and its appearance on screen.

EVIDENCE FOR THE NOTEBOOK
- The table and the explanation.

LAB 4.2.3 Break the semaphores — 1.5 hours

Goal. See why the semaphore arrangement matters.

Code. `code/u4/present`

1. In a copy of the presenter, use one ready-to-present semaphore per frame slot instead of per image, and run with validation on, in a window and headless.
2. Then make the acquire semaphores per image instead of per slot, and run again.
3. Record what validation reports, and explain each arrangement's flaw from the specification.

DONE WHEN
- [ ] You have the validation output for both variants.
- [ ] Your explanation says, for each, which semaphore could be reused while still in use, and by what.

EVIDENCE FOR THE NOTEBOOK
- The outputs and explanations.

LAB 4.2.4 Resize without stalling — 1.5 hours

Goal. Find what a resize costs.

Code. `code/u4/present`

1. Time the recreation in `Presenter::recreate`, including `vkDeviceWaitIdle`, during a series of window resizes.
2. Replace the idle wait with waiting for the frame ring's timeline, so that only this queue's frames must finish, and confirm with validation that it is still correct.

DONE WHEN
- [ ] Both versions survive repeated resizes with validation silent.
- [ ] You have the cost of a recreation in both versions.

EVIDENCE FOR THE NOTEBOOK
- The timings and the changed code.

Problem set

1. A surface reports `minImageCount` 2 and `maxImageCount` 0. The program asks for three images. How many does it get, and what does the maximum of zero mean?
2. Why does the submission wait for the acquire semaphore at `COLOR_ATTACHMENT_OUTPUT` and not at `ALL_COMMANDS`? What would change if it waited at the top of the pipeline?
3. The acquire barrier transitions the image from `UNDEFINED`. Why is that correct even though the image was presented with contents a moment ago?
4. With FIFO, three images and two frames in flight on a 60 Hz display, the GPU takes 2 ms per frame. Describe the steady state: where does the CPU wait, and how old is a frame when it appears?
5. Why does the program need a separate path for headless surfaces at all, given that it never looks at what is presented?

COMPLETION CHECKLIST
- ☐ I can create a surface from a window or headlessly, and choose a swapchain's format, present mode, image count and extent.
- ☐ I can acquire, render and present with the right semaphores and barriers, and explain why each semaphore belongs where it does.
- ☐ I can recreate a swapchain on resize, out-of-date and suboptimal results, and minimised windows.
- ☐ I can measure and reason about frame timing and latency under each present mode.
- ☐ Lab 4.2.1–4.2.4 done-when criteria all hold, with evidence filed.

CHAPTER 4.3

Drawing with data

TIME	6 hours
YOU BUILD	A scene of textured cubes on a textured ground: meshes uploaded through a staging buffer, a uniform buffer for each frame in flight, four textures with full mipmap chains generated on the GPU and checked against the CPU, trilinear and anisotropic sampling, and a bindless table of textures chosen per draw by a push constant.
NEEDS	Chapters 4.1 and 4.2.

WHY IT MATTERS

Chapter 4.1 drew from host-visible buffers and coloured faces by their normals. Real rendering draws meshes from device-local memory, reads per-frame data such as the camera from uniform buffers that must not be overwritten while the GPU reads them, and samples textures, which need mipmaps to avoid aliasing and the right colour space to look right. It also chooses among many textures per draw, which is where Vulkan's descriptor model is most constraining and where descriptor indexing, the "bindless" style, removes most of the constraints. This chapter puts all of these together and verifies each, including the mipmaps, against the CPU.

SKIP TEST

1. Which barrier does a vertex buffer filled by `vkCmdCopyBuffer` need before a draw reads it?
2. Why does each frame in flight have its own uniform buffer?
3. How is a mipmap chain generated on the GPU, and which layouts does each level pass through?
4. Why does averaging the bytes of an sRGB texture give the wrong colour for its smaller mip levels?
5. What must a device support for a shader to index an array of textures with a value from a push constant?

If all five are easy, read the section on bindless textures and do Labs 4.3.2 and 4.3.4.

Core ideas

Meshes in device-local memory

Vertex and index data are read every frame by every draw, so they belong in device-local memory, filled through a staging buffer as Chapter 1.2 described. One staging buffer holds both, and two copies move them into their buffers. The barrier after the copies has two destinations: the vertex attribute fetch at `VERTEX_ATTRIBUTE_INPUT` and the index fetch at `INDEX_INPUT`, each with its own access type:

Uploading a mesh through one staging buffer — code/u4/textured/main.cpp

```
MeshBuffers uploadThroughStaging(const vkf::Context& ctx, const u4::Mesh& mesh,
                                 const char* name) {
    const VkDeviceSize vertexBytes = mesh.vertices.size() * sizeof(u4::MeshVertex);
    const VkDeviceSize indexBytes = mesh.indices.size() * sizeof(uint16_t);
    vkf::Buffer staging =
        vkf::createBuffer(ctx, vertexBytes + indexBytes, VK_BUFFER_USAGE_TRANSFER_SRC_BIT,
                          vkf::MemoryUse::Upload, "mesh staging");
    std::memcpy(staging.data<uint8_t>(), mesh.vertices.data(), vertexBytes);
    std::memcpy(staging.data<uint8_t>() + vertexBytes, mesh.indices.data(), indexBytes);
```

```
    MeshBuffers gpu{
        vkf::createBuffer(ctx, vertexBytes,
                          VK_BUFFER_USAGE_VERTEX_BUFFER_BIT | VK_BUFFER_USAGE_TRANSFER_DST_BIT,
                          vkf::MemoryUse::DeviceLocal, name),
        vkf::createBuffer(ctx, indexBytes,
                          VK_BUFFER_USAGE_INDEX_BUFFER_BIT | VK_BUFFER_USAGE_TRANSFER_DST_BIT,
                          vkf::MemoryUse::DeviceLocal, name),
        static_cast<uint32_t>(mesh.indices.size())};
    vkf::submitNow(ctx, [&](VkCommandBuffer cmd) {
        const VkBufferCopy vertexRegion{0, 0, vertexBytes};
        const VkBufferCopy indexRegion{vertexBytes, 0, indexBytes};
        vkCmdCopyBuffer(cmd, staging, gpu.vertices, 1, &vertexRegion);
        vkCmdCopyBuffer(cmd, staging, gpu.indices, 1, &indexRegion);
        vkf::memoryBarrier(cmd, VK_PIPELINE_STAGE_2_COPY_BIT, VK_ACCESS_2_TRANSFER_WRITE_BIT,
                           VK_PIPELINE_STAGE_2_VERTEX_ATTRIBUTE_INPUT_BIT |
                               VK_PIPELINE_STAGE_2_INDEX_INPUT_BIT,
                           VK_ACCESS_2_VERTEX_ATTRIBUTE_READ_BIT | VK_ACCESS_2_INDEX_READ_BIT);
    });
    return gpu;
}
```

The vertices now carry texture coordinates, `u` and `v`, as well as positions and normals. Chapter 4.1's cube already had them. The vertex shader reads the camera's matrix from a uniform buffer in set 0 and the model matrix from push constants, and passes the texture coordinate on:

The vertex shader: a per-frame uniform and a per-draw push constant — code/u4/textured/textured.vert

```
#version 460

layout(location = 0) in vec3 inPosition;
layout(location = 2) in vec2 inUv;

layout(set = 0, binding = 0) uniform FrameUniforms {
    mat4 viewProjection;
} frame;

layout(push_constant) uniform DrawConstants {
    mat4 model;
    uint textureIndex;
} draw;

layout(location = 0) out vec2 uv;

void main() {
    gl_Position = frame.viewProjection * draw.model * vec4(inPosition, 1.0);
    uv = inUv;
}
```

Data for each frame in flight

The camera moves every frame, so its matrix is rewritten every frame. With two frames in flight, the CPU writes frame n + 1's matrix while the GPU may still be reading frame n's, so the two must be in different memory. The program keeps one small host-visible uniform buffer and one descriptor set per frame slot:

A uniform buffer and a descriptor set for each frame in flight — code/u4/textured/main.cpp

```
std::vector<vkf::Buffer> uniforms;
std::vector<VkDescriptorSet> frameSets;
for (uint32_t slot = 0; slot < framesInFlight; ++slot) {
    uniforms.push_back(vkf::createBuffer(ctx, sizeof(FrameUniforms),
                                         VK_BUFFER_USAGE_UNIFORM_BUFFER_BIT,
                                         vkf::MemoryUse::Upload, "frame uniforms"));
    frameSets.push_back(pool.allocate(frameLayout, "frame uniforms"));
    vkf::DescriptorWriter()
        .buffer(0, VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER, uniforms.back())
        .update(ctx, frameSets.back());
}
```

Each frame writes its slot's buffer after the frame ring has waited for that slot `code/u4/textured/main.cpp`

```
u4::FrameRing ring(ctx, ctx.mainQueue().family, framesInFlight);
std::string slotsUsed;
for (uint32_t f = 0; f < frames; ++f) {
    const VkCommandBuffer cmd = ring.begin();
    const float orbit = 2 * u4::Pi * float(f) / float(std::max(frames - 1, 1u));
    *uniforms[ring.slot()].data<FrameUniforms>() = {cameraAt(orbit, extent)};
    slotsUsed += " " + std::to_string(ring.slot());
    beginTargets(cmd, target.targets);
    drawScene(cmd, scene, frameSets[ring.slot()], tableSet, target.targets);
    if (f + 1 == frames) copyToReadback(cmd, target.targets, target.readback);
    ring.submit(ctx.mainQueue());
}
ring.drain();
```

The write is safe because the frame ring's wait in `begin` returns only when the frame that last used the slot has finished on the GPU. The program prints which slot each frame used, `0 1 0 1`, as a check.

> **KEY IDEA**
> Anything the CPU writes every frame and the GPU reads needs one copy per frame in flight, indexed by the frame's slot. Anything written once and read every frame, such as meshes and textures, needs one copy, protected by a barrier after it is written.

Textures and their mipmaps

A texture viewed from far away covers fewer pixels than it has texels, and sampling it directly skips most of its texels, which shows as shimmering moiré patterns. *Mipmaps* solve this: a chain of copies of the texture, each half the size of the one before, down to a single texel. The sampler picks the level whose texels are about the size of a pixel, and with *trilinear* filtering blends between the two nearest levels.

The program's textures are 256 × 256 with nine levels. It uploads level 0 from a staging buffer and generates the others on the GPU with `vkCmdBlitImage`, which scales a region of one image level into another with filtering. Each level is the blit of the one above, which must first move from `TRANSFER_DST_OPTIMAL`, where it was written, to `TRANSFER_SRC_OPTIMAL`, from which it is read (Figure 4.3.1):

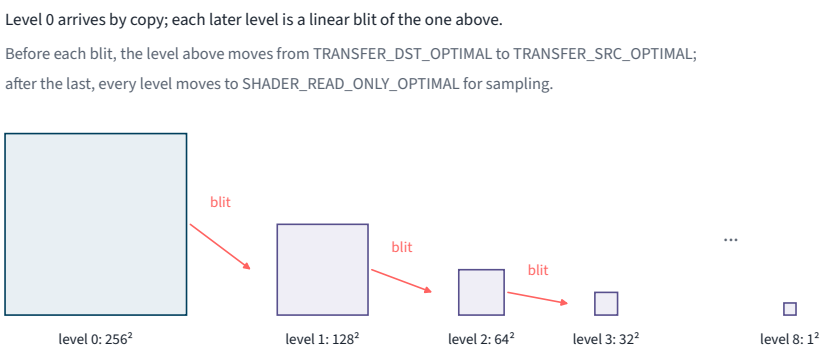


Figure 4.3.1. Generating a mipmap chain with blits. Each level is written as a blit destination, then read as the source of the next.

One blit per level, with a barrier on the level above before each — code/u4/textured/main.cpp

```
// Leaves the last level in TRANSFER_DST_OPTIMAL and the others in TRANSFER_SRC_OPTIMAL.
void generateMipmaps(VkCommandBuffer cmd, VkImage image, uint32_t levels) {
    for (uint32_t level = 1; level < levels; ++level) {
        vkf::imageBarrier(
            cmd, image, VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL,
            VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL,
            level == 1 ? VK_PIPELINE_STAGE_2_COPY_BIT : VK_PIPELINE_STAGE_2_BLIT_BIT,
            VK_ACCESS_2_TRANSFER_WRITE_BIT, VK_PIPELINE_STAGE_2_BLIT_BIT,
            VK_ACCESS_2_TRANSFER_READ_BIT, vkf::colorRange(level - 1, 1));
        const auto above = static_cast<int32_t>(TextureSize >> (level - 1));
        const auto size = static_cast<int32_t>(TextureSize >> level);
        const VkImageBlit blit{
            .srcSubresource = {VK_IMAGE_ASPECT_COLOR_BIT, level - 1, 0, 1},
            .srcOffsets = {{0, 0, 0}, {above, above, 1}},
            .dstSubresource = {VK_IMAGE_ASPECT_COLOR_BIT, level, 0, 1},
            .dstOffsets = {{0, 0, 0}, {size, size, 1}},
        };
        vkCmdBlitImage(cmd, image, VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL, image,
                       VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL, 1, &blit, VK_FILTER_LINEAR);
    }
}
```

At the end every level but the last is in `TRANSFER_SRC_OPTIMAL` and the last in `TRANSFER_DST_OPTIMAL`, so two barriers, one per range, move them all to `SHADER_READ_ONLY_OPTIMAL` for the fragment shader. Only the last level's barrier needs a source access, because the other levels were last written before an earlier barrier, which already made their writes available.

Every level to SHADER_READ_ONLY_OPTIMAL — code/u4/textured/main.cpp

```
void finishMipmaps(VkCommandBuffer cmd, VkImage image, uint32_t levels) {
    const VkImageMemoryBarrier2 barriers[] = {
        {
            .sType = VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER_2,
            .srcStageMask = VK_PIPELINE_STAGE_2_BLIT_BIT,
            .srcAccessMask = VK_ACCESS_2_NONE,
            .dstStageMask = VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT,
            .dstAccessMask = VK_ACCESS_2_SHADER_SAMPLED_READ_BIT,
            .oldLayout = VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL,
            .newLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL,
            .srcQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
            .dstQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
            .image = image,
            .subresourceRange = vkf::colorRange(0, levels - 1),
        },
        {
            .sType = VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER_2,
            .srcStageMask = VK_PIPELINE_STAGE_2_BLIT_BIT,
            .srcAccessMask = VK_ACCESS_2_TRANSFER_WRITE_BIT,
            .dstStageMask = VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT,
            .dstAccessMask = VK_ACCESS_2_SHADER_SAMPLED_READ_BIT,
            .oldLayout = VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL,
            .newLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL,
            .srcQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
            .dstQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
            .image = image,
            .subresourceRange = vkf::colorRange(levels - 1, 1),
        },
    };
    const VkDependencyInfo dependency{
        .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
        .imageMemoryBarrierCount = 2,
        .pImageMemoryBarriers = barriers,
    };
    vkCmdPipelineBarrier2(cmd, &dependency);
}
```

Blitting with linear filtering is an optional feature of each format, so the program checks for it and falls back to generating the levels on the CPU:

Linear blits are a format feature code/u4/textured/main.cpp

```
bool canBlitLinear(const vkf::Context& ctx, VkFormat format) {
    VkFormatProperties properties;
    vkGetPhysicalDeviceFormatProperties(ctx.physicalDevice(), format, &properties);
    const VkFormatFeatureFlags needed = VK_FORMAT_FEATURE_BLIT_SRC_BIT |
                                        VK_FORMAT_FEATURE_BLIT_DST_BIT |
                                        VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT;
    return (properties.optimalTilingFeatures & needed) == needed;
}
```

Mipmaps in the right colour space

The textures are stored as `VK_FORMAT_R8G8B8A8_SRGB`: their bytes are sRGB-encoded, as image files are, and the hardware decodes them to linear values when a shader samples them and encodes when a blit or a render writes them. Averaging must happen in linear light. The average of black and white is a linear 0.5, which sRGB encodes as 188, whereas averaging the bytes gives 128. A blit between sRGB images does this correctly, and the program's CPU reference does the same, so that it can check every level of every texture to within one step of 8 bits:

The CPU's mipmap chain, averaged in linear light code/u4/textured/main.cpp

```
// Averages in linear light, as a blit of an _SRGB image does.
std::vector<std::vector<uint8_t>> cpuMipChain(const std::vector<uint8_t>& base) {
    std::vector<std::vector<uint8_t>> levels{base};
    for (uint32_t size = TextureSize / 2; size >= 1; size /= 2) {
        const std::vector<uint8_t>& above = levels.back();
        std::vector<uint8_t> level(size_t(size) * size * 4);
        for (uint32_t y = 0; y < size; ++y) {
            for (uint32_t x = 0; x < size; ++x) {
                for (uint32_t c = 0; c < 4; ++c) {
                    float sum = 0;
                    for (uint32_t i = 0; i < 4; ++i) {
                        const size_t texel =
                            (size_t(2 * y + i / 2) * size * 2 + 2 * x + i % 2);
                        const uint8_t v = above[texel * 4 + c];
                        sum += c < 3 ? toLinear(v) : float(v) / 255.0f;
                    }
                    const size_t at = (size_t(y) * size + x) * 4 + c;
                    level[at] = c < 3 ? toSrgb(sum / 4)
                                      : static_cast<uint8_t>(std::lround(sum / 4 * 255));
                }
            }
        }
        levels.push_back(std::move(level));
    }
    return levels;
}
```

The checker texture's single-texel level is (188, 188, 188), and the program prints what naive byte averaging would have given, 128, which is a visibly darker grey at a distance.

Samplers for textures

The sampler enables trilinear filtering by setting `mipmapMode` to `LINEAR` and `maxLod` to `VK_LOD_CLAMP_NONE`, and *anisotropic filtering*, which takes several samples along the direction in which a texture is compressed on screen. Surfaces seen at a grazing angle, like the ground here, are compressed far more in one direction than the other, and anisotropic filtering keeps them sharp where plain trilinear filtering would blur them. It is an optional feature, `samplerAnisotropy`, with a device limit on the number of samples.

Trilinear, anisotropic, repeating — code/u4/textured/main.cpp

```
vkf::Unique<VkSampler> createSampler(const vkf::Context& ctx, bool mipmaps) {
    const bool anisotropy = mipmaps && ctx.features().core.samplerAnisotropy;
    const float maxAnisotropy =
        std::min(16.0f, ctx.properties().core.limits.maxSamplerAnisotropy);
    const VkSamplerCreateInfo info{
        .sType = VK_STRUCTURE_TYPE_SAMPLER_CREATE_INFO,
        .magFilter = VK_FILTER_LINEAR,
        .minFilter = VK_FILTER_LINEAR,
        .mipmapMode = VK_SAMPLER_MIPMAP_MODE_LINEAR,
        .addressModeU = VK_SAMPLER_ADDRESS_MODE_REPEAT,
        .addressModeV = VK_SAMPLER_ADDRESS_MODE_REPEAT,
        .addressModeW = VK_SAMPLER_ADDRESS_MODE_REPEAT,
        .anisotropyEnable = anisotropy,
        .maxAnisotropy = anisotropy ? maxAnisotropy : 1.0f,
        .minLod = 0.0f,
        .maxLod = mipmaps ? VK_LOD_CLAMP_NONE : 0.0f,
    };
    VkSampler sampler = VK_NULL_HANDLE;
    VKF_CHECK(vkCreateSampler(ctx.device(), &info, nullptr, &sampler));
    return {ctx.device(), sampler};
}
```

Many textures: the bindless table

Each cube shows a different texture. One way to draw them is a descriptor set per texture, bound before each draw. Another is to bind one array of textures once and let each draw choose an element. That is what the program does, with a push constant holding the index. The table lives in set 1 as an array of sampled images, separate from a single sampler, which the shader combines with `sampler2D(image, sampler)`:

Indexing a table of textures with a push constant — code/u4/textured/bindless.frag

```
layout(set = 1, binding = 0) uniform texture2D textureTable[];
layout(set = 1, binding = 1) uniform sampler tableSampler;

layout(push_constant) uniform DrawConstants {
    mat4 model;
    uint textureIndex;
} draw;

layout(location = 0) in vec2 uv;
layout(location = 0) out vec4 outColor;

void main() {
    outColor = texture(
        sampler2D(textureTable[nonuniformEXT(draw.textureIndex)], tableSampler), uv);
}
```

Indexing an array of images with a value that is not a compile-time constant needs *descriptor indexing*, core in Vulkan 1.2 but with optional features. `runtimeDescriptorArray` allows the array's size to be unspecified in the shader; `descriptorBindingPartiallyBound` allows the table to have more slots than the program has filled; and `shaderSampledImageArrayNonUniformIndexing` allows an index that differs between invocations, which the shader marks with `nonuniformEXT`. The index here comes from a push constant and so is the same for every invocation of a draw, but marking it costs nothing and keeps the shader correct if the index ever varies. The program checks for the features:

Using the table only where the device supports it — code/u4/textured/main.cpp

```
const VkPhysicalDeviceVulkan12Features& f12 = ctx.features().v12;
const bool bindless = !args.flag("--no-bindless") && f12.runtimeDescriptorArray &&
                      f12.descriptorBindingPartiallyBound &&
                      f12.shaderSampledImageArrayNonUniformIndexing;
const bool blit = !args.flag("--cpu-mips") && canBlitLinear(ctx, TextureFormat);
```

A uniform buffer in set 0; a table of up to 64 textures and a sampler in set 1 code/u4/textured/main.cpp

```
const VkPhysicalDeviceLimits& limits = ctx.properties().core.limits;
const auto frameLayout =
    vkf::DescriptorSetLayoutBuilder()
        .add(0, VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER, VK_SHADER_STAGE_VERTEX_BIT)
        .build(ctx);
const uint32_t capacity =
    bindless ? std::min(TableCapacity, limits.maxPerStageDescriptorSampledImages)
             : TextureCount;
const auto tableLayout =
    vkf::DescriptorSetLayoutBuilder()
        .add(0, VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE, VK_SHADER_STAGE_FRAGMENT_BIT,
             capacity, bindless ? VK_DESCRIPTOR_BINDING_PARTIALLY_BOUND_BIT : 0)
        .add(1, VK_DESCRIPTOR_TYPE_SAMPLER, VK_SHADER_STAGE_FRAGMENT_BIT)
        .build(ctx);
const VkPushConstantRange push{
    VK_SHADER_STAGE_VERTEX_BIT | VK_SHADER_STAGE_FRAGMENT_BIT, 0,
    sizeof(DrawConstants)};
const auto layout =
    vkf::createPipelineLayout(ctx, {frameLayout.get(), tableLayout.get()}, {push});
```

Only four of the sixty-four slots are written, and partial binding makes the rest legal as long as no shader reads them.

Writing the textures into the table's first slots, and the sampler code/u4/textured/main.cpp

```
vkf::DescriptorPool pool(ctx);
auto writeTable = [&](VkSampler tableSampler, const char* name) {
    const VkDescriptorSet set = pool.allocate(tableLayout, name);
    vkf::DescriptorWriter writer;
    for (uint32_t t = 0; t < TextureCount; ++t) {
        writer.image(0, VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE, textures[t].view,
                     VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL, VK_NULL_HANDLE, t);
    }
    writer.image(1, VK_DESCRIPTOR_TYPE_SAMPLER, VK_NULL_HANDLE,
                 VK_IMAGE_LAYOUT_UNDEFINED, tableSampler);
    writer.update(ctx, set);
    return set;
};
const VkDescriptorSet tableSet = writeTable(sampler, "texture table");
const VkDescriptorSet flatTableSet = writeTable(flatSampler, "texture table, level 0");
```

Binding both sets once, then one push constant and one draw per object code/u4/textured/main.cpp

```
void drawScene(VkCommandBuffer cmd, const Scene& scene, VkDescriptorSet frameSet,
               VkDescriptorSet tableSet, const Targets& targets) {
    beginRendering(cmd, targets);
    vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_GRAPHICS, scene.pipeline);
    u4::setViewportAndScissor(cmd, targets.extent);
    const VkDescriptorSet sets[] = {frameSet, tableSet};
    vkCmdBindDescriptorSets(cmd, VK_PIPELINE_BIND_POINT_GRAPHICS, scene.layout, 0, 2, sets, 0,
                            nullptr);
    for (const Object& object : scene.objects) {
        const VkDeviceSize offset = 0;
        vkCmdBindVertexBuffers(cmd, 0, 1, &object.mesh->vertices.buffer, &offset);
        vkCmdBindIndexBuffer(cmd, object.mesh->indices, 0, VK_INDEX_TYPE_UINT16);
        const DrawConstants constants{object.model, object.textureIndex};
        vkCmdPushConstants(cmd, scene.layout,
                           VK_SHADER_STAGE_VERTEX_BIT | VK_SHADER_STAGE_FRAGMENT_BIT, 0,
                           sizeof(constants), &constants);
        vkCmdDrawIndexed(cmd, object.mesh->indexCount, 1, 0, 0, 0);
    }
    vkCmdEndRendering(cmd);
}
```

Without descriptor indexing, a shader may index an array of textures only with constant expressions. The fallback shader, chosen by `--no-bindless` or on devices without the features, uses a `switch` with a constant index in each branch:

Without descriptor indexing: constant indices only `code/u4/textured/fixed.frag`

```
void main() {
    // Without descriptor indexing, only constant indices into an image array are valid.
    switch (draw.textureIndex) {
        case 0u: outColor = texture(sampler2D(textureTable[0], tableSampler), uv); break;
        case 1u: outColor = texture(sampler2D(textureTable[1], tableSampler), uv); break;
        case 2u: outColor = texture(sampler2D(textureTable[2], tableSampler), uv); break;
        default: outColor = texture(sampler2D(textureTable[3], tableSampler), uv); break;
    }
}
```

The textured scene on the M2 Pro, with the bindless table

```
$ ./build/bin/u4_textured
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
textures: 4 x 256x256 VK_FORMAT_R8G8B8A8_SRGB, 9 mip levels blitted on the GPU
  checker's 1x1 level: (188, 188, 188); averaging sRGB bytes gives 128
  every level matches the CPU's linear-light box filter within 1
sampler: trilinear with 16x anisotropic filtering
descriptors: a bindless table of 64 sampled images, 4 written, and one sampler
8 frames, 2 in flight, frame uniform buffers used: 0 1 0 1 0 1 0 1
wrote textured.png and, sampling level 0 only, textured-no-mips.png
  cube faces show textures 0 1 2 3: as drawn
  far ground, rows 90-142: mean 187.0, deviation 12.4
  near ground, rows 435-599: mean 132.2, deviation 123.3
  far ground with level 0 only: mean 129.7, deviation 125.6
PASS textured meshes with mipmaps and bindless textures
```

The program checks the rendered scene as well as the textures. The cube faces must show the textures their draws chose, and the far ground, where many texels fall in each pixel, must be a uniform grey with mipmaps. Its brightness varies by 12 levels with them and by 126 without, which is the moiré of Figure 4.3.2.

PITFALL

Descriptor sets cannot be updated while a pending command buffer uses them, unless the binding was created with `VK_DESCRIPTOR_BINDING_UPDATE_AFTER_BIND_BIT` and its pool and layout with the matching flags. A bindless table that grows while frames are in flight needs that flag, or one table per frame in flight.

COMING FROM OPENGL

OpenGL binds textures to numbered texture units and uniform blocks to binding points, and changes them freely between draws. `glGenerateMipmap` builds the chain in one call, choosing the filter

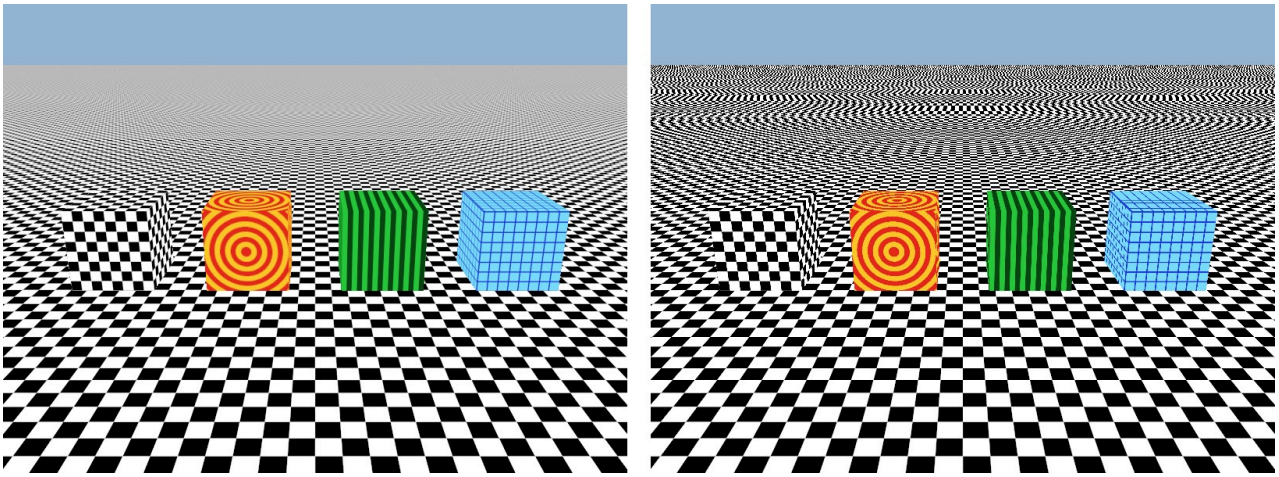


Figure 4.3.2. The scene with mipmaps and with level 0 only. Without mipmaps, the distant checkerboard breaks into moiré.

and the colour space handling itself. Bindless textures exist in OpenGL as `ARB_bindless_texture`, with 64-bit texture handles stored in buffers, which neither macOS nor every desktop driver supports. Vulkan's descriptor indexing is the portable form of the same idea.

Reading

- The Vulkan specification, "Copy Commands" (`vkCmdBlitImage`), "Samplers", and "Descriptor Indexing" in "Resource Descriptors".
- The Vulkan Guide, "Descriptor Arrays", "VK_EXT_descriptor_indexing" and "Image Copies".
- The sRGB transfer function, as defined in IEC 61966-2-1, or in the Vulkan specification's "Formats" chapter.

Labs

LAB 4.3.1 The scene with and without its features — 1 hour

Goal. Run every path through the program.

Code. `code/u4/textured`

1. Run `build/bin/u4_textured`, then with `--no-bindless`, then with `--cpu-mips`.
2. Compare `textured.png` and `textured-no-mips.png` on screen.

DONE WHEN

- [ ] All three runs pass with validation silent.
- [ ] You can explain each of the program's checks on the ground's brightness.

EVIDENCE FOR THE NOTEBOOK

- The three outputs.

LAB 4.3.2 Mipmaps in the wrong colour space — 1.5 hours

Goal. See the error the linear-light average prevents.

Code. `code/u4/textured`

1. Change the CPU path to average sRGB bytes directly, and run with `--cpu-mips`.
2. Create the textures as `VK_FORMAT_R8G8B8A8_UNORM` while uploading the same bytes, and run with blits.
3. Compare the far ground's brightness and the cube faces in each case with the correct version.

DONE WHEN

- [ ] You have the three images and the far ground's mean in each.
- [ ] You can explain each difference with the sRGB transfer function.

EVIDENCE FOR THE NOTEBOOK

- The images, the means and the explanation.

LAB 4.3.3 Anisotropy and level of detail — 1.5 hours

Goal. Measure what the sampler's settings do to the image.

Code. `code/u4/textured`

1. Render with anisotropy off, and with `maxAnisotropy` of 2, 4, 8 and 16.

2. Measure the sharpness of the ground's middle distance, for example the deviation of a band of rows, for each.
3. Add a `mipLodBias` of +1 and −1 and describe the effect.

DONE WHEN
- [ ] You have the deviation for each setting.
- [ ] You can explain what anisotropic filtering does that trilinear filtering cannot.

EVIDENCE FOR THE NOTEBOOK
- The table and two representative crops.

LAB 4.3.4 A larger table — 2 hours

Goal. Draw sixty-four textures from the bindless table.

Code. `code/u4/textured`

1. Generate sixty-four textures, each a different colour, and draw a grid of sixty-four cubes, each using its own texture through the push constant.
2. Check on the CPU that each cube shows its texture's colour.
3. Time recording and submission against a version that binds a separate descriptor set per cube.

DONE WHEN
- [ ] Both versions render the same image, checked on the CPU.
- [ ] You have the CPU time of each.

EVIDENCE FOR THE NOTEBOOK
- The image and the timings.

Problem set

1. A mesh's vertex buffer is written by a compute shader instead of a copy. Change the staging barrier's source and destination accordingly.
2. With three frames in flight, how many uniform buffers does the program need, and what goes wrong if it has two?
3. A 1000 × 600 texture is mipmapped down to one texel. How many levels are there, and what are their sizes? How does `vkCmdBlitImage` handle the odd sizes?
4. The checker texture is half black and half white. Show that its 1 × 1 level is 188 in sRGB, and compute what a UNORM texture of the same bytes would give.
5. Why does `finishMipmaps` use two image barriers, with different source accesses, instead of one?

COMPLETION CHECKLIST
- [ ] I can upload meshes through staging and synchronise them with draws.
- [ ] I can keep per-frame data per frame in flight and per-object data in push constants.
- [ ] I can generate and check mipmaps on the GPU, in the right colour space.
- [ ] I can create samplers with trilinear and anisotropic filtering, and use a bindless table of textures.
- [ ] Lab 4.3.1–4.3.4 done-when criteria all hold, with evidence filed.

CHAPTER 4.4

Compute meets graphics

TIME	7 hours
YOU BUILD	A frame that simulates a million particles in a compute shader and draws them as points from the same buffer; culls 50,000 cubes against the view in another compute shader, which writes the indirect draw that renders the survivors; and tone-maps the rendered image in a third. Every part is checked against the CPU, and each pass is timed.
NEEDS	Chapters 4.1 to 4.3, and Unit 3.

WHY IT MATTERS

The most effective modern renderers let the GPU decide what to draw. Compute shaders simulate, animate and cull, the graphics pipeline draws what they produced with parameters they wrote, and another compute pass post-processes the image. The CPU's part shrinks to recording a few commands per frame, whatever the scene's size. Each of these parts has been covered separately (compute shaders in Unit 2, barriers in Unit 3, the graphics pipeline in Chapters 4.1 to 4.3), and this chapter puts them in one frame, with the barriers between compute and graphics that join them.

SKIP TEST

1. A compute shader writes particle positions that a draw then reads as vertex attributes. Which stages and accesses does the barrier between them name?
2. How can a compute shader decide how many instances a draw renders, without the CPU reading anything back?
3. The frame's simulation writes the particle buffer that the previous frame's draw read. What hazard is that, and what barrier prevents it?
4. Why render into a high-dynamic-range image and convert to the display format in a separate pass?

If all four are easy, read the section on GPU culling and do Labs 4.4.2 and 4.4.4.

Core ideas

A frame in four passes

Each frame has four passes on one queue (Figure 4.4.1). A compute pass advances the particles. A second compute pass tests every cube against the camera's view frustum and writes the indices of the visible ones, with an indirect draw command whose instance count is the number of visible cubes. A graphics pass draws the cubes with that command and the particles as points, into a 16-bit floating-point image. A last compute pass converts the image for display. The frame then ends with a copy for checking, or a blit to the swapchain when there is a window.

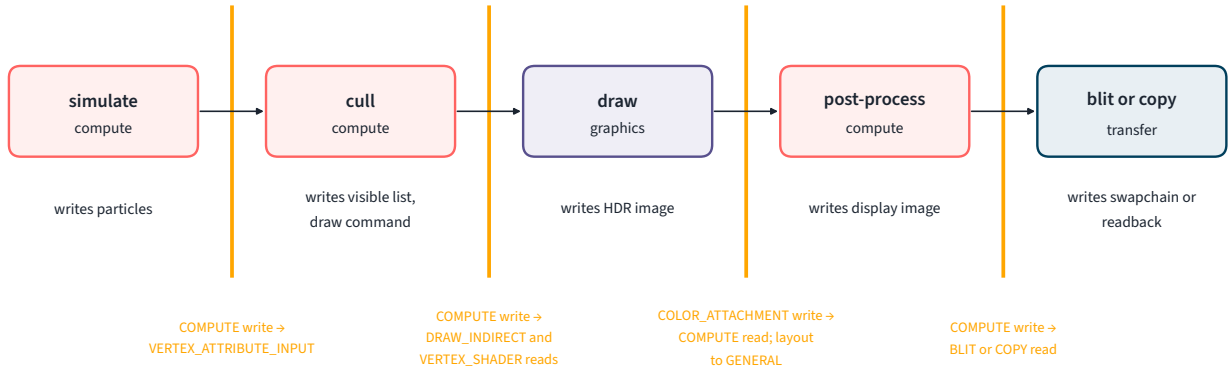


Figure 4.4.1. The four passes of a frame and the barriers between them.

Recording a frame, with a timestamp after each pass — code/u4/particles/main.cpp

```
void recordFrame(VkCommandBuffer cmd, vkf::GpuTimer& timer, const Resources& r,
                 const Pipelines& p, const Mat4& viewProjection) {
    timer.reset(cmd);
    timer.stamp(cmd);
    simulatePass(cmd, r, p);
    timer.stamp(cmd);
    FrustumConstants frustum{.count = r.instanceCount};
    const std::array<u4::Vec4, 6> planes = u4::frustumPlanes(viewProjection);
    std::copy(planes.begin(), planes.end(), frustum.planes);
    cullPass(cmd, r, p, frustum);
    cullToDraw(cmd, r);
    timer.stamp(cmd);
    drawPass(cmd, r, p, viewProjection);
    timer.stamp(cmd);
    postPass(cmd, r, p);
    timer.stamp(cmd);
}
```

One buffer, two roles

The particles live in one device-local buffer, created with both `STORAGE_BUFFER` and `VERTEX_BUFFER` usage, so that the simulation writes it as a storage buffer and the draw reads it as vertices, with no copy in between:

The particle buffer and the draw command buffer, each with every usage it needs — code/u4/particles/main.cpp

```
r.particles = vkf::createBuffer(
    ctx, initial.size() * sizeof(Particle),
    VK_BUFFER_USAGE_STORAGE_BUFFER_BIT | VK_BUFFER_USAGE_VERTEX_BUFFER_BIT |
        VK_BUFFER_USAGE_TRANSFER_DST_BIT | VK_BUFFER_USAGE_TRANSFER_SRC_BIT,
    vkf::MemoryUse::DeviceLocal, "particles");
r.drawCommand = vkf::createBuffer(
    ctx, sizeof(VkDrawIndexedIndirectCommand),
    VK_BUFFER_USAGE_STORAGE_BUFFER_BIT | VK_BUFFER_USAGE_INDIRECT_BUFFER_BIT |
        VK_BUFFER_USAGE_TRANSFER_DST_BIT | VK_BUFFER_USAGE_TRANSFER_SRC_BIT,
    vkf::MemoryUse::DeviceLocal, "draw command");
```

A particle: a position and a velocity, each padded to four floats for `std430` — code/u4/shared/galaxy.hpp

```
struct Particle {
    float position[4];
    float velocity[4];
};
```

The simulation pulls each particle towards the origin with a softened inverse-square force and advances it with semi-implicit Euler integration. The force always points along the particle's position, so each particle's

angular momentum is conserved, up to rounding, which gives the program something exact to check after many steps.

One step of the simulation — code/u4/particles/simulate.comp

```
// The force is parallel to r, so each particle's angular momentum r x v is conserved.
void main() {
    uint i = gl_GlobalInvocationID.x;
    if (i ≥ sim.count) return;
    vec3 r = particles[i].position.xyz;
    vec3 v = particles[i].velocity.xyz;
    float d2 = dot(r, r) + sim.softening * sim.softening;
    vec3 a = -sim.strength * r * inversesqrt(d2 * d2 * d2);
    v += a * sim.dt;
    r += v * sim.dt;
    particles[i].position.xyz = r;
    particles[i].velocity.xyz = v;
}
```

The simulation pass is bracketed by two buffer barriers. The first orders this frame's writes after the previous frame's: after its simulation writes, a write after write, and after its vertex fetches, a write after read. The second makes the new positions visible to the vertex attribute fetch of this frame's draw.

The simulation, ordered against the previous frame's draw and before this frame's — code/u4/particles/main.cpp

```
void simulatePass(VkCommandBuffer cmd, const Resources& r, const Pipelines& p) {
    vkf::bufferBarrier(
        cmd, r.particles,
        VK_PIPELINE_STAGE_2_VERTEX_ATTRIBUTE_INPUT_BIT |
            VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
        VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT, VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
        VK_ACCESS_2_SHADER_STORAGE_READ_BIT | VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT);
    vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, p.simulate);
    vkCmdBindDescriptorSets(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, p.simulateLayout, 0, 1,
                            &p.simulateSet, 0, nullptr);
    const SimulationConstants constants{Gravity.dt, Gravity.strength, Gravity.softening,
                                        r.particleCount};
    vkCmdPushConstants(cmd, p.simulateLayout, VK_SHADER_STAGE_COMPUTE_BIT, 0,
                       sizeof(constants), &constants);
    vkCmdDispatch(cmd, vkf::groupCount(r.particleCount, 256), 1, 1);
    vkf::bufferBarrier(cmd, r.particles, VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
                       VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
                       VK_PIPELINE_STAGE_2_VERTEX_ATTRIBUTE_INPUT_BIT,
                       VK_ACCESS_2_VERTEX_ATTRIBUTE_READ_BIT);
}
```

The draw then binds the same buffer as a vertex buffer and draws one point per particle. The vertex shader reads position and velocity as attributes and colours each particle by its speed:

A million points from the buffer the simulation wrote — code/u4/particles/main.cpp

```
vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_GRAPHICS, p.particles);
vkCmdPushConstants(cmd, p.particleLayout, VK_SHADER_STAGE_VERTEX_BIT, 0,
                   sizeof(viewProjection), &viewProjection);
const VkDeviceSize offset = 0;
vkCmdBindVertexBuffers(cmd, 0, 1, &r.particles.buffer, &offset);
vkCmdDraw(cmd, r.particleCount, 1, 0, 0);
```

The particle's vertex shader — code/u4/particles/particle.vert

```
layout(location = 0) in vec4 inPosition;
layout(location = 1) in vec4 inVelocity;

layout(push_constant) uniform Camera {
    mat4 viewProjection;
} camera;

layout(location = 0) out vec3 color;

void main() {
    gl_Position = camera.viewProjection * vec4(inPosition.xyz, 1.0);
    gl_PointSize = 1.0;
    float speed = clamp(length(inVelocity.xyz) / 1.5, 0.0, 1.0);
    color = mix(vec3(0.15, 0.3, 1.0), vec3(1.0, 0.55, 0.2), speed) * 0.12;
}
```

GPU culling and indirect draws

The 50,000 cubes are described by a buffer of positions and sizes. Each frame, a compute shader tests each cube's bounding sphere against the six planes of the view frustum, which the CPU extracts from the camera's matrix, and appends the index of each visible cube to a list with an atomic counter. The counter is the `instanceCount` field of a `VkDrawIndexedIndirectCommand` in the same buffer:

The draw command, as the shader sees it — code/u4/particles/cull.comp

```
layout(std430, set = 0, binding = 2) buffer DrawCommand {
    uint indexCount;
    uint instanceCount;
    uint firstIndex;
    int vertexOffset;
    uint firstInstance;
} command;
```

Testing a bounding sphere against the frustum and appending the survivors — code/u4/particles/cull.comp

```
void main() {
    uint i = gl_GlobalInvocationID.x;
    if (i ⩾ frustum.count) return;
    vec4 s = positionScale[i];
    float radius = s.w * 1.7320508;
    for (int p = 0; p < 6; ++p) {
        if (dot(frustum.planes[p].xyz, s.xyz) + frustum.planes[p].w < -radius) return;
    }
    uint slot = atomicAdd(command.instanceCount, 1u);
    visible[slot] = i;
}
```

Before the cull, a small `vkCmdUpdateBuffer` resets the command: the index count of the cube mesh, and an instance count of zero. Its barriers order it after the previous frame's indirect read and before this frame's atomic additions:

Resetting the command, then culling — code/u4/particles/main.cpp

```
void cullPass(VkCommandBuffer cmd, const Resources& r, const Pipelines& p,
              const FrustumConstants& frustum) {
    vkf::bufferBarrier(cmd, r.drawCommand, VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT,
                       VK_ACCESS_2_NONE, VK_PIPELINE_STAGE_2_CLEAR_BIT,
                       VK_ACCESS_2_TRANSFER_WRITE_BIT);
    const VkDrawIndexedIndirectCommand reset{r.cube.indexCount, 0, 0, 0, 0};
    vkCmdUpdateBuffer(cmd, r.drawCommand, 0, sizeof(reset), &reset);
    vkf::bufferBarrier(
        cmd, r.drawCommand, VK_PIPELINE_STAGE_2_CLEAR_BIT, VK_ACCESS_2_TRANSFER_WRITE_BIT,
        VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
        VK_ACCESS_2_SHADER_STORAGE_READ_BIT | VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT);
    vkf::bufferBarrier(cmd, r.visible, VK_PIPELINE_STAGE_2_VERTEX_SHADER_BIT, VK_ACCESS_2_NONE,
                       VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
                       VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT);
    vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, p.cull);
    vkCmdBindDescriptorSets(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, p.cullLayout, 0, 1,
                            &p.cullSet, 0, nullptr);
    vkCmdPushConstants(cmd, p.cullLayout, VK_SHADER_STAGE_COMPUTE_BIT, 0, sizeof(frustum),
                       &frustum);
    vkCmdDispatch(cmd, vkf::groupCount(r.instanceCount, 256), 1, 1);
}
```

After the cull, two buffer barriers prepare its outputs for their different readers: the draw command for the indirect stage, which reads it before any shader runs, and the visible list for the vertex shader:

Two consumers, two destination scopes — code/u4/particles/main.cpp

```
void cullToDraw(VkCommandBuffer cmd, const Resources& r) {
    const VkBufferMemoryBarrier2 barriers[] = {
        {.sType = VK_STRUCTURE_TYPE_BUFFER_MEMORY_BARRIER_2,
         .srcStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
         .srcAccessMask = VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
         .dstStageMask = VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT,
         .dstAccessMask = VK_ACCESS_2_INDIRECT_COMMAND_READ_BIT,
         .srcQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
         .dstQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
         .buffer = r.drawCommand,
         .size = VK_WHOLE_SIZE},
        {.sType = VK_STRUCTURE_TYPE_BUFFER_MEMORY_BARRIER_2,
         .srcStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
         .srcAccessMask = VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
         .dstStageMask = VK_PIPELINE_STAGE_2_VERTEX_SHADER_BIT,
         .dstAccessMask = VK_ACCESS_2_SHADER_STORAGE_READ_BIT,
         .srcQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
         .dstQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
         .buffer = r.visible,
         .size = VK_WHOLE_SIZE},
    };
    const VkDependencyInfo dependency{
        .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
        .bufferMemoryBarrierCount = 2,
        .pBufferMemoryBarriers = barriers,
    };
    vkCmdPipelineBarrier2(cmd, &dependency);
}
```

The draw reads its parameters from the buffer with `vkCmdDrawIndexedIndirect`. Each instance's vertex shader looks up its cube through the visible list, using `gl_InstanceIndex`:

Drawing however many cubes the cull kept — code/u4/particles/main.cpp

```
vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_GRAPHICS, p.cubes);
vkCmdBindDescriptorSets(cmd, VK_PIPELINE_BIND_POINT_GRAPHICS, p.cubeLayout, 0, 1,
                        &p.cubeSet, 0, nullptr);
vkCmdPushConstants(cmd, p.cubeLayout, VK_SHADER_STAGE_VERTEX_BIT, 0,
                   sizeof(viewProjection), &viewProjection);
u4::bindMesh(cmd, r.cube);
vkCmdDrawIndexedIndirect(cmd, r.drawCommand, 0, 1, sizeof(VkDrawIndexedIndirectCommand));
```

Each instance fetches its cube through the list of visible indices — code/u4/particles/cube.vert

```
layout(std430, set = 0, binding = 0) readonly buffer Instances {
    vec4 positionScale[];
};

layout(std430, set = 0, binding = 1) readonly buffer Visible {
    uint visible[];
};

layout(push_constant) uniform Camera {
    mat4 viewProjection;
} camera;

layout(location = 0) out vec3 normal;
layout(location = 1) out vec3 albedo;

void main() {
    uint id = visible[gl_InstanceIndex];
    vec4 s = positionScale[id];
    gl_Position = camera.viewProjection * vec4(inPosition * s.w + s.xyz, 1.0);
    normal = inNormal;
    uint hash = id * 0x9E3779B9u;
    albedo = vec3(uvec3(hash >> 8, hash >> 16, hash >> 24) & 255u) / 255.0 * 0.3 + 0.1;
}
```

The CPU never learns how many cubes were drawn during the frame. To check the cull, the program reads back the last frame's command and list and compares them with a cull computed on the CPU with the same planes. A cube whose sphere touches a plane may legitimately fall either way after rounding, so the check allows disagreement only for those:

Comparing the GPU's cull with the CPU's — code/u4/particles/main.cpp

```
readback.drawCommand.invalidate();
readback.visible.invalidate();
const auto* command = readback.drawCommand.data<VkDrawIndexedIndirectCommand>();
std::vector<uint32_t> gpuVisible(
    readback.visible.data<uint32_t>(),
    readback.visible.data<uint32_t>() + command->instanceCount);
std::sort(gpuVisible.begin(), gpuVisible.end());
const std::array<u4::Vec4, 6> planes = u4::frustumPlanes(viewProjection);
uint32_t cpuCount = 0, disagree = 0, borderline = 0;
for (uint32_t i = 0; i < instanceCount; ++i) {
    const Vec3 centre{instances[i].x, instances[i].y, instances[i].z};
    const float radius = instances[i].w * 1.7320508f;
    const bool cpu = u4::sphereVisible(planes, centre, radius);
    const bool onGpu = std::binary_search(gpuVisible.begin(), gpuVisible.end(), i);
    cpuCount += cpu ? 1 : 0;
    bool nearPlane = false;
    for (const u4::Vec4& q : planes) {
        const float d =
            q.x * centre.x + q.y * centre.y + q.z * centre.z + q.w + radius;
        nearPlane = nearPlane || std::abs(d) < 1e-4f;
    }
    if (cpu != onGpu) ++disagree;
    if (cpu != onGpu && nearPlane) ++borderline;
}
const bool cullOk = command->indexCount == r.cube.indexCount && disagree == borderline;
vkf::print("culling: {} of {} cubes visible on the GPU, {} on the CPU\n",
```

```
            command->instanceCount, instanceCount, cpuCount);
vkf::print("  {} disagree ({} of them touching a frustum plane)\n", disagree,
            borderline);
```

Post-processing in compute

The scene is drawn into a `VK_FORMAT_R16G16B16A16_SFLOAT` image, which can hold values above 1. A million faint additive points pile up at the centre to brightnesses that no 8-bit format can store. The post-process is a compute shader that reads it as a storage image, compresses its range with a simple tone map, applies the sRGB curve itself, since sRGB storage images are not portable (Chapter 1.2), and writes an 8-bit image:

Tone mapping in a compute shader — code/u4/particles/post.comp

```
layout(set = 0, binding = 0, rgba16f) uniform readonly image2D hdrImage;
layout(set = 0, binding = 1, rgba8) uniform writeonly image2D displayImage;

layout(push_constant) uniform Post {
    float exposure;
} post;

vec3 encodeSrgb(vec3 c) {
    vec3 curve = 1.055 * pow(c, vec3(1.0 / 2.4)) - 0.055;
    return mix(c * 12.92, curve, greaterThan(c, vec3(0.0031308)));
}

void main() {
    ivec2 p = ivec2(gl_GlobalInvocationID.xy);
    if (any(greaterThanEqual(p, imageSize(displayImage)))) return;
    vec3 hdr = imageLoad(hdrImage, p).rgb * post.exposure;
    imageStore(displayImage, p, vec4(encodeSrgb(hdr / (1.0 + hdr)), 1.0));
}
```

The barrier into the post-process moves the rendered image from colour attachment writes to compute reads and from `COLOR_ATTACHMENT_OPTIMAL` to `GENERAL`, and prepares the output image in the same call:

From the graphics pass to the compute pass — code/u4/particles/main.cpp

```
void postPass(VkCommandBuffer cmd, const Resources& r, const Pipelines& p) {
    vkf::imageBarrier(cmd, r.hdr, VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL,
                      VK_IMAGE_LAYOUT_GENERAL, VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT,
                      VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT,
                      VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
                      VK_ACCESS_2_SHADER_STORAGE_READ_BIT);
    vkf::imageBarrier(cmd, r.display, VK_IMAGE_LAYOUT_UNDEFINED, VK_IMAGE_LAYOUT_GENERAL,
                      VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT | VK_PIPELINE_STAGE_2_COPY_BIT |
                          VK_PIPELINE_STAGE_2_BLIT_BIT,
                      VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
                      VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
                      VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT);
    vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, p.post);
    vkCmdBindDescriptorSets(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, p.postLayout, 0, 1,
                            &p.postSet, 0, nullptr);
    const float exposure = 1.0f;
    vkCmdPushConstants(cmd, p.postLayout, VK_SHADER_STAGE_COMPUTE_BIT, 0, sizeof(exposure),
                       &exposure);
    vkCmdDispatch(cmd, vkf::groupCount(r.extent.width, 16),
                  vkf::groupCount(r.extent.height, 16), 1);
}
```

With a window, a blit copies and scales the result into the acquired swapchain image. The presenter is told to wait for the acquire semaphore at the blit stage, since the blit is the first thing to touch the image:

A swapchain written by blits — code/u4/particles/main.cpp

```
u4::Presenter presenter(
    ctx, *display,
    {.swapchain = {.srgb = false, .usage = VK_IMAGE_USAGE_TRANSFER_DST_BIT},
     .framesInFlight = framesInFlight,
     .imageWaitStage = VK_PIPELINE_STAGE_2_BLIT_BIT});
```

Blitting the frame into the swapchain image — code/u4/particles/main.cpp

```
void blitToSwapchain(VkCommandBuffer cmd, const Resources& r, const u4::Frame& frame) {
    vkf::imageBarrier(cmd, frame.image, VK_IMAGE_LAYOUT_UNDEFINED,
                      VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL, VK_PIPELINE_STAGE_2_BLIT_BIT,
                      VK_ACCESS_2_NONE, VK_PIPELINE_STAGE_2_BLIT_BIT,
                      VK_ACCESS_2_TRANSFER_WRITE_BIT);
    const VkImageBlit blit{
        .srcSubresource = {VK_IMAGE_ASPECT_COLOR_BIT, 0, 0, 1},
        .srcOffsets = {{0, 0, 0}, {int32_t(r.extent.width), int32_t(r.extent.height), 1}},
        .dstSubresource = {VK_IMAGE_ASPECT_COLOR_BIT, 0, 0, 1},
        .dstOffsets = {{0, 0, 0},
                       {int32_t(frame.extent.width), int32_t(frame.extent.height), 1}},
    };
    vkCmdBlitImage(cmd, r.display, VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL, frame.image,
                   VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL, 1, &blit, VK_FILTER_LINEAR);
    vkf::imageBarrier(cmd, frame.image, VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL,
                      VK_IMAGE_LAYOUT_PRESENT_SRC_KHR, VK_PIPELINE_STAGE_2_BLIT_BIT,
                      VK_ACCESS_2_TRANSFER_WRITE_BIT, VK_PIPELINE_STAGE_2_NONE,
                      VK_ACCESS_2_NONE);
}
```

Results

The program checks all three kinds of output. The particles' angular momenta must be unchanged to within 10^{-3}, relative, and a sample of a thousand particles must follow the same orbits as the CPU's integration of the same steps. The cull must match the CPU's, and every pixel of the tone-mapped image must match a CPU tone map of the rendered image.

Checking conservation and a sample of orbits against the CPU — code/u4/particles/main.cpp

```
readback.particles.invalidate();
const Particle* gpu = readback.particles.data<Particle>();
double worstMomentum = 0;
for (uint32_t i = 0; i < particleCount; ++i) {
    const Vec3 before = u4::angularMomentum(initial[i]);
    const Vec3 after = u4::angularMomentum(gpu[i]);
    worstMomentum = std::max(worstMomentum, double(u4::length(after - before)) /
                                            double(u4::length(before)));
}
double worstPosition = 0;
const uint32_t stride = std::max(1u, particleCount / 1000);
for (uint32_t i = 0; i < particleCount; i += stride) {
    Particle reference = initial[i];
    for (uint32_t step = 0; step < recorded; ++step) u4::stepOnCpu(reference, Gravity);
    const Vec3 a{gpu[i].position[0], gpu[i].position[1], gpu[i].position[2]};
    const Vec3 b{reference.position[0], reference.position[1], reference.position[2]};
    worstPosition = std::max(worstPosition, double(u4::length(a - b)));
}
const bool particlesOk = worstMomentum < 1e-3 && worstPosition < 1e-3;
vkf::print("particles: angular momentum kept to {:.1e} (largest relative change)\n",
           worstMomentum);
vkf::print("  sampled orbits within {:.1e} of the CPU's\n", worstPosition);
```

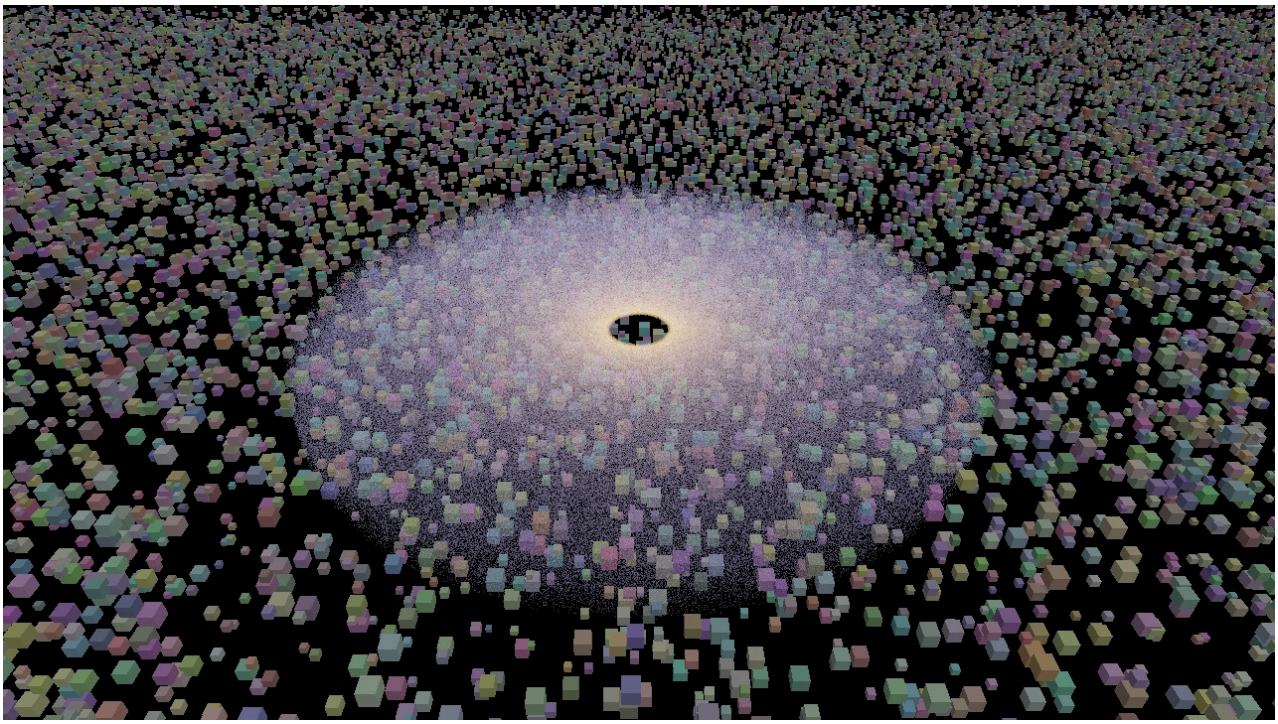


Figure 4.4.2. The last frame: a million particles orbiting the origin, among the cubes that survived culling.

Sixty frames offscreen on the M2 Pro, with validation on

```
$ ./build/bin/u4_particles
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
1000000 particles (31 MB), 50000 cubes to cull, 1280x720, 2 frames in flight
60 frames: CPU frame time median 2.59 ms
GPU time per pass, median of 60 frames:
  simulate        0.304 ms
  cull            0.017 ms
  draw            2.243 ms
  post-process    0.030 ms
  frame           2.595 ms
particles: angular momentum kept to 4.3e-06 (largest relative change)
  sampled orbits within 2.4e-07 of the CPU's
culling: 14941 of 50000 cubes visible on the GPU, 14941 on the CPU
  0 disagree (0 of them touching a frustum plane)
post-process: every pixel within 0 of the CPU's tone map; 65% of pixels lit
wrote particles.png
PASS particles, GPU culling and post-processing
```

Drawing takes most of the frame. A million points and fifteen thousand cubes take 2.2 ms to draw, against 0.3 ms to simulate the million particles and 0.017 ms to cull fifty thousand cubes. The cull costs far less than it saves, because it removes 70% of the cubes before any vertex is processed. On a GPU with a dedicated compute queue, the simulation of the next frame could also run alongside the current frame's drawing, which Chapter 4.7 does.

> KEY IDEA
> Compute and graphics share memory, queues and synchronisation rules. A buffer written by a compute shader can be a vertex buffer, an index buffer or an indirect command, and an image rendered by the graphics pipeline can be a compute shader's input. Joining them needs only a barrier whose destination is the stage that reads the data.

COMING FROM OPENGL

OpenGL 4.3 can do all of this: compute shaders write a buffer bound as a vertex buffer, `glDrawElementsIndirect` reads a command from a buffer, and `glMemoryBarrier` takes `GL_VERTEX_ATTRIB_ARRAY_BARRIER_BIT` or `GL_COMMAND_BARRIER_BIT` before the draw. The differences are in control. Drivers usually implement OpenGL's barrier by waiting for all earlier work, and it cannot express that the draw's other inputs are unaffected, and cannot put the compute work on another queue.

Reading

- The Vulkan specification, "Drawing Commands" (in particular "Indirect Drawing") and the graphics stages in "Synchronization and Cache Control".
- The Vulkan Guide, "Synchronization Examples": the compute-to-graphics cases.
- Arm, *Mali GPU Best Practices Developer Guide*, on compute and graphics interaction on tile-based GPUs.

Labs

LAB 4.4.1 Run and time the frame — 1 hour

Goal. See where the frame's time goes.

Code. `code/u4/particles`

1. Run `VKF_VALIDATION=0 build/bin/u4_particles` and record the time of each pass.
2. Run with `--n` set to 250,000 and 4,000,000 and record how each pass scales.

DONE WHEN

- [ ] Every run passes its checks.
- [ ] You can say which pass grows with the number of particles and which do not, and why.

EVIDENCE FOR THE NOTEBOOK

- The table of pass times.

LAB 4.4.2 Remove each barrier — 1.5 hours

Goal. Connect each barrier to the hazard it prevents.

Code. `code/u4/particles`

1. Remove, one at a time, the second barrier of the simulation pass, the barrier after the command's reset, and the barrier into the post-process.
2. Run each with validation on and record the report and whether the checks still pass.

DONE WHEN

- [ ] You have the three reports.
- [ ] Each is explained by the hazard type and the stages it names.

EVIDENCE FOR THE NOTEBOOK

- The reports and explanations.

LAB 4.4.3 Cull more finely — 2 hours

Goal. Extend GPU culling.

Code. `code/u4/particles`

1. Add a distance test to the cull: cubes beyond a given distance are not drawn.
2. Count, with a second atomic counter, how many cubes each test rejected, and read the counts back.
3. Check the new cull against the CPU.

DONE WHEN

- [ ] The GPU's counts match the CPU's.
- [ ] You have the draw pass's time with and without the distance test.

EVIDENCE FOR THE NOTEBOOK

- The changes and the timings.

LAB 4.4.4 Exposure and the HDR image — 1.5 hours

Goal. Use the high-dynamic-range image.

Code. `code/u4/particles`

1. Make the exposure a command-line option and render with exposures of 0.25, 1 and 4.
2. Replace the tone map with plain clamping and compare the centre of the image.
3. Update the CPU's reference so that the check still passes.

DONE WHEN

- [ ] Every variant passes its pixel check.
- [ ] You can explain what the tone map preserves that clamping loses.

EVIDENCE FOR THE NOTEBOOK

- The images and the explanation.

Problem set

1. Write the barrier that would be needed if the particles were drawn as instanced quads, reading positions in the vertex shader from a storage buffer instead of as vertex attributes.
2. Why must the draw command's instance count be reset each frame, and why with `vkCmdUpdateBuffer` and not a compute shader? Is there another way?
3. The cull appends visible indices with an atomic counter, so their order varies from frame to frame. Does that matter for this scene? When would it?
4. Estimate the vertex shader invocations saved per frame by culling, and the cost of the cull itself, from the measurements above.
5. The simulation pass's first barrier has two source stages. What would go wrong with only `COMPUTE_SHADER`, and with only `VERTEX_ATTRIBUTE_INPUT`?

COMPLETION CHECKLIST

- [ ] I can use one buffer as both a storage buffer and a vertex buffer, with the barriers between the roles.
- [ ] I can cull on the GPU and draw with indirect parameters a compute shader wrote.
- [ ] I can render to a high-dynamic-range image and post-process it in compute.
- [ ] I can check GPU-driven rendering against the CPU, allowing exactly for legitimate differences.
- [ ] Lab 4.4.1–4.4.4 done-when criteria all hold, with evidence filed.

CHAPTER 4.5

From OpenGL to Vulkan

TIME	6 hours
YOU BUILD	One scene of 10,000 turning cubes, drawn by an OpenGL 4.1 renderer and a Vulkan renderer behind the same interface and compared pixel by pixel, with the CPU cost of each API's draws, submission and frame measured; and a catalogue that translates the OpenGL the course has met, compute shaders and `glMemoryBarrier` included, into Vulkan.
NEEDS	Chapters 4.1 to 4.4, and the Handbook's part on Vulkan and OpenGL.

WHY IT MATTERS

Much Vulkan code replaces OpenGL code that already works. The new renderer must draw the same pictures as the old one, and the port is worth its effort only if it removes a cost that the old one could not avoid. This chapter ports a small renderer while keeping both versions alive behind one interface, so that every step can be checked against the original, and measures where each spends its CPU time. It then collects the translations a larger port needs, including those for OpenGL's compute shaders, which macOS cannot run but most OpenGL code on other platforms uses.

SKIP TEST

1. An OpenGL projection matrix is used unchanged in Vulkan. What happens to the image, and to depth?
2. Which OpenGL calls does `vkCmdPushConstants` replace in a port, and why is it cheaper?
3. Which `glMemoryBarrier` bit corresponds to a Vulkan barrier whose destination is `DRAW_INDIRECT` with `INDIRECT_COMMAND_READ`?
4. An OpenGL compute shader writes a buffer that `glCopyBufferSubData` then copies. Which barrier does OpenGL need there, and which does Vulkan?

If all four are easy, read the two catalogues and do Labs 4.5.2 and 4.5.4.

Core ideas

One scene, two renderers

The program draws a grid of 10,000 small cubes, each turning about its own axis, into a 512 × 512 image, with one draw call per cube. That is the hardest case for an OpenGL driver, and a common one in real programs. Both renderers take the same list of draws, a matrix and a colour for each cube, and both produce an RGBA image with its first row at the top:

Everything a draw needs, in either API — code/u4/glport/scene.hpp

```
struct DrawData {
    u4::Mat4 mvp;
    float color[4];
};
```

The same frames for both renderers — code/u4/glport/main.cpp

```
template <class Renderer>
Measured runFrames(Renderer& renderer, uint32_t objects, uint32_t warmup, uint32_t frames,
                   const u4::Mat4& projection) {
    Measured measured;
    for (uint32_t frame = 0; frame < warmup + frames; ++frame) {
        const std::vector<glport::DrawData> draws =
            glport::buildDraws(objects, frame, projection);
        const glport::FrameTiming timing = renderer.renderFrame(draws);
        if (frame ⩾ warmup) measured.add(timing);
    }
    return measured;
}
```

The OpenGL renderer uses the core profile of OpenGL 4.1, the last version macOS supports, so that the program runs on every platform the course does. OpenGL cannot draw without a context, and on most platforms a context needs a window, here an invisible one that GLFW creates. The Vulkan renderer needs neither window nor surface.

An OpenGL 4.1 core context in an invisible window — code/u4/glport/gl_renderer.cpp

```
GlRenderer::GlRenderer(uint32_t width, uint32_t height, const u4::Mesh& mesh)
    : width_(width), height_(height) {
    if (!glfwInit()) throw std::runtime_error("glfwInit failed");
    glfwWindowHint(GLFW_VISIBLE, GLFW_FALSE);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 4);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 1);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
    glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GLFW_TRUE);
    window_ = glfwCreateWindow(64, 64, "u4_glport", nullptr, nullptr);
    if (window_ == nullptr) {
        glfwTerminate();
        throw std::runtime_error("cannot create an OpenGL 4.1 core context");
    }
    glfwMakeContextCurrent(window_);
    glfwSwapInterval(0);
    gl_ = loadGl();
```

Beyond version 1.1, OpenGL's functions must be looked up at run time on Windows, and the set available depends on the context, which is why libraries such as GLAD and GLEW exist. The port uses a table of its own, filled through GLFW. Vulkan's loader exports every core function, as Chapter 1.1 showed, so the Vulkan renderer needs no table.

The OpenGL functions the port calls, loaded into a table — code/u4/glport/gl_api.hpp

```
#define GLPORT_FUNCTIONS(X)                                                              \
    X(GenVertexArrays) X(BindVertexArray) X(DeleteVertexArrays) X(GenBuffers) X(BindBuffer)   \
    X(BufferData) X(DeleteBuffers) X(VertexAttribPointer) X(EnableVertexAttribArray)         \
    X(CreateShader) X(ShaderSource) X(CompileShader) X(GetShaderiv) X(GetShaderInfoLog)      \
    X(DeleteShader) X(CreateProgram) X(AttachShader) X(LinkProgram) X(GetProgramiv)          \
    X(GetProgramInfoLog) X(DeleteProgram) X(UseProgram) X(GetUniformLocation)                \
    X(UniformMatrix4fv) X(Uniform4fv) X(GenFramebuffers) X(BindFramebuffer)                  \
    X(FramebufferRenderbuffer) X(CheckFramebufferStatus) X(DeleteFramebuffers)               \
    X(GenRenderbuffers) X(BindRenderbuffer) X(RenderbufferStorage) X(DeleteRenderbuffers)    \
    X(Viewport) X(ClearColor) X(ClearDepth) X(Clear) X(Enable) X(DepthFunc) X(DrawElements)  \
    X(ReadPixels) X(PixelStorei) X(FenceSync) X(ClientWaitSync) X(DeleteSync) X(Flush)       \
    X(GetError) X(GetString)

struct GlApi {
#define GLPORT_DECLARE(name) decltype(&::gl##name) name = nullptr;
    GLPORT_FUNCTIONS(GLPORT_DECLARE)
#undef GLPORT_DECLARE
};

GlApi loadGl();
```

The port, piece by piece

The vertex and index buffers translate most directly. OpenGL chooses where the data lives from a usage hint, and records the vertex format in a *vertex array object*. Vulkan places the buffers in device-local memory through a staging copy, and the vertex format moves into the pipeline (Chapter 4.1).

OpenGL: buffers, and the vertex format in a vertex array object — code/u4/glport/gl_renderer.cpp

```
gl_.GenVertexArrays(1, &vertexArray_);
gl_.BindVertexArray(vertexArray_);
gl_.GenBuffers(1, &vertexBuffer_);
gl_.BindBuffer(GL_ARRAY_BUFFER, vertexBuffer_);
gl_.BufferData(GL_ARRAY_BUFFER, GLsizeiptr(mesh.vertices.size() * sizeof(u4::MeshVertex)),
               mesh.vertices.data(), GL_STATIC_DRAW);
gl_.GenBuffers(1, &indexBuffer_);
gl_.BindBuffer(GL_ELEMENT_ARRAY_BUFFER, indexBuffer_);
gl_.BufferData(GL_ELEMENT_ARRAY_BUFFER, GLsizeiptr(mesh.indices.size() * sizeof(uint16_t)),
               mesh.indices.data(), GL_STATIC_DRAW);
gl_.VertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, sizeof(u4::MeshVertex),
                        reinterpret_cast<const void*>(offsetof(u4::MeshVertex, position)));
gl_.VertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, sizeof(u4::MeshVertex),
                        reinterpret_cast<const void*>(offsetof(u4::MeshVertex, normal)));
gl_.EnableVertexAttribArray(0);
gl_.EnableVertexAttribArray(1);
indexCount_ = GLsizei(mesh.indices.size());
```

Vulkan: device-local buffers, uploaded through a staging buffer — code/u4/glport/vk_renderer.cpp

```
const VkDeviceSize vertexBytes = mesh.vertices.size() * sizeof(u4::MeshVertex);
const VkDeviceSize indexBytes = mesh.indices.size() * sizeof(uint16_t);
vertices_ = vkf::createBuffer(
    ctx, vertexBytes, VK_BUFFER_USAGE_VERTEX_BUFFER_BIT | VK_BUFFER_USAGE_TRANSFER_DST_BIT,
    vkf::MemoryUse::DeviceLocal, "cube vertices");
indices_ = vkf::createBuffer(
    ctx, indexBytes, VK_BUFFER_USAGE_INDEX_BUFFER_BIT | VK_BUFFER_USAGE_TRANSFER_DST_BIT,
    vkf::MemoryUse::DeviceLocal, "cube indices");
vkf::upload(ctx, vertices_, mesh.vertices.data(), vertexBytes);
vkf::upload(ctx, indices_, mesh.indices.data(), indexBytes);
indexCount_ = static_cast<uint32_t>(mesh.indices.size());
```

The shaders change more. OpenGL compiles GLSL source when the program runs and links the stages into a *program object*, whose loose `uniform` variables the renderer finds by name. Vulkan's GLSL has no loose uniforms, so the matrix and colour become a push-constant block, and the shaders are compiled to SPIR-V when the program is built.

OpenGL: GLSL 4.10, compiled at run time — code/u4/glport/gl_renderer.cpp

```
const char* const VertexSource = R"(#version 410 core
layout(location = 0) in vec3 inPosition;
layout(location = 1) in vec3 inNormal;
uniform mat4 mvp;
uniform vec4 tint;
out vec3 faceColor;
void main() {
    gl_Position = mvp * vec4(inPosition, 1.0);
    faceColor = (inNormal * 0.5 + 0.5) * tint.rgb;
}
)";

const char* const FragmentSource = R"(#version 410 core
in vec3 faceColor;
layout(location = 0) out vec4 outColor;
void main() {
    outColor = vec4(faceColor, 1.0);
}
)";
```

Vulkan: the same vertex shader, with a push-constant block — code/u4/glport/scene.vert

```
#version 460

layout(location = 0) in vec3 inPosition;
layout(location = 1) in vec3 inNormal;

layout(push_constant) uniform DrawData {
    mat4 mvp;
    vec4 tint;
} draw;

layout(location = 0) out vec3 faceColor;

void main() {
    gl_Position = draw.mvp * vec4(inPosition, 1.0);
    faceColor = (inNormal * 0.5 + 0.5) * draw.tint.rgb;
}
```

OpenGL: compiling, linking, and finding the uniforms — code/u4/glport/gl_renderer.cpp

```
auto compile = [&](GLenum stage, const char* source) {
    const GLuint shader = gl_.CreateShader(stage);
    gl_.ShaderSource(shader, 1, &source, nullptr);
    gl_.CompileShader(shader);
    GLint ok = GL_FALSE;
    gl_.GetShaderiv(shader, GL_COMPILE_STATUS, &ok);
    if (!ok) {
        char log[1024] = {};
        gl_.GetShaderInfoLog(shader, sizeof(log), nullptr, log);
        throw std::runtime_error(std::string("GLSL compile error: ") + log);
    }
    return shader;
};
const GLuint vertexShader = compile(GL_VERTEX_SHADER, VertexSource);
const GLuint fragmentShader = compile(GL_FRAGMENT_SHADER, FragmentSource);
program_ = gl_.CreateProgram();
gl_.AttachShader(program_, vertexShader);
gl_.AttachShader(program_, fragmentShader);
gl_.LinkProgram(program_);
GLint linked = GL_FALSE;
gl_.GetProgramiv(program_, GL_LINK_STATUS, &linked);
if (!linked) throw std::runtime_error("GLSL link error");
gl_.DeleteShader(vertexShader);
gl_.DeleteShader(fragmentShader);
mvpLocation_ = gl_.GetUniformLocation(program_, "mvp");
colorLocation_ = gl_.GetUniformLocation(program_, "tint");
```

Vulkan: a push-constant range, a layout and a pipeline — code/u4/glport/vk_renderer.cpp

```
const VkPushConstantRange push{VK_SHADER_STAGE_VERTEX_BIT, 0, sizeof(DrawData)};
layout_ = vkf::createPipelineLayout(ctx, {}, {push});
const auto vertexShader = vkf::loadShader(ctx, vkf::shaderPath("scene.vert"));
const auto fragmentShader = vkf::loadShader(ctx, vkf::shaderPath("scene.frag"));
const u4::MeshInput input = u4::meshInput({0, 1});
const VkFormat depthFormat = u4::chooseDepthFormat(ctx);
pipeline_ =
    u4::createGraphicsPipeline(ctx, {.layout = layout_,
                                     .vertexShader = vertexShader,
                                     .fragmentShader = fragmentShader,
                                     .vertexBindings = std::span(&input.binding, 1),
                                     .vertexAttributes = input.attributes,
                                     .colorFormat = ColorFormat,
                                     .depthFormat = depthFormat,
                                     .name = "scene"});
```

OpenGL renders offscreen into a *framebuffer object* with *renderbuffers* attached, and enables depth testing and culling as global state. In Vulkan those states are in the pipeline, already created, and the targets are images that dynamic rendering clears through their load operations:

OpenGL: a framebuffer object, and state switched on — code/u4/glport/gl_renderer.cpp

```
gl_.GenRenderbuffers(1, &colorBuffer_);
gl_.BindRenderbuffer(GL_RENDERBUFFER, colorBuffer_);
gl_.RenderbufferStorage(GL_RENDERBUFFER, GL_RGBA8, GLsizei(width), GLsizei(height));
gl_.GenRenderbuffers(1, &depthBuffer_);
gl_.BindRenderbuffer(GL_RENDERBUFFER, depthBuffer_);
gl_.RenderbufferStorage(GL_RENDERBUFFER, GL_DEPTH_COMPONENT32F, GLsizei(width),
                        GLsizei(height));
gl_.GenFramebuffers(1, &framebuffer_);
gl_.BindFramebuffer(GL_FRAMEBUFFER, framebuffer_);
gl_.FramebufferRenderbuffer(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0, GL_RENDERBUFFER,
                            colorBuffer_);
gl_.FramebufferRenderbuffer(GL_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, GL_RENDERBUFFER,
                            depthBuffer_);
if (gl_.CheckFramebufferStatus(GL_FRAMEBUFFER) != GL_FRAMEBUFFER_COMPLETE) {
    throw std::runtime_error("the OpenGL framebuffer is incomplete");
}
gl_.Enable(GL_DEPTH_TEST);
gl_.DepthFunc(GL_LESS);
gl_.Enable(GL_CULL_FACE);
```

Vulkan: attachments that clear on load — code/u4/glport/vk_renderer.cpp

```
const VkRenderingAttachmentInfo color{
    .sType = VK_STRUCTURE_TYPE_RENDERING_ATTACHMENT_INFO,
    .imageView = color_.view,
    .imageLayout = VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL,
    .loadOp = VK_ATTACHMENT_LOAD_OP_CLEAR,
    .storeOp = VK_ATTACHMENT_STORE_OP_STORE,
    .clearValue = {.color = Background},
};
const VkRenderingAttachmentInfo depth{
    .sType = VK_STRUCTURE_TYPE_RENDERING_ATTACHMENT_INFO,
    .imageView = depth_.view,
    .imageLayout = VK_IMAGE_LAYOUT_DEPTH_ATTACHMENT_OPTIMAL,
    .loadOp = VK_ATTACHMENT_LOAD_OP_CLEAR,
    .storeOp = VK_ATTACHMENT_STORE_OP_DONT_CARE,
    .clearValue = {.depthStencil = {1.0f, 0}},
};
const VkRenderingInfo rendering{
    .sType = VK_STRUCTURE_TYPE_RENDERING_INFO,
    .renderArea = {{0, 0}, extent_},
    .layerCount = 1,
    .colorAttachmentCount = 1,
    .pColorAttachments = &color,
    .pDepthAttachment = &depth,
};
vkCmdBeginRendering(cmd_, &rendering);
```

The two frames have the same structure and differ mainly in cost. For each cube, OpenGL sets two uniforms and draws, while Vulkan pushes one block of constants and draws. The Vulkan frame also records into a command buffer that it then submits, whereas OpenGL's driver keeps a command buffer of its own and submits it when it chooses.

OpenGL: a frame of draws — code/u4/glport/gl_renderer.cpp

```
gl_.BindFramebuffer(GL_FRAMEBUFFER, framebuffer_);
gl_.Viewport(0, 0, GLsizei(width_), GLsizei(height_));
gl_.ClearColor(0.10f, 0.10f, 0.15f, 1.0f);
gl_.ClearDepth(1.0);
gl_.Clear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
gl_.UseProgram(program_);
gl_.BindVertexArray(vertexArray_);
for (const DrawData& draw : draws) {
    gl_.UniformMatrix4fv(mvpLocation_, 1, GL_FALSE, &draw.mvp.m[0][0]);
    gl_.Uniform4fv(colorLocation_, 1, draw.color);
    gl_.DrawElements(GL_TRIANGLES, indexCount_, GL_UNSIGNED_SHORT, nullptr);
}
```

Vulkan: the same frame, recorded into a command buffer — code/u4/glport/vk_renderer.cpp

```
VKF_CHECK(vkResetCommandPool(ctx_->device(), pool_, 0));
vkf::beginCommands(cmd_);
beginRendering();
u4::setViewportAndScissor(cmd_, extent_);
vkCmdBindPipeline(cmd_, VK_PIPELINE_BIND_POINT_GRAPHICS, pipeline_);
const VkDeviceSize offset = 0;
vkCmdBindVertexBuffers(cmd_, 0, 1, &vertices_.buffer, &offset);
vkCmdBindIndexBuffer(cmd_, indices_, 0, VK_INDEX_TYPE_UINT16);
for (const DrawData& draw : draws) {
    vkCmdPushConstants(cmd_, layout_, VK_SHADER_STAGE_VERTEX_BIT, 0, sizeof(draw), &draw);
    vkCmdDrawIndexed(cmd_, indexCount_, 1, 0, 0, 0);
}
vkCmdEndRendering(cmd_);
vkf::endCommands(cmd_);
```

To measure each frame to completion, both renderers wait for the GPU, OpenGL with a *sync object* and Vulkan with a fence:

OpenGL: a fence in the command stream, flushed and waited on — code/u4/glport/gl_renderer.cpp

```
const GLsync fence = gl_.FenceSync(GL_SYNC_GPU_COMMANDS_COMPLETE, 0);
gl_.Flush();
timing.submitMs = timer.elapsedMs() - timing.issueMs;
const GLenum waited =
    gl_.ClientWaitSync(fence, GL_SYNC_FLUSH_COMMANDS_BIT, 10'000'000'000);
gl_.DeleteSync(fence);
if (waited != GL_ALREADY_SIGNALED && waited != GL_CONDITION_SATISFIED) {
    throw std::runtime_error("glClientWaitSync did not see the frame finish");
}
```

Vulkan: a submission with a fence — code/u4/glport/vk_renderer.cpp

```
vkf::submit(ctx_->mainQueue(), std::span(&cmd_, 1), {}, {}, fence_);
timing.submitMs = timer.elapsedMs() - timing.issueMs;
vkf::waitFence(*ctx_, fence_);
VKF_CHECK(vkResetFences(ctx_->device(), 1, fence_.ptr()));
```

The two APIs also order an image's rows differently. `glReadPixels` returns the bottom row first, so the OpenGL renderer reverses the rows before returning them.

OpenGL: one call, bottom row first — code/u4/glport/gl_renderer.cpp

```
std::vector<uint8_t> bottomUp(size_t(width_) * height_ * 4);
gl_.BindFramebuffer(GL_FRAMEBUFFER, framebuffer_);
gl_.PixelStorei(GL_PACK_ALIGNMENT, 1);
gl_.ReadPixels(0, 0, GLsizei(width_), GLsizei(height_), GL_RGBA, GL_UNSIGNED_BYTE,
               bottomUp.data());
```

Vulkan: a layout transition, a copy, and a barrier to the host — code/u4/glport/vk_renderer.cpp

```
vkf::submitNow(*ctx_, [&](VkCommandBuffer cmd) {
    vkf::imageBarrier(cmd, color_, VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL,
                      VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL,
                      VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT,
                      VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT, VK_PIPELINE_STAGE_2_COPY_BIT,
                      VK_ACCESS_2_TRANSFER_READ_BIT);
    const VkBufferImageCopy region{
        .imageSubresource = {VK_IMAGE_ASPECT_COLOR_BIT, 0, 0, 1},
        .imageExtent = {extent_.width, extent_.height, 1},
    };
    vkCmdCopyImageToBuffer(cmd, color_, VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL, readback_, 1,
                           &region);
    vkf::bufferBarrier(cmd, readback_, VK_PIPELINE_STAGE_2_COPY_BIT,
                       VK_ACCESS_2_TRANSFER_WRITE_BIT, VK_PIPELINE_STAGE_2_HOST_BIT,
                       VK_ACCESS_2_HOST_READ_BIT);
});
readback_.invalidate();
return {readback_.data<uint8_t>(), readback_.data<uint8_t>() + readback_.size};
```

Clip space

The scene's matrices were written for OpenGL, whose normalised device coordinates have +y pointing up and depth from −1 to 1. Vulkan's have +y pointing down and depth from 0 to 1 (Chapter 4.1). The port keeps the OpenGL projection and multiplies it by a matrix that converts one clip space to the other:

OpenGL's projection, and the conversion to Vulkan's clip space — code/u4/glport/scene.hpp

```
inline u4::Mat4 perspectiveOpenGl(float fovY, float aspect, float zNear, float zFar) {
    const float f = 1.0f / std::tan(fovY / 2);
    u4::Mat4 r;
    r.m[0][0] = f / aspect;
    r.m[1][1] = f;
    r.m[2][2] = (zFar + zNear) / (zNear - zFar);
    r.m[2][3] = -1;
    r.m[3][2] = 2 * zFar * zNear / (zNear - zFar);
    r.m[3][3] = 0;
    return r;
}

// Maps OpenGL clip space (+y up, depth -1 to 1) to Vulkan's (+y down, depth 0 to 1).
inline u4::Mat4 vulkanFromOpenGlClip() {
    u4::Mat4 r;
    r.m[1][1] = -1;
    r.m[2][2] = 0.5f;
    r.m[3][2] = 0.5f;
    return r;
}
```

One projection for each renderer — code/u4/glport/main.cpp

```
const u4::Mat4 glProjection =
    glport::perspectiveOpenGl(u4::Pi / 4, 1.0f, 0.5f, 100.0f);
const u4::Mat4 vkProjection = glport::vulkanFromOpenGlClip() * glProjection;
const Measured glTimes = runFrames(gl, objects, warmup, frames, glProjection);
const Measured vkTimes = runFrames(vk, objects, warmup, frames, vkProjection);
```

The conversion leaves the image the same way up as OpenGL's, so every triangle keeps its winding on the screen, and the pipeline keeps OpenGL's counter-clockwise front face and back-face culling. Two alternatives suit ports that cannot change their matrices. A viewport with a negative height, allowed since Vulkan 1.1, flips y in the viewport transform. The extension `VK_EXT_depth_clip_control` lets a pipeline accept OpenGL's depth range of −1 to 1.

Checking the port

The two images are compared pixel by pixel. Rasterisation rules leave implementations some freedom, so the check tolerates differences of two levels in a channel, and up to 0.5% of pixels beyond that. It also requires the cubes to cover a quarter of the image, so that two empty images cannot agree:

Comparing the two images — code/u4/glport/main.cpp

```
const std::vector<uint8_t> glPixels = gl.readPixels();
const std::vector<uint8_t> vkPixels = vk.readPixels();
size_t differing = 0, covered = 0;
for (size_t i = 0; i < glPixels.size(); i += 4) {
    int worst = 0;
    for (size_t c = 0; c < 3; ++c) {
        worst = std::max(worst, std::abs(int(glPixels[i + c]) - int(vkPixels[i + c])));
    }
    differing += worst > 2 ? 1 : 0;
    covered += vkPixels[i] != 26 || vkPixels[i + 1] != 26 || vkPixels[i + 2] != 38;
}
const size_t pixels = glPixels.size() / 4;
const double share = 100.0 * double(differing) / double(pixels);
vkf::print(
    "images: {} of {} pixels ({:.2f}%) differ by more than 2 levels; cubes "
    "cover {:.0f}%\n",
    differing, pixels, share, 100.0 * double(covered) / double(pixels));
```

Thirty frames on the M2 Pro, with validation on

```
$ ./build/bin/u4_glport
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
OpenGL: 4.1 Metal - 90.5
10000 draws per frame, 512x512, median of 30 frames after 5 warm-up frames
                            OpenGL 4.1      Vulkan
  CPU, draw calls              6.95 ms     16.90 ms
  CPU, submit                  0.03 ms      2.30 ms
  frame, to completion         7.60 ms     21.22 ms
  (the Vulkan times include the validation layers: measure with VKF_VALIDATION=0)
images: 2 of 262144 pixels (0.00%) differ by more than 2 levels; cubes cover 57%
wrote glport-gl.png and glport-vk.png
PASS the OpenGL and Vulkan images agree
```

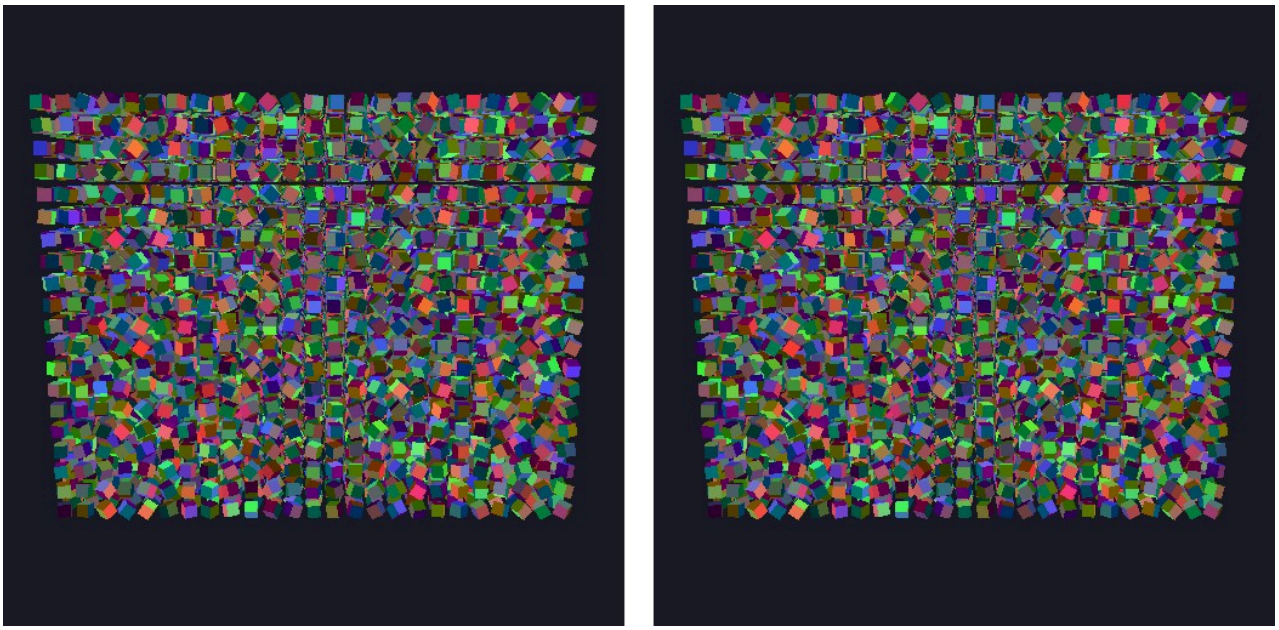


Figure 4.5.1. The scene as OpenGL (left) and Vulkan (right) drew it. The two images agree to within two levels in all but 2 of 262,144 pixels.

Where the CPU time goes

Two hundred frames on the M2 Pro, with validation off

```
$ VKF_VALIDATION=0 ./build/bin/u4_glport --frames 200 --warmup 20
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
OpenGL: 4.1 Metal - 90.5
10000 draws per frame, 512x512, median of 200 frames after 20 warm-up frames
                          OpenGL 4.1      Vulkan
  CPU, draw calls             6.89 ms      0.43 ms
  CPU, submit                 0.02 ms      0.96 ms
  frame, to completion        7.57 ms      3.28 ms
images: 0 of 262144 pixels (0.00%) differ by more than 2 levels; cubes cover 57%
wrote glport-gl.png and glport-vk.png
PASS the OpenGL and Vulkan images agree
```

OpenGL spends 6.9 ms issuing 10,000 draws, 0.69 µs per draw for two uniform updates and a draw call. Vulkan records them in 0.43 ms, 43 ns per draw for a push constant and a draw, and submits them in 0.96 ms. The frame completes in 3.3 ms against OpenGL's 7.6 ms. At each draw an OpenGL driver checks the call, works out what state has changed since the last draw and translates it for the hardware. Vulkan did that work once, when it created the pipeline.

On this machine both APIs run on Metal. Apple implements OpenGL on Metal, as its version string shows, and MoltenVK translates Vulkan to Metal. MoltenVK records Vulkan commands into a list of its own and encodes them for Metal when the command buffer is submitted, which is why submission here costs more than recording. A native driver does most of that work as the commands are recorded. The ratio between the two APIs differs from driver to driver, but the cost of a Vulkan draw is lower than an OpenGL draw's on every platform, and more predictable.

With validation on, recording takes 16.9 ms, about forty times longer, because the layer checks every command. Measure performance with validation off, and keep it on while developing.

> PITFALL
> OpenGL's driver submits work while the program is still issuing draws, so its frame overlaps the CPU's work with the GPU's. A port that records a whole frame, submits it and waits loses that overlap. Frames in flight (Chapter 3.6) restore it by recording the next frame while the GPU executes this one.

A catalogue for larger ports

The port above meets a few of the differences between the APIs. Table 4.5.1 lists those that change results in larger ports, where a direct translation of each call would still draw the wrong picture.

Topic	OpenGL	Vulkan
Clip space	+y up, depth from −1 to 1	+y down, depth from 0 to 1
Image rows	The first row of a framebuffer, and of `glReadPixels`, is the bottom one	The first row is the top one
Shader resources	Loose `uniform` variables found by name; `binding` alone	Blocks only: push constants, or uniform buffers with `set` and `binding`
Built-ins	`gl_VertexID`; `gl_InstanceID` counts from zero in every draw	`gl_VertexIndex`; `gl_InstanceIndex` starts at the draw's first instance
Default state	Depth test, culling and blending off until enabled	Everything stated in the pipeline
sRGB output	Encoded only with `GL_FRAMEBUFFER_SRGB` enabled	Decided by the attachment's format
Point size	`glPointSize`, or `gl_PointSize` with `GL_PROGRAM_POINT_SIZE` enabled	The vertex shader must write `gl_PointSize`
Shader compilation	GLSL compiled by the driver when the program runs	SPIR-V compiled at build time, pipelines created at start-up

Table 4.5.1. Differences that change results

The larger difference is synchronisation. OpenGL's driver sees every access it performs itself (rendering, copies, clears, uploads and readbacks) and orders them without being asked. It cannot see the writes that shaders make through storage buffers, images and atomic counters, so OpenGL requires `glMemoryBarrier` after those writes and no others. Its bits name the kind of *later* access that must see the writes. A Vulkan barrier needs both sides: the source is the stage of the shader that wrote, with `SHADER_STORAGE_WRITE`, and Table 4.5.2 gives the destination for each bit.

`glMemoryBarrier` bit	Later access	Vulkan destination
`SHADER_STORAGE`	Storage buffer reads and writes	The reading shader's stage, `SHADER_STORAGE_READ` and `SHADER_STORAGE_WRITE`
`ATOMIC_COUNTER`	Atomic counters	The same as `SHADER_STORAGE`
`SHADER_IMAGE_ACCESS`	Image loads, stores and atomics	The shader's stage, `SHADER_STORAGE_READ` and `SHADER_STORAGE_WRITE`, with the image in `GENERAL`
`TEXTURE_FETCH`	Sampling	The shader's stage, `SHADER_SAMPLED_READ`, with the image in `SHADER_READ_ONLY_OPTIMAL`
`UNIFORM`	Uniform buffer reads	The shader's stage, `UNIFORM_READ`
`VERTEX_ATTRIB_ARRAY`	Vertex attribute fetch	`VERTEX_ATTRIBUTE_INPUT`, `VERTEX_ATTRIBUTE_READ`
`ELEMENT_ARRAY`	Index fetch	`INDEX_INPUT`, `INDEX_READ`
`COMMAND`	Indirect draw and dispatch parameters	`DRAW_INDIRECT`, `INDIRECT_COMMAND_READ`
`BUFFER_UPDATE`, `TEXTURE_UPDATE`, `PIXEL_BUFFER`	Copies, uploads and readbacks	`COPY`, `TRANSFER_READ` or `TRANSFER_WRITE`, then `HOST` with `HOST_READ` for the CPU
`FRAMEBUFFER`	Rendering, blits and `glReadPixels` through a framebuffer	`COLOR_ATTACHMENT_OUTPUT` or the fragment tests, with attachment access and layout, or `BLIT`
`CLIENT_MAPPED_BUFFER`	CPU reads through a persistent mapping	`HOST`, `HOST_READ`, after a fence

Table 4.5.2. `glMemoryBarrier` bits, without their `GL_` prefix and `_BARRIER_BIT` suffix, and the Vulkan barriers that replace them

Compute shaders arrived in OpenGL 4.3, which macOS lacks. The listing below is the frame of Chapter 4.4 as an OpenGL 4.3 program would record it, with the bindings that do not change between frames left out:

```
glUseProgram(simulate);
glUniform1f(dtLocation, dt);
glDispatchCompute((particleCount + 255) / 256, 1, 1);
glMemoryBarrier(GL_VERTEX_ATTRIB_ARRAY_BARRIER_BIT);

const DrawElementsIndirectCommand reset{cubeIndexCount, 0, 0, 0, 0};
glBindBuffer(GL_DRAW_INDIRECT_BUFFER, drawCommand);
glBufferSubData(GL_DRAW_INDIRECT_BUFFER, 0, sizeof(reset), &reset);
glUseProgram(cull);
glUniform4fv(planesLocation, 6, planes);
glDispatchCompute((cubeCount + 255) / 256, 1, 1);
glMemoryBarrier(GL_COMMAND_BARRIER_BIT | GL_SHADER_STORAGE_BARRIER_BIT);

glBindFramebuffer(GL_FRAMEBUFFER, hdrFramebuffer);
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
glUseProgram(drawCubes);
glBindVertexArray(cubeVertexArray);
glDrawElementsIndirect(GL_TRIANGLES, GL_UNSIGNED_SHORT, nullptr);
glUseProgram(drawPoints);
glBindVertexArray(particleVertexArray);
glDrawArrays(GL_POINTS, 0, particleCount);

glUseProgram(post);
glBindImageTexture(0, hdrImage, 0, GL_FALSE, 0, GL_READ_ONLY, GL_RGBA16F);
glBindImageTexture(1, displayImage, 0, GL_FALSE, 0, GL_WRITE_ONLY, GL_RGBA8);
glDispatchCompute((width + 15) / 16, (height + 15) / 16, 1);
glMemoryBarrier(GL_FRAMEBUFFER_BARRIER_BIT);
```

It has three memory barriers, the last of which belongs to the blit that follows the frame. Chapter 4.4's frame records eleven barriers before that point, and only three of them (the one after the simulation and the two after the cull) correspond to `glMemoryBarrier` calls. The other eight cover hazards that OpenGL's

driver handles without being asked: the next frame's simulation overwriting particles this frame drew, a write after read; the reset of the draw command after its last indirect read, and before the cull's atomics; the cull's writes to the visible list after the last frame's vertex shaders read it; the draw into the HDR image, and the post-process's reads of it; and every layout transition. A port that translates only the `glMemoryBarrier` calls will pass its tests on some GPUs and corrupt frames on others. Synchronisation validation reports the barriers that such a port is missing.

> KEY IDEA
> Port behind an interface, with both renderers alive and their images compared at each step. Translate each call by what it meant, using the catalogues, and then add the barriers that OpenGL's driver inferred.

> COMING FROM OPENGL
> A large port need not happen at once. OpenGL 4.6 accepts SPIR-V through `glShaderBinary` and `glSpecializeShader`, so during a transition one GLSL source can serve both renderers, compiled by the same tools, with the differences of Table 4.5.1 kept apart by the predefined macro `VULKAN`. The extensions `GL_EXT_memory_object` and `GL_EXT_semaphore` let OpenGL use memory and semaphores that Vulkan created, so that a program can move one pass at a time to Vulkan while OpenGL draws the rest. macOS offers neither.

Reading

- The OpenGL 4.6 core profile specification, "Shader Memory Access", for exactly what `glMemoryBarrier` covers.
- The Vulkan Guide, "Decoder Ring": Vulkan's terms in the vocabulary of other APIs.
- The Khronos wiki page "Synchronization Examples" in the Vulkan-Docs repository, for the Vulkan side of each translation.
- The Mesa documentation for Zink, an OpenGL implementation on Vulkan: the same translation in the opposite direction.

Labs

LAB 4.5.1 Measure both APIs — 1 hour

Goal. Find the cost of a draw in each API on your machine.

Code. `code/u4/glport`

1. Run `VKF_VALIDATION=0 build/bin/u4_glport --frames 200 --warmup 20` with `--objects` set to 1000, 10,000 and 50,000.
2. Compute the CPU time per draw for each API, with Vulkan's recording and submission separately.
3. Run once with validation on and record what it costs.

DONE WHEN

- [ ] You have a table of costs per draw for both APIs at the three sizes.
- [ ] You can explain how your driver divides Vulkan's cost between recording and submission.

EVIDENCE FOR THE NOTEBOOK

- The table and the explanation.

LAB 4.5.2 Instancing in both — 2 hours

Goal. Measure how instancing changes the difference in CPU cost between the APIs.

Code. `code/u4/glport`

1. Draw all the cubes with one instanced draw in each API, reading each cube's matrix and colour from a buffer per instance.
2. In OpenGL 4.1, use instanced vertex attributes with `glVertexAttribDivisor`, adding the functions you need to the table. In Vulkan, use a second vertex binding with `VK_VERTEX_INPUT_RATE_INSTANCE`.
3. Measure as in Lab 4.5.1.

DONE WHEN

- [ ] Both images pass the comparison.
- [ ] You can explain how the difference between the APIs changed, from what each driver does per draw.

EVIDENCE FOR THE NOTEBOOK

- The changes and the timings.

LAB 4.5.3 Clip space without the matrix — 1 hour

Goal. Convert clip space in the viewport.

Code. `code/u4/glport`

1. Remove `vulkanFromOpenGlClip` from the Vulkan renderer's projection and run. Describe the image and the check's result.
2. Flip y with a negative viewport height instead, and run again.
3. Replace the Vulkan renderer's projection with `u4::perspective` from Chapter 4.1, without the negative viewport.

DONE WHEN

- [ ] Both corrected versions pass the comparison.
- [ ] You can explain why OpenGL's depth range did not change this image with the viewport flip alone, and which objects it would have clipped.

EVIDENCE FOR THE NOTEBOOK

- The three results and the explanation.

LAB 4.5.4 Port a compute frame — 1 hour

Goal. Find every barrier that OpenGL inferred.

Code. `code/u4/particles`

1. For each `glMemoryBarrier` in this chapter's OpenGL 4.3 frame, write the Vulkan barrier that replaces it, using Table 4.5.2.
2. List every other hazard in the frame that needs a Vulkan barrier, with its type and the two accesses.
3. Match your two lists against the eleven barriers that `u4_particles` records.

DONE WHEN

- [ ] Every barrier of `u4_particles` matches an entry in one of your lists.
- [ ] For each entry of your second list, you can say how OpenGL's driver knew about the hazard.

EVIDENCE FOR THE NOTEBOOK

- The two lists and the matching.

Problem set

1. At the costs per draw you measured in Lab 4.5.1, how many draws can each API issue in half of a 60 Hz frame, counting Vulkan's recording and submission together?
2. Why does the OpenGL renderer need a window, and the Vulkan renderer not even a surface?
3. A port's vertex shader indexes a per-instance array with the instance number, in a draw whose first instance is 100. What must change between OpenGL and Vulkan?
4. OpenGL needs no `glMemoryBarrier` between a draw that renders into a texture and a compute shader that reads it with `imageLoad`. What does Vulkan need there, and how did OpenGL's driver know?
5. The comparison allows differences of two levels, and up to 0.5% of pixels beyond that. Why not demand identical images? What would let a broken port pass if the coverage test were removed?

COMPLETION CHECKLIST

- ☐ I can port an OpenGL renderer to Vulkan behind one interface and check each step against the original image.
- ☐ I can convert between OpenGL's and Vulkan's clip spaces, by matrix or by viewport.
- ☐ I can measure the CPU cost of draws in both APIs and explain the difference.
- ☐ I can translate `glMemoryBarrier` calls into Vulkan barriers, and find the hazards that OpenGL handled silently.
- ☐ Lab 4.5.1–4.5.4 done-when criteria all hold, with evidence filed.

CHAPTER 4.6

Scaling up

TIME	6 hours
YOU BUILD	A program that records 100,000 draws on up to eight threads into secondary command buffers and checks every pixel of the result against a CPU painting; that measures pipeline creation with and without a pipeline cache, saved to disk and validated before reuse; and that reads the memory budget before and after a large allocation.
NEEDS	Chapters 4.1 to 4.5, and Chapter 1.2's sub-allocator.

WHY IT MATTERS

Costs that do not matter in a small scene become the limits of a large one, such as a city: recording every command on one thread, compiling a pipeline at the moment it is first needed, allocating memory one object at a time, and rebuilding descriptor sets each frame. Vulkan has a remedy for each of these, but the program has to use it. This chapter measures recording and pipeline creation on the course's machine, reads the memory budget, and surveys the strategies large renderers use for memory, descriptors and profiling.

SKIP TEST

1. Two threads record into command buffers allocated from the same pool. What is wrong?
2. What must a secondary command buffer know about the rendering it will run inside, and how is that expressed with dynamic rendering?
3. What does a pipeline cache store, and how does a program know that a cache file read from disk can be used?
4. Why should a program not allocate one block of device memory per buffer?

If all four are easy, read the sections on pipeline caches and on profiling, and do Labs 4.6.2 and 4.6.4.

Core ideas

Recording on many threads

A frame with many draws can take milliseconds to record. Vulkan lets several threads record at once, with one rule: a command pool, and every command buffer allocated from it, may be used by only one thread at a time. The specification calls such objects *externally synchronised*, and it is the program's job to keep them so. Each recording thread therefore has a pool of its own:

A pool and a secondary command buffer per thread — code/u4/scaling/main.cpp

```
// Command pools are externally synchronised, so each recording thread has its own.
struct Worker {
    vkf::Unique<VkCommandPool> pool;
    VkCommandBuffer secondary = VK_NULL_HANDLE;
};

Worker createWorker(const vkf::Context& ctx) {
    Worker worker{.pool = vkf::createCommandPool(ctx, ctx.mainQueue().family, 0)};
    const VkCommandBufferAllocateInfo info{
        .sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_ALLOCATE_INFO,
        .commandPool = worker.pool,
        .level = VK_COMMAND_BUFFER_LEVEL_SECONDARY,
        .commandBufferCount = 1,
    };
    VKF_CHECK(vkAllocateCommandBuffers(ctx.device(), &info, &worker.secondary));
    return worker;
}
```

The threads' work can be combined in two ways. Each thread can record a primary command buffer of its own, and the frame submits them in order. Alternatively, each thread can record a *secondary* command buffer, and one primary executes them all with `vkCmdExecuteCommands`. Secondaries are the usual way to split one rendering pass across threads. A secondary that runs inside dynamic rendering must declare the formats of the attachments it draws into, through `VkCommandBufferInheritanceRenderingInfo`, and begin with `VK_COMMAND_BUFFER_USAGE_RENDER_PASS_CONTINUE_BIT`:

Recording a share of the draws into a secondary command buffer — code/u4/scaling/main.cpp

```
void recordSecondary(VkCommandBuffer cmd, const Scene& scene,
                     std::span<const RectConstants> rects) {
    const VkFormat format = ColorFormat;
    const VkCommandBufferInheritanceRenderingInfo rendering{
        .sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_INHERITANCE_RENDERING_INFO,
        .colorAttachmentCount = 1,
        .pColorAttachmentFormats = &format,
        .rasterizationSamples = VK_SAMPLE_COUNT_1_BIT,
    };
    const VkCommandBufferInheritanceInfo inheritance{
        .sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_INHERITANCE_INFO,
        .pNext = &rendering,
    };
    const VkCommandBufferBeginInfo begin{
        .sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO,
        .flags = VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT |
                 VK_COMMAND_BUFFER_USAGE_RENDER_PASS_CONTINUE_BIT,
        .pInheritanceInfo = &inheritance,
    };
    VKF_CHECK(vkBeginCommandBuffer(cmd, &begin));
    vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_GRAPHICS, scene.pipeline);
    u4::setViewportAndScissor(cmd, scene.extent);
    for (const RectConstants& rect : rects) {
        vkCmdPushConstants(cmd, scene.layout,
                           VK_SHADER_STAGE_VERTEX_BIT | VK_SHADER_STAGE_FRAGMENT_BIT, 0,
                           sizeof(rect), &rect);
        vkCmdDraw(cmd, 6, 1, 0, 0);
    }
    VKF_CHECK(vkEndCommandBuffer(cmd));
}
```

The primary begins rendering with `VK_RENDERING_CONTENTS_SECONDARY_COMMAND_BUFFERS_BIT`, which says that the pass's commands come from secondaries, and executes them in the order given:

One primary that executes every thread's secondary — code/u4/scaling/main.cpp

```
void recordPrimary(VkCommandBuffer cmd, const Target& target,
                   std::span<const VkCommandBuffer> secondaries) {
    vkf::beginCommands(cmd);
```

```
    vkf::imageBarrier(cmd, target.color, VK_IMAGE_LAYOUT_UNDEFINED,
                      VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL, VK_PIPELINE_STAGE_2_COPY_BIT,
                      VK_ACCESS_2_NONE, VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT,
                      VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT);
    const VkRenderingAttachmentInfo color{
        .sType = VK_STRUCTURE_TYPE_RENDERING_ATTACHMENT_INFO,
        .imageView = target.color.view,
        .imageLayout = VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL,
        .loadOp = VK_ATTACHMENT_LOAD_OP_CLEAR,
        .storeOp = VK_ATTACHMENT_STORE_OP_STORE,
        .clearValue = {.color = Background},
    };
    const VkRenderingInfo rendering{
        .sType = VK_STRUCTURE_TYPE_RENDERING_INFO,
        .flags = VK_RENDERING_CONTENTS_SECONDARY_COMMAND_BUFFERS_BIT,
        .renderArea = {{0, 0}, target.extent},
        .layerCount = 1,
        .colorAttachmentCount = 1,
        .pColorAttachments = &color,
    };
    vkCmdBeginRendering(cmd, &rendering);
    vkCmdExecuteCommands(cmd, static_cast<uint32_t>(secondaries.size()), secondaries.data());
    vkCmdEndRendering(cmd);
    copyToReadback(cmd, target);
    vkf::endCommands(cmd);
}
```

The threads are started first and wait on a latch, so that the measurement covers recording alone:

Recording in parallel — code/u4/scaling/main.cpp

```
double recordInParallel(const vkf::Context& ctx, std::vector<Worker>& workers,
                        uint32_t threads, const Scene& scene) {
    const size_t share = (scene.rects.size() + threads - 1) / threads;
    std::vector<std::exception_ptr> errors(threads);
    std::latch start(1);
    std::vector<std::thread> pool;
    for (uint32_t t = 0; t < threads; ++t) {
        pool.emplace_back([&, t] {
            start.wait();
            try {
                const size_t first = std::min(scene.rects.size(), t * share);
                const size_t count = std::min(share, scene.rects.size() - first);
                VKF_CHECK(vkResetCommandPool(ctx.device(), workers[t].pool, 0));
                recordSecondary(workers[t].secondary, scene,
                                std::span(scene.rects).subspan(first, count));
            } catch (...) {
                errors[t] = std::current_exception();
            }
        });
    }
    const vkf::CpuTimer timer;
    start.count_down();
    for (std::thread& thread : pool) thread.join();
    const double ms = timer.elapsedMs();
    for (const std::exception_ptr& error : errors) {
        if (error) std::rethrow_exception(error);
    }
    return ms;
}
```

Each draw is an 8 × 8 rectangle whose corners the vertex shader makes from `gl_VertexIndex` and a push-constant block, so the draws need no vertex buffer. The rectangles cover whole pixels and their colours are exact in 8 bits, so the CPU can paint the expected image exactly, overlaps included. A secondary executed out of order would paint some overlaps the wrong way round, and every frame's image is compared with the painting.

A rectangle from six vertex indices `code/u4/scaling/rect.vert`

```
layout(push_constant) uniform Rect {
    vec4 bounds;
    vec4 color;
    vec2 targetSize;
} rect;

void main() {
    const vec2 corners[6] = vec2[](vec2(0, 0), vec2(0, 1), vec2(1, 1),
                                   vec2(0, 0), vec2(1, 1), vec2(1, 0));
    vec2 pixel = mix(rect.bounds.xy, rect.bounds.zw, corners[gl_VertexIndex]);
    gl_Position = vec4(pixel / rect.targetSize * 2.0 - 1.0, 0.0, 1.0);
}
```

100,000 draws on up to eight threads, and pipeline creation, with validation off

```
$ VKF_VALIDATION=0 ./build/bin/u4_scaling --draws 100000 --repeat 9 --pipelines 32 --fresh
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
multithreaded recording: 100000 draws into 1024x1024, median of 9 frames
  threads   record ms   speed-up   submit ms   image
        1        3.49       1.00       11.78   matches the CPU painting
        2        3.78       0.92       11.50   matches the CPU painting
        4        1.03       3.39       11.24   matches the CPU painting
        8        0.76       4.59       11.52   matches the CPU painting
pipeline cache: 32 pipeline variants, specialised differently from any earlier run
  no cache                        214.0 ms
  empty cache, new variants       174.0 ms
  the same cache, same variants     2.6 ms
  cache from disk, new variants   172.4 ms
  wrote 4465 bytes to pipeline-cache.bin; its header matches this device
memory budget (VK_EXT_memory_budget):
  before: heap 0 (device local), 32.0 GiB: budget 24.96 GiB, in use 0.009 GiB
  with 256 MB more: heap 0 (device local), 32.0 GiB: budget 24.96 GiB, in use 0.259 GiB
PASS multithreaded recording and pipeline caches
```

Recording 100,000 draws takes 3.5 ms on one thread and 0.8 ms on eight, four and a half times faster. Two threads are no faster than one here, which Lab 4.6.1 asks you to investigate on your own machine. Submission takes about 11.5 ms however many threads recorded. As Chapter 4.5 found, MoltenVK encodes the Metal commands when a command buffer is submitted, on one thread, so on this machine multithreaded recording speeds up the cheaper part of the work. A driver that does its work while commands are recorded gains far more from the same change.

> **PITFALL**
> Measure multithreaded recording with validation off. The validation layer protects its own state with locks, so threads queue for them. With validation on, recording 20,000 draws takes 20.5 ms on one thread and 154 ms on eight. The layer's thread-safety checks remain worth running during development, because they report two threads caught using one pool at the same time.

Pipeline caches

Creating a pipeline compiles its shaders for the GPU, together with every piece of state that affects the generated code. Each pipeline takes milliseconds, and a renderer with thousands of them cannot afford to compile one at the moment it first needs it. A `VkPipelineCache` lets the driver keep the results of compilation and reuse them when an identical pipeline is created again. The program creates pipelines that differ only in their specialisation constants:

Pipelines that differ only in two specialisation constants — code/u4/scaling/main.cpp

```
double createVariants(const vkf::Context& ctx, const u4::GraphicsPipelineDesc& base,
                      VkPipelineCache cache, uint32_t first, uint32_t count, float salt) {
    std::vector<vkf::Unique<VkPipeline>> pipelines;
    const vkf::CpuTimer timer;
    for (uint32_t v = first; v < first + count; ++v) {
        vkf::Specialization specialization;
        specialization.set(0, int32_t(1 + v % 8)).set(1, float(v) * 0.37f + salt);
        u4::GraphicsPipelineDesc desc = base;
        desc.specialization = specialization.info();
        desc.cache = cache;
        pipelines.push_back(u4::createGraphicsPipeline(ctx, desc));
    }
    return timer.elapsedMs();
}
```

Without a cache, with an empty cache, and with the same cache again — code/u4/scaling/main.cpp

```
const double none =
    createVariants(ctx, variantDesc, VK_NULL_HANDLE, 0, variants, salt);
const vkf::Unique<VkPipelineCache> cache = createCache(ctx, {});
const double first = createVariants(ctx, variantDesc, cache, variants, variants, salt);
const double again = createVariants(ctx, variantDesc, cache, variants, variants, salt);
```

A cache's contents can be saved with `vkGetPipelineCacheData` and passed back as initial data in a later run. They are useful only to the same driver on the same device, and a header says which made them: `VkPipelineCacheHeaderVersionOne` holds the vendor, the device and a `pipelineCacheUUID` that changes whenever compiled code would. Drivers ignore data that does not match, but checking first lets the program delete a stale file instead of keeping it:

Creating, checking and saving a pipeline cache — code/u4/scaling/main.cpp

```
vkf::Unique<VkPipelineCache> createCache(const vkf::Context& ctx,
                                         const std::vector<char>& data) {
    const VkPipelineCacheCreateInfo info{
        .sType = VK_STRUCTURE_TYPE_PIPELINE_CACHE_CREATE_INFO,
        .initialDataSize = data.size(),
        .pInitialData = data.empty() ? nullptr : data.data(),
    };
    VkPipelineCache cache = VK_NULL_HANDLE;
    VKF_CHECK(vkCreatePipelineCache(ctx.device(), &info, nullptr, &cache));
    return {ctx.device(), cache};
}

// A cache made by another device or driver is useless; its header says which made it.
bool cacheMatchesDevice(const vkf::Context& ctx, const std::vector<char>& data) {
    VkPipelineCacheHeaderVersionOne header{};
    if (data.size() < sizeof(header)) return false;
    std::memcpy(&header, data.data(), sizeof(header));
    const VkPhysicalDeviceProperties& device = ctx.properties().core;
    return header.headerVersion == VK_PIPELINE_CACHE_HEADER_VERSION_ONE &&
           header.vendorID == device.vendorID && header.deviceID == device.deviceID &&
           std::memcmp(header.pipelineCacheUUID, device.pipelineCacheUUID, VK_UUID_SIZE) == 0;
}

std::vector<char> cacheData(const vkf::Context& ctx, VkPipelineCache cache) {
    size_t size = 0;
    VKF_CHECK(vkGetPipelineCacheData(ctx.device(), cache, &size, nullptr));
    std::vector<char> data(size);
    VKF_CHECK(vkGetPipelineCacheData(ctx.device(), cache, &size, data.data()));
    data.resize(size);
    return data;
}
```

On the M2 Pro, 32 new pipelines take 170 to 215 ms to create, 5 to 7 ms each, with or without an empty cache. Created again from the same cache, they take 2.6 ms. A cache loaded from disk does not help with pipelines it has never seen.

What a cache holds depends on the driver. MoltenVK's is 4,465 bytes however many pipelines it has seen. It keeps the translation of SPIR-V into Metal's shading language, while Metal keeps compiled code in a system-wide cache of its own. That system cache makes a second run fast even without a pipeline cache, so `--fresh` salts the specialisation constants. Without the salt, a supposedly cold creation would find its code already compiled. Desktop drivers keep similar caches on disk, so every measurement of cold creation must make sure that nothing has compiled those pipelines before.

Large renderers combine a cache with other strategies. They create pipelines on background threads before they are needed; `VK_EXT_graphics_pipeline_library` links pipelines from separately compiled parts; and `VK_EXT_shader_object` replaces pipelines with shaders whose state is set at draw time, as OpenGL's was.

Memory at scale

Chapter 1.2 gave the reasons to sub-allocate: allocations are slow, their number is limited, and they come in coarse units. A large renderer adds two concerns. Its resources are created and destroyed constantly, so its allocator must free and reuse space without fragmenting its blocks. The memory it may use is also shared with other programs, so it must know how much it may take.

The Vulkan Memory Allocator, AMD's open-source library, is the usual answer to the first. It sub-allocates with several strategies, honours `bufferImageGranularity` and alignment, gives a resource memory of its own when the driver prefers that through `VkMemoryDedicatedRequirements`, and can defragment. The course writes its own allocators to show what such a library does. Most programs should use one.

The second is answered by `VK_EXT_memory_budget`, which reports for each heap a *budget*, how much this process can use without problems, and how much it is using. Both change while the program runs, as other programs allocate and free.

Reading the budget of every heap — code/u4/scaling/main.cpp

```
void printBudget(const vkf::Context& ctx, const char* when) {
    VkPhysicalDeviceMemoryBudgetPropertiesEXT budget{
        .sType = VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MEMORY_BUDGET_PROPERTIES_EXT,
    };
    VkPhysicalDeviceMemoryProperties2 memory{
        .sType = VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MEMORY_PROPERTIES_2,
        .pNext = &budget,
    };
    vkGetPhysicalDeviceMemoryProperties2(ctx.physicalDevice(), &memory);
    for (uint32_t h = 0; h < memory.memoryProperties.memoryHeapCount; ++h) {
        const VkMemoryHeap& heap = memory.memoryProperties.memoryHeaps[h];
        vkf::print("  {}: heap {}{}, {:.1f} GiB: budget {:.2f} GiB, in use {:.3f} GiB\n", when,
                   h, heap.flags & VK_MEMORY_HEAP_DEVICE_LOCAL_BIT ? " (device local)" : "",
                   double(heap.size) / (1 << 30), double(budget.heapBudget[h]) / (1 << 30),
                   double(budget.heapUsage[h]) / (1 << 30));
    }
}
```

On the M2 Pro, the single heap of 32 GiB of unified memory offers a budget of 25 GiB, and allocating 256 MiB raises the usage by 0.25 GiB. A program near its budget should release caches or lower its resolution. Past the budget, allocations may fail with `VK_ERROR_OUT_OF_DEVICE_MEMORY`, or succeed and be paged out. `VK_EXT_memory_priority` lets a program say which allocations matter most when the driver must choose.

Descriptors at scale

Descriptor strategies follow from how often resources change. Table 4.6.1 lists the common ones, and most renderers combine several.

Strategy	When it fits
Sets by frequency of change	Set 0 for the frame, set 1 for the material, set 2 for the object, each bound only when it changes (Chapter 4.3)
Push constants	Small values that change with every draw, such as a matrix or an index (Chapters 4.1 and 4.5)
Bindless arrays	Every texture in one large array, indexed in the shader, with descriptor indexing (Chapter 4.3)
Buffer device addresses	Buffers reached by address, with no buffer descriptors at all (Chapter 2.6)
Pools reset per frame	Sets allocated afresh each frame, freed together by `vkResetDescriptorPool`
`VK_EXT_descriptor_buffer`	Descriptors written into buffers the program manages, without pools or sets

Table 4.6.1. Descriptor strategies

Profiling a frame

Optimising a frame starts with finding its limit. If the GPU idles between frames, the CPU is the limit, and the place to look is recording, submission and waiting, each timed on the CPU as the programs of Chapters 4.5 and 4.6 do. If the CPU waits for the GPU, timestamps around each pass, as in Chapter 4.4, show where the GPU's time goes. Pipeline statistics queries count the invocations of each shader stage, where the feature `pipelineStatisticsQuery` exists. MoltenVK lacks it.

Below that level, vendors' tools read the hardware's counters: NVIDIA Nsight Graphics, AMD's Radeon GPU Profiler, Arm Performance Studio and Qualcomm's Snapdragon Profiler. RenderDoc captures a frame on Windows, Linux and Android and replays it call by call, and on macOS, Xcode's Metal debugger can capture the Metal commands that MoltenVK generates. Before trusting any measurement, confirm that validation is off, that the GPU's clocks are not idling between frames, and that nothing cached the work from an earlier run.

> KEY IDEA
> A large renderer records on many threads, each with a pool of its own, creates its pipelines before it needs them and keeps them in a cache, and sub-allocates memory within the budget. Measure each with validation off, and learn what your driver does behind each call before trusting a number.

> COMING FROM OPENGL
> An OpenGL context belongs to one thread at a time, so commands are issued from one thread, and drivers parallelise internally where they can. `glGetProgramBinary` is OpenGL's pipeline cache, and a binary can be rejected after any driver update, leaving the program to compile again. OpenGL leaves memory to the driver, and there is no portable way to ask it how much is left, only vendor extensions such as `GL_NVX_gpu_memory_info`.

Reading

- Arseny Kapoulkine, "Writing an efficient Vulkan renderer", in *GPU Zen 2*, 2019: memory, descriptors, command buffers and pipelines in a production renderer.
- NVIDIA, "Vulkan Dos and Don'ts", on the developer blog.
- AMD GPUOpen, the documentation of the Vulkan Memory Allocator, in particular its sections on budgets and on choosing memory types.
- The Vulkan Guide, "Pipeline Cache".

Labs

LAB 4.6.1 Scale the recording — 1 hour

Goal. Measure how recording scales on your machine.
Code. `code/u4/scaling`

1. Run `VKF_VALIDATION=0 build/bin/u4_scaling --draws 100000 --repeat 9` three times and record the recording and submission times.
2. Add your machine's number of cores to the thread counts the program tries, and run again.

DONE WHEN

- [ ] You have a table of recording speed-up against threads.
- [ ] You can explain where the speed-up stops growing, why submission does not change, and whether two threads beat one on your machine.

EVIDENCE FOR THE NOTEBOOK

- The table and the explanation.

LAB 4.6.2 A pipeline cache across runs

1.5 hours

Goal. Measure pipeline creation from a cache that an earlier run saved.

Code. `code/u4/scaling`

1. Change the test of the cache from disk to create the same variants as the empty-cache test, which the previous run saved.
2. Run the program twice without `--fresh`, then twice with it.

DONE WHEN

- [ ] The second run without `--fresh` creates its variants from the disk cache faster than without a cache.
- [ ] You can explain, from what MoltenVK's cache and Metal's system cache each hold, every number of the four runs, or the equivalent for your driver.

EVIDENCE FOR THE NOTEBOOK

- The four outputs and the explanation.

LAB 4.6.3 Account for memory

1.5 hours

Goal. Reconcile the program's allocations with the budget.

Code. `code/u4/scaling`

1. Count the bytes the program allocates through `vkf::createBuffer` and `vkf::createImage`, with each allocation's size from its memory requirements.
2. Print the count with the heap's usage before and after the program's allocations.

DONE WHEN

- [ ] The change in the heap's usage matches your count to within 1%.
- [ ] You can explain any difference, from the allocation granularity and from memory the driver allocates for itself.

EVIDENCE FOR THE NOTEBOOK

- The changes and the output.

LAB 4.6.4 Profile a frame

1 hour

Goal. Compare a profiler's view with your own timestamps.

Code. `code/u4/particles`

1. Capture a frame of `u4_particles --window` with the frame debugger for your platform: RenderDoc on Windows or Linux, or Xcode's Metal debugger on macOS.

2. Find the time of each pass in the capture and compare it with the program's timestamps.

DONE WHEN

- [ ] You have the capture and the comparison.
- [ ] You can name the most expensive pass and explain any difference between the tool's times and the timestamps.

EVIDENCE FOR THE NOTEBOOK

- A screenshot of the capture and the comparison.

Problem set

1. Recording 100,000 draws takes 3.5 ms on one thread and 0.8 ms on eight, and submitting them 11.5 ms. What frame time can the CPU sustain, and which change would shorten it most?
2. Why does each recording thread need its own command pool? What could go wrong if two threads shared one, even if each used its own command buffer?
3. What does `VkCommandBufferInheritanceRenderingInfo` let the driver do when it records a secondary, and what would it have to do without it?
4. A pipeline cache saved before a driver update is loaded after it. What happens with and without the header check, and what does the program lose in each case?
5. A device's `maxMemoryAllocationCount` is 4,096, and a scene has 20,000 meshes, each with a vertex and an index buffer. Design the allocation scheme, and say how it handles meshes that are loaded and unloaded as the camera moves.

COMPLETION CHECKLIST

- [ ] I can record one rendering pass on many threads with secondary command buffers and a pool per thread.
- [ ] I can cache pipelines, save the cache, and check its header before reuse.
- [ ] I can sub-allocate memory and keep a program within its memory budget.
- [ ] I can choose a descriptor strategy by how often resources change.
- [ ] I can find whether the CPU or the GPU limits a frame, and profile the one that does.
- [ ] Lab 4.6.1–4.6.4 done-when criteria all hold, with evidence filed.

CHAPTER 4.7

Capstone: a GPU-driven sandbox

TIME	10 hours
YOU BUILD	One program that combines the parts of the course. A million particles are simulated on an async compute queue, with a copy of their state for each frame in flight; 50,000 cubes are culled on the GPU and drawn with an indirect draw; the high-dynamic-range image is tone-mapped in compute and presented to a window or run headless. Every frame is a render graph from Chapter 3.8, synchronisation validation stays silent, every pass is timed, and the simulation, the culling and the image are checked against the CPU. The chapter ends with the course's final project.
NEEDS	The whole course, in particular Chapters 3.6, 3.8, 4.2 and 4.4.

WHY IT MATTERS

Each unit built a part of a modern renderer on its own. Real programs fail at the seams between the parts: a buffer shared by two queues, a swapchain image that arrives with a semaphore of its own, a frame in flight that still reads what the next frame writes. This chapter assembles the parts into one program, organises each frame as a render graph, and measures where the time goes. In the final project, you extend the program with a feature of your own, to the same standard of evidence as everything before it.

SKIP TEST

1. The simulation of frame $n + 1$ runs while frame n is still drawing its particles. What must be true of the buffers they use?
2. How does an acquired swapchain image enter a render graph, and what must its first barrier wait for?
3. Each frame slot compiles its own graph. What changes between two executions of the same graph, and how?
4. Async compute barely changes the frame time here. What would you measure to decide whether to keep it?

Even if all four are easy, read the sections on state across frames and on measurement, and do the final project.

Core ideas

The frame

The sandbox renders Chapter 4.4's scene: a disc of a million particles orbiting a centre among 50,000 cubes. Each frame updates its parameters, simulates the particles, culls the cubes, draws the cubes and the particles into a 16-bit floating-point image, and tone-maps that image in a compute shader. With a window, the result is blitted into the swapchain image and presented. The simulation runs on an async compute queue, and the frame waits for it only where the particles are drawn (Figure 4.7.1).

The frame as a render graph

Every frame is a graph of seven passes, nine with a window. The resources fall into three groups. The frame's parameters and its colour and depth images are transients, created by the graph. The particle state, the cubes, the list of visible cubes and the draw command are imported, because they outlive a frame. The display image is imported too, with the use the previous frame's last pass left it in:

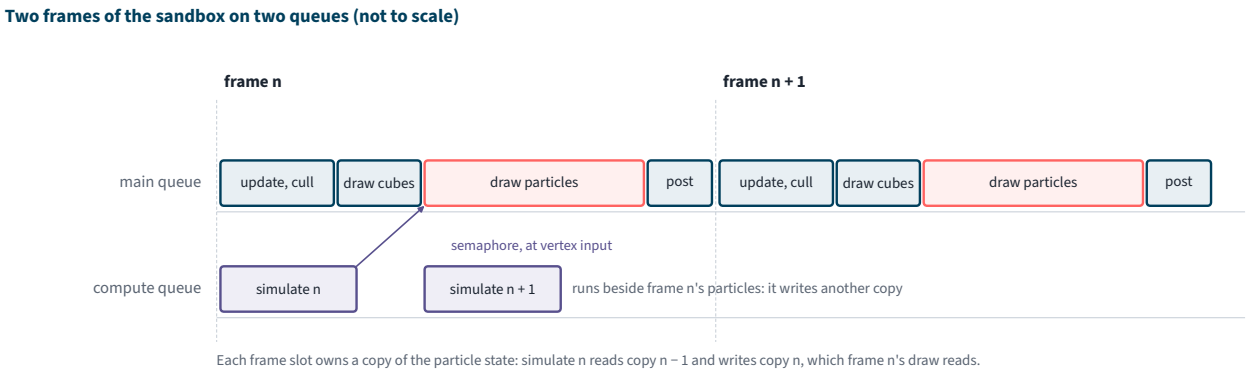


Figure 4.7.1. The passes of two frames on the two queues. The next frame's simulation can run while this frame draws its particles, because it writes another copy of the state.

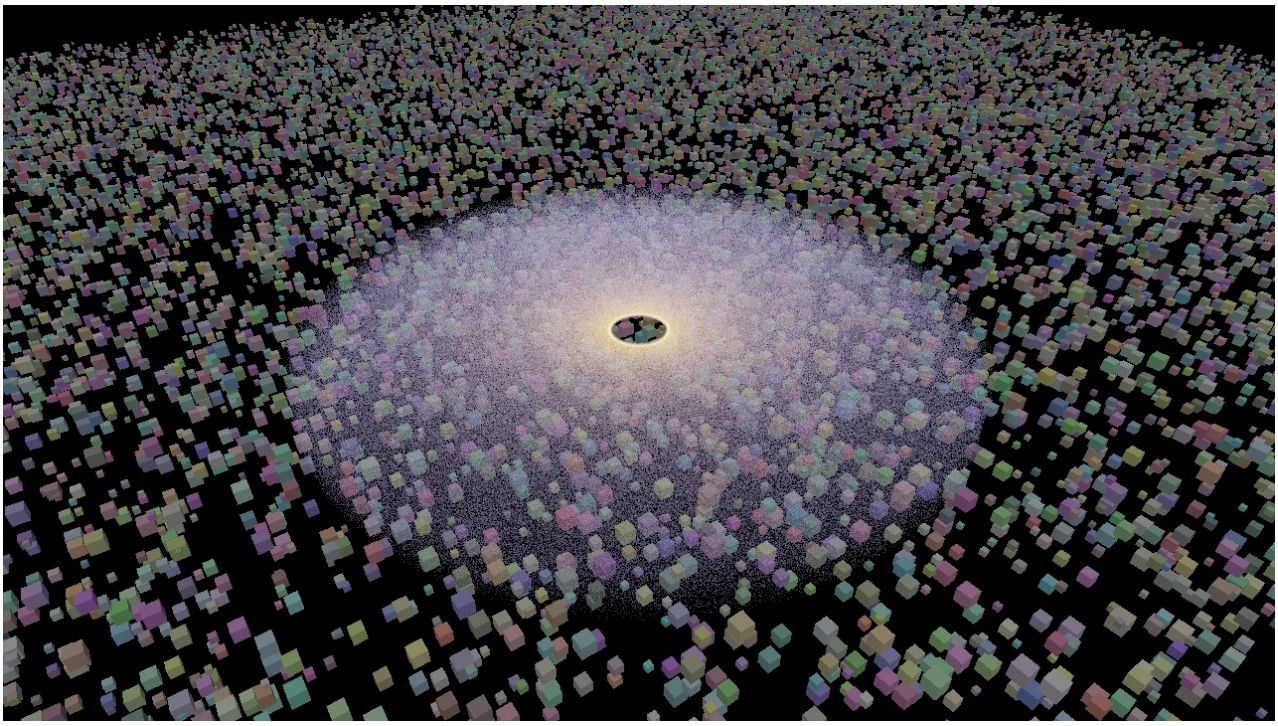


Figure 4.7.2. The sandbox at the end of a run, as the program wrote it.

The frame's resources — code/u4/sandbox/main.cpp

```
std::unique_ptr<rg::Graph> buildFrameGraph(const vkf::Context& ctx, const World& w,
                                           const Pipelines& p, Slot& slot,
                                           const FrameParams& params) {
    auto graph = std::make_unique<rg::Graph>(ctx);
    rg::Graph& g = *graph;
    const rg::Resource frame =
        g.createBuffer("frame params", sizeof(FrameParams),
                       VK_BUFFER_USAGE_STORAGE_BUFFER_BIT | VK_BUFFER_USAGE_TRANSFER_DST_BIT);
    const uint32_t slots = static_cast<uint32_t>(w.particles.size());
    const rg::Resource previous =
        g.importBuffer("previous particles",
                       w.particles[(slot.index + slots - 1) % slots].buffer, rg::ComputeWrite);
    const rg::Resource particles =
        g.importBuffer("particles", w.particles[slot.index].buffer, rg::ComputeRead);
    const rg::Resource instances = g.importBuffer("instances", w.instances);
    const rg::Resource visible = g.importBuffer("visible", slot.visible, VertexStorageRead);
    const rg::Resource command =
        g.importBuffer("draw command", slot.command, rg::IndirectRead);
    const rg::Resource hdr = g.createImage(
        "hdr", {.format = HdrFormat,
```

```
                .width = w.extent.width,
                .height = w.extent.height,
                .usage = VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT | VK_IMAGE_USAGE_STORAGE_BIT});
    const rg::Resource depth =
        g.createImage("depth", {.format = w.depthFormat,
                                .width = w.extent.width,
                                .height = w.extent.height,
                                .usage = VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT,
                                .aspect = VK_IMAGE_ASPECT_DEPTH_BIT});
    const rg::Resource display =
        g.importImage("display", w.display, w.display.view, vkf::colorRange(),
                      w.present ? BlitRead : rg::ComputeWrite);
```

Three uses are new: `vkCmdUpdateBuffer` writes in the clear stage, the cull's results are read by vertex shaders through storage buffers, and the swapchain image arrives from the presentation engine. Each is one `rg::Use`, declared once:

Uses that Chapter 3.8's presets did not need — code/u4/sandbox/main.cpp

```
// vkCmdUpdateBuffer executes in the CLEAR stage.
constexpr rg::Use BufferUpdate{VK_PIPELINE_STAGE_2_CLEAR_BIT, VK_ACCESS_2_TRANSFER_WRITE_BIT};
constexpr rg::Use VertexStorageRead{VK_PIPELINE_STAGE_2_VERTEX_SHADER_BIT,
                                    VK_ACCESS_2_SHADER_STORAGE_READ_BIT};
constexpr rg::Use DepthTest{
    VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT | VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT,
    VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_READ_BIT,
    VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL};

constexpr rg::Use BlitRead{VK_PIPELINE_STAGE_2_BLIT_BIT, VK_ACCESS_2_TRANSFER_READ_BIT,
                           VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL};
constexpr rg::Use BlitWrite{VK_PIPELINE_STAGE_2_BLIT_BIT, VK_ACCESS_2_TRANSFER_WRITE_BIT,
                            VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL};
// A freshly acquired image: its first barrier must wait at the stage of the acquire semaphore.
constexpr rg::Use Acquired{VK_PIPELINE_STAGE_2_BLIT_BIT, VK_ACCESS_2_NONE,
                           VK_IMAGE_LAYOUT_UNDEFINED};
```

The compute work comes first. `update` writes the frame's parameters, `simulate` advances the particles on the compute queue, and the cull resets and fills the draw command and the visible list exactly as in Chapter 4.4:

Updating, simulating and culling — code/u4/sandbox/main.cpp

```
g.addPass("update", [&g, frame, &params](VkCommandBuffer cmd) {
     vkCmdUpdateBuffer(cmd, g.buffer(frame), 0, sizeof(params), &params);
 }).write(frame, BufferUpdate);
g.addPass("simulate",
          [&w, &p, &slot](VkCommandBuffer cmd) {
              const SimulationConstants constants{Gravity.dt, Gravity.strength,
                                                  Gravity.softening, w.particleCount};
              vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, p.simulate);
              vkCmdBindDescriptorSets(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, p.layout, 0, 1,
                                      &slot.set, 0, nullptr);
              vkCmdPushConstants(cmd, p.layout, VK_SHADER_STAGE_COMPUTE_BIT, 0,
                                 sizeof(constants), &constants);
              vkCmdDispatch(cmd, vkf::groupCount(w.particleCount, 256), 1, 1);
          })
    .read(previous, rg::ComputeRead)
    .write(particles, rg::ComputeWrite)
    .queue(rg::Queue::Compute);
g.addPass("reset command", [&g, command, &w](VkCommandBuffer cmd) {
     const VkDrawIndexedIndirectCommand reset{w.cube.indexCount, 0, 0, 0, 0};
     vkCmdUpdateBuffer(cmd, g.buffer(command), 0, sizeof(reset), &reset);
 }).write(command, BufferUpdate);
g.addPass("cull",
          [&w, &p, &slot](VkCommandBuffer cmd) {
              vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, p.cull);
              vkCmdBindDescriptorSets(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, p.layout, 0, 1,
                                      &slot.set, 0, nullptr);
              vkCmdDispatch(cmd, vkf::groupCount(w.instanceCount, 256), 1, 1);
```

```
        })
    .read(frame, rg::ComputeRead)
    .read(instances, rg::ComputeRead)
    .read(command, rg::ComputeRead)
    .write(command, rg::ComputeWrite)
    .write(visible, rg::ComputeWrite);
```

The cubes are drawn with the indirect command the cull wrote, then the particles from the buffer the simulation wrote, added into the same image. The post-process is the frame's side effect:

Drawing the visible cubes — code/u4/sandbox/main.cpp

```
g.addPass("draw cubes",
          [&g, &w, &p, &slot, hdr, depth, command](VkCommandBuffer cmd) {
              beginDraw(cmd, g.view(hdr), g.view(depth), w.extent, true);
              vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_GRAPHICS, p.cubes);
              vkCmdBindDescriptorSets(cmd, VK_PIPELINE_BIND_POINT_GRAPHICS, p.layout, 0, 1,
                                      &slot.set, 0, nullptr);
              u4::bindMesh(cmd, w.cube);
              vkCmdDrawIndexedIndirect(cmd, g.buffer(command), 0, 1,
                                       sizeof(VkDrawIndexedIndirectCommand));
              vkCmdEndRendering(cmd);
          })
    .read(frame, VertexStorageRead)
    .read(instances, VertexStorageRead)
    .read(visible, VertexStorageRead)
    .read(command, rg::IndirectRead)
    .write(hdr, rg::ColorAttachment)
    .write(depth, rg::DepthAttachment);
```

Drawing the particles and tone-mapping the image — code/u4/sandbox/main.cpp

```
g.addPass("draw particles",
          [&g, &w, &p, &slot, hdr, depth](VkCommandBuffer cmd) {
              beginDraw(cmd, g.view(hdr), g.view(depth), w.extent, false);
              vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_GRAPHICS, p.particles);
              vkCmdBindDescriptorSets(cmd, VK_PIPELINE_BIND_POINT_GRAPHICS, p.layout, 0, 1,
                                      &slot.set, 0, nullptr);
              const VkDeviceSize offset = 0;
              vkCmdBindVertexBuffers(cmd, 0, 1, w.particles[slot.index].buffer.ptr(),
                                     &offset);
              vkCmdDraw(cmd, w.particleCount, 1, 0, 0);
              vkCmdEndRendering(cmd);
          })
    .read(frame, VertexStorageRead)
    .read(particles, rg::VertexInput)
    .read(hdr, rg::ColorAttachment)
    .write(hdr, rg::ColorAttachment)
    .read(depth, DepthTest);
g.addPass("post-process",
          [&w, &p, &slot](VkCommandBuffer cmd) {
              vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, p.post);
              vkCmdBindDescriptorSets(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, p.layout, 0, 1,
                                      &slot.set, 0, nullptr);
              vkCmdDispatch(cmd, vkf::groupCount(w.extent.width, 16),
                            vkf::groupCount(w.extent.height, 16), 1);
          })
    .read(hdr, rg::ComputeRead)
    .write(display, rg::ComputeWrite)
    .sideEffect();
```

The graph derives the barriers that Chapter 4.4 wrote by hand, and splits the frame into three batches:

The frame's plan, with validation on

```
$ ./build/bin/u4_sandbox --plan
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
```

```
compiled: 7 passes, 7 kept, culled: none
transient memory: 11264 KiB with aliasing, 11264 KiB without
  frame params buffer      164 bytes at 0        passes 0-5
  hdr      image   7536640 bytes at 256      passes 4-6
  depth    image   3997696 bytes at 7536896  passes 4-5
batch 0 on the main queue: update reset command cull draw cubes
batch 1 on the compute queue: simulate
batch 2 on the main queue: draw particles post-process; waits for batch 1 at
    VERTEX_ATTRIBUTE_INPUT
pass 0 update
pass 1 simulate
  memory previous particles, particles
    src COMPUTE_SHADER   SHADER_STORAGE_WRITE
    dst COMPUTE_SHADER   SHADER_STORAGE_READ|SHADER_STORAGE_WRITE
...
pass 4 draw cubes
  memory draw command
    src COMPUTE_SHADER   SHADER_STORAGE_WRITE
    dst DRAW_INDIRECT    INDIRECT_COMMAND_READ
...
pass 5 draw particles
  memory hdr
    src COLOR_ATTACHMENT_OUTPUT COLOR_ATTACHMENT_WRITE
    dst COLOR_ATTACHMENT_OUTPUT COLOR_ATTACHMENT_READ|COLOR_ATTACHMENT_WRITE
pass 6 post-process
  image  hdr COLOR_ATTACHMENT_OPTIMAL → GENERAL
    src COLOR_ATTACHMENT_OUTPUT COLOR_ATTACHMENT_WRITE
    dst COMPUTE_SHADER   SHADER_STORAGE_READ
...
PASS GPU-driven sandbox
```

State across frames

With three frames in flight, frame $n + 1$'s simulation may start while frame n is still drawing the particles it simulated. If both used one buffer, the simulation would overwrite positions the draw was reading. Each frame slot therefore owns one copy of the particle state: the simulation of frame n reads copy $n - 1$ and writes copy n, which frame n's draw reads.

The previous frame's state in, this frame's state out — code/u4/sandbox/simulate.comp

```
// The previous frame's state in, this frame's state out: each frame slot owns one copy.
layout(std430, set = 0, binding = 1) readonly buffer Previous {
    Particle previous[];
};

layout(std430, set = 0, binding = 7) writeonly buffer Next {
    Particle next[];
};

layout(push_constant) uniform Simulation {
    float dt;
    float strength;
    float softening;
    uint count;
} sim;
```

The copies are used by both queue families, so they are created with `CONCURRENT` sharing, which `vkf::createBuffer` does not offer:

A buffer both queue families may use without ownership transfers — code/u4/sandbox/main.cpp

```
// Both queue families use the particles: CONCURRENT buffers need no ownership transfers.
struct SharedBuffer {
    vkf::Unique<VkDeviceMemory> memory;
    vkf::Unique<VkBuffer> buffer;
};

SharedBuffer createSharedBuffer(const vkf::Context& ctx, VkDeviceSize size,
```

```
                                VkBufferUsageFlags usage) {
    const uint32_t families[] = {ctx.mainQueue().family, ctx.computeQueue().family};
    const bool concurrent = families[0] != families[1];
    const VkBufferCreateInfo info{
        .sType = VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO,
        .size = size,
        .usage = usage,
        .sharingMode = concurrent ? VK_SHARING_MODE_CONCURRENT : VK_SHARING_MODE_EXCLUSIVE,
        .queueFamilyIndexCount = concurrent ? 2u : 0u,
        .pQueueFamilyIndices = families,
    };
    VkBuffer buffer = VK_NULL_HANDLE;
    VKF_CHECK(vkCreateBuffer(ctx.device(), &info, nullptr, &buffer));
    VkMemoryRequirements requirements;
    vkGetBufferMemoryRequirements(ctx.device(), buffer, &requirements);
    const VkMemoryAllocateInfo allocate{
        .sType = VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO,
        .allocationSize = requirements.size,
        .memoryTypeIndex = vkf::findMemoryType(ctx, requirements.memoryTypeBits,
                                               VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT),
    };
    VkDeviceMemory memory = VK_NULL_HANDLE;
    VKF_CHECK(vkAllocateMemory(ctx.device(), &allocate, nullptr, &memory));
    VKF_CHECK(vkBindBufferMemory(ctx.device(), buffer, memory, 0));
    ctx.name(buffer, "particles");
    return {{ctx.device(), memory}, {ctx.device(), buffer}};
}
```

Two dependencies between frames cross the graph's boundary. The first is declared: the imported previous copy's earlier use is the previous frame's simulation, so the graph orders this frame's read after that write, on the compute queue where both run. The second comes from frame pacing. The copy that frame n writes was last drawn by frame $n - 3$, on the main queue, and the only thing that orders the new write after that draw is the frame loop's wait for frame $n - 3$ to finish before reusing its slot. If the number of frames in flight changes and the number of copies does not, the program has a race.

Presentation

With a window, two more passes join the graph. The swapchain image is imported once, without an image, and every frame `rebind` points it at the image just acquired. Its declared earlier use, `Acquired`, names the stage at which the frame waits for the acquire semaphore, so that the graph's first barrier on the image chains after that wait, as Chapter 4.2 required:

Blitting to the swapchain and presenting — code/u4/sandbox/main.cpp

```
slot.swapchainImage = g.importImage("swapchain image", VK_NULL_HANDLE, VK_NULL_HANDLE,
                                    vkf::colorRange(), Acquired);
g.addPass(
     "blit to swapchain",
     [&g, &w, display, target = slot.swapchainImage](VkCommandBuffer cmd) {
         const VkImageBlit blit{
             .srcSubresource = {VK_IMAGE_ASPECT_COLOR_BIT, 0, 0, 1},
             .srcOffsets = {{0, 0, 0},
                            {int32_t(w.extent.width), int32_t(w.extent.height), 1}},
             .dstSubresource = {VK_IMAGE_ASPECT_COLOR_BIT, 0, 0, 1},
             .dstOffsets = {{0, 0, 0},
                            {int32_t(w.presentExtent.width),
                             int32_t(w.presentExtent.height), 1}},
         };
         vkCmdBlitImage(cmd, g.image(display), VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL,
                        g.image(target), VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL, 1,
                        &blit, VK_FILTER_LINEAR);
     })
    .read(display, BlitRead)
    .write(slot.swapchainImage, BlitWrite);
g.addPass("present").read(slot.swapchainImage, rg::Present).sideEffect();
```

The frame loop paces the slots with one timeline semaphore, as in Chapter 3.6, and hands the graph the acquire semaphore to wait on and the semaphores to signal:

The frame loop, offscreen and with a window — code/u4/sandbox/main.cpp

```
const vkf::Unique<VkSemaphore> timeline = vkf::createTimelineSemaphore(ctx, 0);
std::vector<std::vector<std::pair<std::string, double>>> passTimes;
std::vector<double> frameMs;
vkf::CpuTimer frameTimer;
uint64_t frame = 0;
for (; (frames == 0 || frame < frames) && !(display && display->closeRequested());
     ++frame) {
    Slot& slot = slots[frame % framesInFlight];
    if (frame >= framesInFlight) {
        waitFor(ctx, timeline, frame + 1 - framesInFlight);
        passTimes.push_back(slot.graph->passTimes());
    }
    params.viewProjection = cameraAt(frame, extent);
    const auto planes = u4::frustumPlanes(params.viewProjection);
    std::copy(planes.begin(), planes.end(), params.planes);
    const vkf::SemaphoreSubmit done{timeline, frame + 1,
                                    VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT};
    if (!presenter) {
        slot.graph->submit({}, std::span(&done, 1));
    } else {
        display->pollEvents();
        const auto image = presenter->acquire(frame % framesInFlight);
        if (!image) break;
        slot.graph->rebind(slot.swapchainImage, VK_NULL_HANDLE, image->image,
                           image->view);
        w.presentExtent = image->extent;
        const vkf::SemaphoreSubmit acquired{image->acquired, 0,
                                            VK_PIPELINE_STAGE_2_BLIT_BIT};
        const vkf::SemaphoreSubmit signals[] = {
            {image->ready, 0, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT}, done};
        slot.graph->submit(std::span(&acquired, 1), signals);
        presenter->present(*image);
    }
    frameMs.push_back(frameTimer.elapsedMs());
    frameTimer.restart();
}
waitFor(ctx, timeline, frame);
```

Checking the whole

A program this size needs checks at its seams. After the last frame, the program copies back the particles, the draw command and the visible list, and the image. Every particle must still be bound, each particle's angular momentum must be unchanged, and a sample of orbits must match the CPU's integration of the same steps:

Checking the simulation after 120 frames — code/u4/sandbox/main.cpp

```
particleReadback.invalidate();
const Particle* gpu = particleReadback.data<Particle>();
uint32_t bound = 0;
double worstMomentum = 0;
for (uint32_t i = 0; i < particleCount; ++i) {
    const Vec3 r{gpu[i].position[0], gpu[i].position[1], gpu[i].position[2]};
    if (std::isfinite(u4::length(r)) && u4::length(r) < 50) ++bound;
    const Vec3 before = u4::angularMomentum(initial[i]);
    const Vec3 after = u4::angularMomentum(gpu[i]);
    worstMomentum = std::max(worstMomentum, double(u4::length(after - before)) /
                                            double(u4::length(before)));
}
double worstPosition = 0;
for (uint32_t i = 0; i < particleCount; i += std::max(1u, particleCount / 1000)) {
    Particle reference = initial[i];
    for (uint64_t step = 0; step < ran; ++step) u4::stepOnCpu(reference, Gravity);
    const Vec3 a{gpu[i].position[0], gpu[i].position[1], gpu[i].position[2]};
    const Vec3 b{reference.position[0], reference.position[1], reference.position[2]};
```

```
    worstPosition = std::max(worstPosition, double(u4::length(a - b)));
}
const bool particlesOk =
    bound == particleCount && worstMomentum < 1e-3 && worstPosition < 1e-3;
vkf::print(
    "particles: {} of {} still bound after {} steps; angular momentum kept to "
    "{:.1e}\n",
    bound, particleCount, ran, worstMomentum);
vkf::print("  sampled orbits within {:.1e} of the CPU's\n", worstPosition);
```

The last frame's cull must match a cull on the CPU with the same planes, allowing disagreement only for cubes that touch a plane, and the image must contain both the cubes and the bright disc of particles. Running with validation on, the frame loop's 120 graph executions must produce no message at all.

A run with validation on: every check passes, and the layer is silent

```
$ ./build/bin/u4_sandbox
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
sandbox: 1000000 particles, 50000 cubes, 1280x720, 3 frames in flight
simulation on an async compute queue, family 1
120 frames: frame time median 2.59 ms (386 frames per second), mean 2.59 ms
GPU time per pass, median of 120 frames:
  update               0.004 ms
  simulate             0.346 ms  (compute queue)
  reset command        0.004 ms
  cull                 0.013 ms
  draw cubes           0.149 ms
  draw particles       2.027 ms
  post-process         0.055 ms
particles: 1000000 of 1000000 still bound after 120 steps; angular momentum kept to 5.5e-06
  sampled orbits within 2.5e-07 of the CPU's
culling, last frame: 14325 cubes visible on the GPU, 14325 on the CPU
  0 disagree (0 of them touching a frustum plane)
image: 63% of pixels lit, 23.1% bright (the particle disc)
wrote sandbox.png
PASS GPU-driven sandbox
```

Where the time goes

Six hundred frames with async compute, with validation off

```
$ VKF_VALIDATION=0 ./build/bin/u4_sandbox --frames 600
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
sandbox: 1000000 particles, 50000 cubes, 1280x720, 3 frames in flight
simulation on an async compute queue, family 1
600 frames: frame time median 2.49 ms (401 frames per second), mean 2.62 ms
GPU time per pass, median of 600 frames:
  update               0.004 ms
  simulate             0.343 ms  (compute queue)
  reset command        0.004 ms
  cull                 0.013 ms
  draw cubes           0.149 ms
  draw particles       1.995 ms
  post-process         0.050 ms
particles: 1000000 of 1000000 still bound after 600 steps; angular momentum kept to 1.4e-05
  sampled orbits within 8.6e-06 of the CPU's
culling, last frame: 14150 cubes visible on the GPU, 14150 on the CPU
  0 disagree (0 of them touching a frustum plane)
image: 63% of pixels lit, 22.9% bright (the particle disc)
wrote sandbox.png
PASS GPU-driven sandbox
```

The same on one queue

```
$ VKF_VALIDATION=0 ./build/bin/u4_sandbox --frames 600 --no-async
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
sandbox: 1000000 particles, 50000 cubes, 1280x720, 3 frames in flight
simulation on the main queue
600 frames: frame time median 2.54 ms (394 frames per second), mean 2.58 ms
GPU time per pass, median of 600 frames:
  update               0.004 ms
  simulate             0.337 ms
  reset command        0.004 ms
  cull                 0.014 ms
  draw cubes           0.357 ms
  draw particles       1.765 ms
  post-process         0.039 ms
particles: 1000000 of 1000000 still bound after 600 steps; angular momentum kept to 1.4e-05
  sampled orbits within 8.6e-06 of the CPU's
culling, last frame: 14150 cubes visible on the GPU, 14150 on the CPU
  0 disagree (0 of them touching a frustum plane)
image: 63% of pixels lit, 22.9% bright (the particle disc)
wrote sandbox.png
PASS GPU-driven sandbox
```

A frame takes about 2.5 ms either way, with async compute ahead by 2%, less than frames vary from run to run. The particle draw dominates the frame, and it slows from 1.77 to 2.00 ms when the simulation runs beside it, which cancels what the overlap saves. The simulation moves 64 MB a step, reading and writing a million 32-byte particles, close to the bandwidth limit that Chapter 2.5 measured, and the particle draw reads the same 32 MB of state. The two therefore compete for the GPU. As Chapter 3.5 found, a second queue helps when one workload leaves idle the resources that the other needs. A simulation limited by arithmetic, such as the N-body forces of Chapter 2.6, would be a better partner for this draw.

The main queue's passes add up to 2.2 ms of the 2.5 ms frame. The CPU, which records seven small passes per frame from graphs compiled once per slot, is not the limit.

KEY IDEA

A real frame consists of many passes joined by synchronisation. Organise it as a graph so that the joins are derived, give everything a frame writes a copy per frame in flight, check the result at every seam, and measure before believing that a technique, async compute included, pays on your hardware.

COMING FROM OPENGL

An OpenGL 4.3 version of this frame is possible on desktop platforms, with compute shaders, indirect draws and `glMemoryBarrier`, and Chapter 4.5 sketched it. It cannot express the second queue, the copies of state per frame in flight (which the driver would otherwise create behind the program's back), or a frame graph that sees every pass before any of them runs.

Reading

- Graham Wihlidal, "Optimizing the Graphics Pipeline with Compute", GDC 2016, on culling and GPU-driven rendering.
- Ulrich Haar and Sebastian Aaltonen, "GPU-Driven Rendering Pipelines", SIGGRAPH 2015 course "Advances in Real-Time Rendering in Games".
- Hans-Kristian Arntzen, "Render graphs and Vulkan: a deep dive", 2017, again, now with a whole frame in mind.

Labs

LAB 4.7.1 Run every configuration — 1 hour

Goal. Learn how the program behaves before you change it.

Code. `code/u4/sandbox`

1. Run `u4_sandbox --plan`, then with `--window`, then `--headless --frames 600` with validation off, with and without `--no-async`.
2. Compare your timings with this chapter's, pass by pass.

DONE WHEN

- [ ] Every run passes its checks, and the validation runs are silent.
- [ ] You can explain each difference between your GPU's timings and the M2 Pro's.

EVIDENCE FOR THE NOTEBOOK

- The outputs and the comparison.

LAB 4.7.2 Move the cull — 1.5 hours

Goal. Judge a change of queue by measurement.

Code. `code/u4/sandbox`

1. Move `cull` to the compute queue, and print the new plan.
2. Measure the frame and the passes with validation off.

DONE WHEN

- [ ] The program passes its checks with validation silent.
- [ ] You can explain the new batches and semaphores, and whether the move paid.

EVIDENCE FOR THE NOTEBOOK

- The plan and the measurements.

LAB 4.7.3 A glow pass — 2 hours

Goal. Add a pass to a graph you did not write.

Code. `code/u4/sandbox`

1. Add a compute pass between the particles and the post-process that adds a blurred copy of the bright parts of the image, through a new transient image.
2. Check the result against a CPU version on a small image, and measure the pass.

DONE WHEN

- [ ] The check passes and validation is silent.
- [ ] You can say which barriers the graph added for your pass, and why each is needed.

EVIDENCE FOR THE NOTEBOOK

- The changes, the check and the plan.

LAB 4.7.4 The final project — 5 hours

Goal. Extend the sandbox with a feature of your own, to the course's standard.

Code. `code/u4/sandbox`

1. Choose a feature, for example: N-body attraction between a subset of the particles with indirect dispatch (Chapter 2.6); two levels of detail, with distant cubes drawn as points by a second indirect draw that the cull fills; occlusion culling against the previous frame's depth; or particles that collide with the cubes.
2. Declare it in the frame graph, and give it a check against the CPU.
3. Measure its cost per frame and its effect on the other passes.
4. Write a one-page report: what you built, how you checked it, what it costs, and what you would change.

DONE WHEN

- [ ] The program runs for 600 frames with every check passing, including yours, and synchronisation validation silent.
- [ ] Your report's measurements can be reproduced from the commands it gives.

EVIDENCE FOR THE NOTEBOOK

- The code, the outputs and the report.

Problem set

1. Why does the particle draw slow down when the simulation runs beside it? What does that tell you about async compute for this frame?
2. The sandbox keeps one copy of the particle state per frame in flight. Could two copies serve three frames in flight? What would have to change?
3. What would go wrong if one compiled graph were shared by every frame slot?
4. The main queue's passes add up to about 2.2 ms, and the frame takes 2.5 ms. Where could the rest go, and how would you find out?
5. Which pass would you optimise first, and how would you check that the optimisation kept the image correct?

COMPLETION CHECKLIST

- [ ] I can organise a whole frame, with compute, graphics and presentation, as a render graph.
- [ ] I can keep state across frames in flight and across queues without races.
- [ ] I can check a large GPU program at its seams as well as at its output.
- [ ] I can measure a frame pass by pass and decide where to spend effort.
- [ ] Lab 4.7.1–4.7.4 done-when criteria all hold, with evidence filed.

UNIT 4

Sign-off

A unit is complete when every chapter's completion checklist is ticked, its evidence is in your notebook and repository, and you can do each review task below without notes. Date each line as you finish it. If someone who writes GPU code for a living will review your work, ask them to sign the last line.

	Chapter	Checklist ticked	Evidence filed	Date
4.1	The graphics pipeline	☐	☐	
4.2	Presentation	☐	☐	
4.3	Drawing with data	☐	☐	
4.4	Compute meets graphics	☐	☐	
4.5	From OpenGL to Vulkan	☐	☐	
4.6	Scaling up	☐	☐	
4.7	Capstone: a GPU-driven sandbox	☐	☐	

UNIT REVIEW

Each task takes an hour or less if the unit has done its work. Do them aloud or in writing, from memory, then check against your notebook and code.

- ☐ Trace one vertex of the textured scene from its buffer to a pixel on screen, naming every stage, transformation and image layout on the way.
- ☐ Explain how the swapchain, the acquire and ready semaphores and frames in flight fit together, and why the ready semaphore belongs to the image.
- ☐ Explain GPU culling with an indirect draw, including the barriers after the cull, and how the course checked it against the CPU.
- ☐ List the differences between OpenGL and Vulkan that change results in a port, and the hazards OpenGL's driver handled that a port must add.
- ☐ Present your final project's report: what it measures, how it was checked, and what you would change.

Signed: student _____ Date _____

Reviewed by (optional) _____ Date _____